The Coding Interview Playbook

The Coding Interview Playbook

120 Original Problems with Tested Python Solutions, Patterns, System Design and Mock Interviews to Prepare for Top Tech Company Interviews

Aadhavan Muthurengan

The Coding Interview Playbook
120 Original Problems with Tested Python Solutions, Patterns, System Design and Mock Interviews to Prepare for Top Tech Company Interviews

Copyright © 2026 Aadhavan Muthurengan. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and certain other noncommercial uses permitted by copyright law. Readers may freely use and adapt the example code in this book for their own study, practice and work.

Disclaimer. This book is provided for educational and informational purposes only. AI products, features, and policies change frequently; verify current details with each provider. The author makes no guarantees about results and is not liable for any losses arising from the use of this information. Nothing in this book is legal, medical, financial, or other professional advice.

Trademarks. Google, Meta, Amazon, Apple, Microsoft and SpaceX are trademarks of their respective owners; their names are used only to describe publicly stated or publicly reported hiring practices. Java is a trademark of Oracle and/or its affiliates. Python, Redis, Kafka and other product names are trademarks of their respective owners. This book is independent and is not affiliated with, sponsored by, or endorsed by any company it mentions. No problem in this book is claimed to have been asked by any particular company.

Examples. Every problem in this book is original. Every solution was run and checked on Python 3.12, 3.13, 3.14 and 3.15. Interview formats described were checked against companies' own published guidance where available, as of October 2026, and change over time.

First edition, 2026.

To my father and my mother,
for your love, your sacrifices, and every lesson you taught me.

To my wife,
for your patience, your encouragement, and your belief in me.

And to our three children,
may you always build the things you dream of.

This book is for you.

Contents

Introduction 9

Part I: Getting Ready 13

1. How Top Tech Companies Interview 15
2. A Six-Step Method for Any Coding Problem 23
3. Big-O Without Fear 29
4. Python for Interviews 35

Part II: The Patterns 41

5. Arrays and Hashing 43
6. Two Pointers 51
7. Sliding Window 59
8. Prefix Sums 67
9. Stacks and Queues 75
10. Binary Search 83
11. Linked Lists 91
12. Trees and Binary Search Trees 99
13. Heaps and Top-K 109
14. Intervals and Greedy Choices 115
15. Backtracking 123
16. Graphs: BFS and DFS 131

17. Advanced Graphs 139
18. Dynamic Programming I: One Dimension 147
19. Dynamic Programming II: Grids, Strings and Knapsacks 157
20. Tries and String Techniques 167
21. Bit Manipulation and Math 177
22. Designing Data Structures 185

Part III: Beyond Algorithms **195**

23. Practical Coding Rounds 197
24. Object-Oriented Design 209
25. System Design Fundamentals 217
26. Behavioral Interviews 225
27. AI-Assisted Rounds, Take-Homes and Online Assessments 231

Part IV: Practice **237**

28. Four Full Mock Interviews 239
29. Your Study Plan 251

Appendix A: Pattern Cheat Sheet **257**

Appendix B: Complexity Reference **261**

Appendix C: From Python to Java and C++ **265**

Appendix D: Problem Index **271**

Appendix E: Glossary **277**

About the Author **281**

Introduction

Every year, hundreds of thousands of people apply for software engineering jobs at the world's best-known technology companies. Most of them will face the same kind of test: a problem they have never seen, a shared editor or whiteboard, about 45 minutes, and an engineer watching how they think. Some candidates freeze. Others solve the problem and still don't get an offer, because they never explained what they were doing. A smaller group walks in with a plan, recognizes the shape of the problem, talks through their reasoning, writes clean code, tests it, and improves it when asked.

This book is about joining that last group.

What Makes This Book Different

Every problem is original. The 120 problems in this book were written for it. They cover the classic ideas every interviewer draws on, such as hashing, two pointers, graphs and dynamic programming, but with new stories, new data and new wording. You won't find them copied from a website, and you won't find claims that "Company X asked this exact question". Nobody can honestly promise that, and memorizing a list of past questions is not how strong candidates prepare.

Every solution is tested. Each problem is a real Python file. Every solution in the book was run on Python 3.12, 3.13, 3.14 and 3.15 against its examples, and then against hundreds of random inputs, compared with a slower brute-force solution that is easy to trust. The examples you see printed were produced by running the code, so the output on the page is what the program really returns. Along the way, those random tests caught real bugs in early

drafts. A few of those bugs appear in this book as **Common Mistake** boxes, because they are exactly the bugs you might write under pressure.

You learn patterns, not answers. Most interview problems are variations on about twenty ideas. Part II teaches those ideas one chapter at a time, each with a short template you can adapt, the signals that tell you the pattern applies, and six problems from easy to hard.

It covers the whole interview. Coding rounds are only part of the process. Part III covers practical coding rounds, object-oriented design, system design, behavioral interviews, AI-assisted interviews and take-home assessments. Part IV puts everything together in four complete mock interviews and a study plan.

How the Book Is Organized

- **Part I, Getting Ready:** how big technology companies interview today, a six-step method for any coding problem, Big-O analysis without fear, and the parts of Python that matter most in interviews.
- **Part II, The Patterns:** eighteen chapters, from arrays and hashing to designing your own data structures, with 109 problems.
- **Part III, Beyond Algorithms:** practical, "real-world" coding rounds, object-oriented design, system design, behavioral questions, and new formats such as AI-assisted interviews.
- **Part IV, Practice:** four full mock interviews, with the conversation between candidate and interviewer, and a study plan for two, six or twelve weeks.
- **Appendices:** a pattern cheat sheet, a complexity reference, a guide to writing the same solutions in Java and C++, an index of every problem, and a glossary.

How Each Problem Is Presented

Every problem follows the same structure, so you always know where to look:

1. **The problem**, with a short story, the exact task and the rules.

2. **Examples**, shown as an interactive Python session.
3. **Think first**, a hint to try before reading on.
4. **The brute-force idea**, because in a real interview, a correct slow solution is the starting point.
5. **The better idea**, explained in plain words.
6. **The solution**, the tested code.
7. **Complexity**, the time and space it needs.
8. **Common Mistake** and **Follow-Up** boxes: what goes wrong, and what an interviewer might ask next.

> **Tip:** Don't just read the solutions. Cover the page, set a timer for 25 minutes, and solve each problem yourself first: out loud, as if someone were listening. Then compare. Reading a solution feels like learning; solving a problem is learning.

The Code

All the code is in Python, the language most candidates choose for interviews because it is short and readable. If your interview will be in Java or C++, read the solutions for the ideas, and use Appendix C to translate the Python tools into their equivalents. Each solution is written the way you would write it in an interview: clear names, short comments where they help, and no clever tricks that you couldn't explain.

A few helper pieces are shared across chapters: a `ListNode` class for linked lists, a `TreeNode` class for trees, and small functions that build them from Python lists. They are introduced where they are first used. The examples for linked-list and tree problems call a small wrapper that builds the structure from a list, runs the solution, and turns the answer back into a list, so that you can read the input and output at a glance.

A Note on Companies and Dates

This book refers to Google, Meta, Amazon, Apple, Microsoft, SpaceX and other companies when describing how interviews work. Where a company

publishes its own guidance for candidates, the book relies on it and says so. Where information comes only from candidates' reports, the book says that too. Interview formats change, especially as AI tools spread, so always confirm the details with your recruiter. The descriptions in this book reflect what was published as of **October 2026**.

Let's start with what actually happens in a technical interview at a large technology company.

PART I

Getting Ready

CHAPTER 1

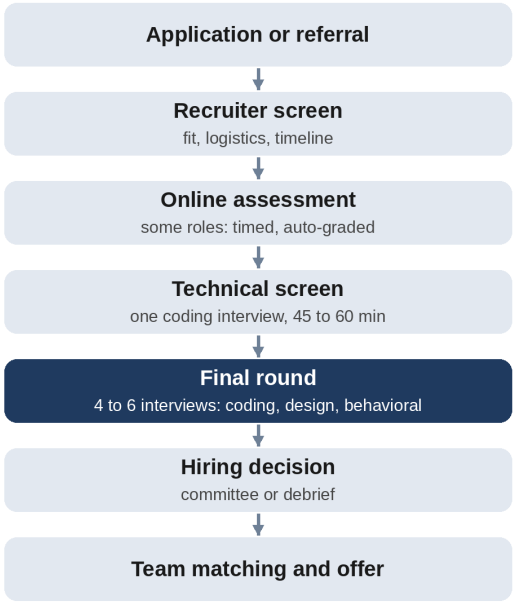
How Top Tech Companies Interview

Before you practice a single problem, it helps to know what you are practicing for. Large technology companies differ in the details, but their software engineering interviews share a common shape, and they look for the same handful of qualities. This chapter describes that shape, what each company says it is looking for, and how interviews have changed with the spread of AI tools.

The Typical Pipeline

For most software engineering roles at large technology companies, the process looks something like this:

1. **Application or referral.** A referral from a current employee usually gets your résumé read sooner, but it doesn't skip any interviews.
2. **Recruiter screen.** A 20- to 30-minute call about your background, the role and the process. Ask your recruiter what the rounds will be and how to prepare; they want you to succeed.
3. **Online assessment.** Some companies, especially for new graduates and interns, send a timed set of coding problems to solve on your own.
4. **Technical screen.** One or two interviews of about 45 to 60 minutes, by video, with an engineer and a shared code editor. Usually one or two coding problems.



The typical interview pipeline: application or referral, recruiter screen, an online assessment for some roles, a technical screen, a final round of four to six interviews, a hiring decision by a committee or debrief, then team matching and an offer.

5. **The final round, or "onsite" loop.** Four to six interviews, often on one day, in person or by video. For most software roles: two or three coding interviews; a system design interview for mid-level and senior candidates; and a behavioral interview about how you work with others.
6. **The decision.** Each interviewer writes feedback independently. At some companies a hiring committee of people who did not interview you makes the decision; at others, the interviewers and hiring manager meet in a "debrief".
7. **Team matching and offer.** Some companies hire into a specific team; others hire first and match you with a team afterward.

The whole process typically takes from three weeks to two months.

What Interviewers Are Looking For

Every company has its own rubric, but four qualities appear again and again in what companies publish for candidates:

- **Problem solving.** Can you take a vague problem, ask good questions, find an approach, and improve it?
- **Coding.** Can you turn the idea into correct, readable code, in a reasonable time, without an IDE doing the work for you?
- **Verification.** Do you test your own code, find your own bugs and think about edge cases?
- **Communication.** Can the interviewer follow your thinking? Do you explain trade-offs, and do you take hints well?

Notice that only one of the four is "getting the right answer". Meta's guidance for candidates says this directly: the way you solve problems can be as important as the answer, and a working but non-optimal solution beats fragments of an optimal one that you didn't finish. Google's long-standing advice to candidates makes the same points: think out loud, ask clarifying questions, state your assumptions and discuss trade-offs.

> **Interviewer's View:** An interviewer has to write down evidence for every rating they give. "The candidate explained why a hash map makes this O(n), tested the empty list without being asked, and found their own off-by-one bug" is strong evidence. A silent candidate who types a correct answer leaves the interviewer with almost nothing to write.

What the Companies Say

Companies publish a surprising amount of advice for candidates. The table summarizes the main points, with the type of source; always check the current version of each company's pages and ask your recruiter.

Company	Key points	Source
Meta	Review core data structures and big-O; practice without autocomplete and under a time limit; talk through your approach. Select roles now include an authorized AI assistant in some technical interviews, with a practice environment provided in advance	Meta careers site: interview preparation pages and hiring process page
Amazon	Behavioral questions are based on Amazon's 16 Leadership Principles; answer with the STAR method; for software roles, roughly half the process is technical; a "Bar Raiser" from outside the hiring team helps protect the hiring standard	Amazon's official interview guides on aboutamazon.com and amazon.jobs
Google	Think out loud, ask clarifying questions, state assumptions, discuss trade-offs; decisions are made by a hiring committee; in 2025 Google said it was bringing back at least one in-person interview round for some roles, partly because of concerns about AI-assisted cheating in virtual interviews	Google's candidate guide; reported remarks by Sundar Pichai in 2025
Apple	Expect open-ended questions about your past work; bring specific examples; ask about the team and how success is measured	Apple careers interview tips
Microsoft	Recruiters single out fundamental coding skills as one of the hardest parts of the interview; know the technical skills the role requires	Microsoft's careers blog
SpaceX	Candidates commonly report a strong emphasis on C++ for many software roles, timed or take-home coding tests, and practical, engineering-flavored problems such as parsing data or modeling state machines	Candidate reports (no official guide found); treat as anecdotal

Two lessons stand out. First, the core skills are the same everywhere: data structures, algorithms, clean code and clear communication. Second, each

company adds its own emphasis. Amazon weights behavioral questions heavily, hardware and aerospace companies often care more about C++ and systems knowledge, and some companies now test how well you work *with* an AI assistant.

Coding Interviews in the Age of AI

AI coding tools have changed interviews in two opposite directions.

- **More in-person rounds.** Because a candidate in a video interview could secretly paste a problem into an AI assistant, several large companies have brought back in-person interviews for some rounds. Google's chief executive Sundar Pichai said in 2025 that the company would add at least one in-person round for some roles.
- **AI-assisted rounds.** Other companies have gone the other way, and made AI part of the test. Meta says that select roles now include an authorized AI assistant in some technical interviews. Candidates who have taken these rounds describe a longer, multi-step task in a small codebase, where they are judged on how well they guide the assistant, check its output and explain the code. Chapter 27 covers how to prepare for these formats.

Either way, the fundamentals matter more, not less. In an in-person round, you need them because there is no assistant. In an AI-assisted round, you need them to tell whether the assistant's code is correct.

New Graduates, Experienced Engineers and Senior Roles

Expectations scale with the level of the role:

- **Interns and new graduates** mostly face coding interviews, often with an online assessment first, plus a short behavioral conversation.
- **Mid-level engineers** add a system design interview, and are expected to code faster and more cleanly.
- **Senior and staff engineers** face harder system design, deeper behavioral questions about leadership and conflict, and sometimes a project

deep-dive about past work. Coding interviews are still there, and still matter.

How to Use This Book

- **If your interview is in two weeks or less**, read Chapters 2 to 4, then do the first three problems of each pattern chapter, and one mock interview. Chapter 29 has a two-week plan.
- **If you have six weeks**, work through Part II in order, solving every problem before reading the solution, then Part III and the mock interviews.
- **If you are aiming at senior roles**, give extra time to Chapters 24 to 26 on design and behavioral interviews.

Key Takeaways

- Most software interviews at large technology companies follow the same pipeline: recruiter screen, technical screen, a final loop of four to six interviews, and a collective decision.
- Interviewers assess problem solving, coding, verification and communication, not just the final answer.
- Companies publish useful guidance; use it, and ask your recruiter about the exact format.
- AI has led to both more in-person rounds and new AI-assisted rounds. In both, strong fundamentals are what count.

Sources and Further Reading

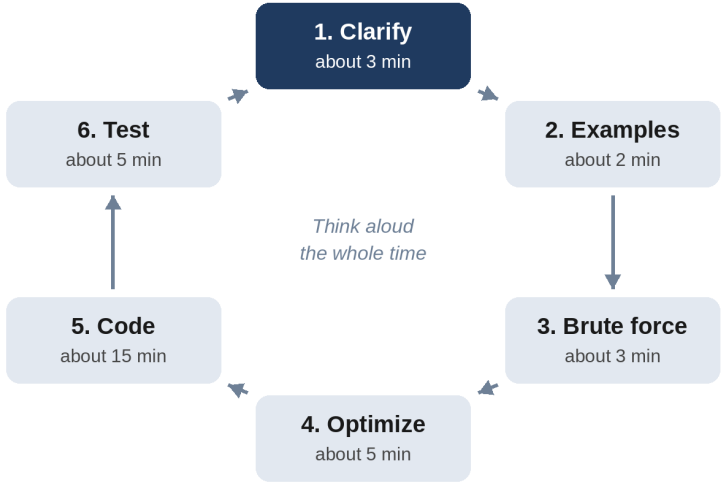
- Meta Careers, "Preparing for your software engineering interview at Meta"; "Preparing for your technical screen"; "Preparing for your full loop interview"; "Hiring process". metacareers.com
- Amazon, "Your complete guide to the Amazon interview process"; "Exploring Amazon's unique interview process" (the Bar Raiser program); "What's it like to interview at Amazon?". aboutamazon.com; amazon.jobs/content/en/how-we-hire

- Google, "Preparing for Google technical internship interviews" (candidate guide). services.google.com
- Reports of Sundar Pichai's remarks on in-person interviews, 2025, for example CXO Digital Pulse, "Google Reinstates In-Person Job Interviews Amid Rising AI Cheating Concerns".
- Apple Careers, "Interview tips". apple.com/careers/us/interview_tips.html
- Microsoft, "How to land a job at Microsoft". news.microsoft.com/life

CHAPTER 2

A Six-Step Method for Any Coding Problem

Strong candidates don't solve problems faster because they are smarter. They solve them faster because they never wonder what to do next. They follow a routine, the same one every time, and it carries them through problems they have never seen. This chapter gives you that routine: six steps you can use for every coding problem in this book and in your interviews.



The six-step method: Clarify, Examples, Brute force, Optimize, Code, Test, with a rough time budget for a 45-minute interview.

Step 1: Clarify

Interview problems are often deliberately vague. Before you think about a solution, make sure you are solving the right problem. Restate it in your own words, then ask about:

- **Inputs:** What types? Can the list be empty? Can numbers be negative, or repeated? How large can the input be?
- **Outputs:** Exactly what should be returned? What if there is no answer: an empty list, -1, an exception?
- **Rules:** Can I modify the input? Is the input sorted? Are there several valid answers, and if so, does it matter which?

The size of the input is the most useful question of all, because it tells you how fast your solution must be. Chapter 3 explains how to turn "up to 100,000 items" into "I need roughly O(n log n) or better".

> **Tip:** Write the answers to your questions at the top of the editor as a comment. It shows the interviewer you are organized, and stops you forgetting a rule halfway through.

Step 2: Examples

Work through one or two small examples by hand, including at least one tricky case. Examples do three jobs: they confirm that you understood the problem, they often reveal the solution's key idea, and they become your test cases at the end.

Good examples are small (four to six items) but not trivial, and they include an edge case: an empty input, a single item, all items equal, or no valid answer.

Step 3: Brute Force

State the simplest correct solution, even if it is slow, and give its complexity: "I could try every pair, which is $O(n^2)$." You usually don't need to code it. Saying it out loud proves that you can solve the problem at all, gives you a fallback, and gives you something to improve.

Step 4: Optimize

Now look for a better approach. Ask yourself:

- **What work does the brute force repeat?** Can a hash map, a running total or a sorted order remember it instead?
- **Which pattern does this look like?** The signals listed at the start of each chapter in Part II are designed for exactly this moment: "longest consecutive run" suggests a sliding window; "sorted" suggests two pointers or binary search; "all combinations" suggests backtracking; "fewest steps" suggests breadth-first search.
- **What is the best possible complexity?** If you must read every item, you can't do better than O(n). If your current solution is O(n), you may already be done.

Before you write any code, describe your approach and its complexity to the interviewer and ask whether it sounds reasonable. This is the cheapest moment to catch a wrong idea.

Step 5: Code

Write the code in clear, small pieces, explaining as you go. Use descriptive names, such as `seen` and `need` rather than `d` and `x`. If a part is fiddly, write a helper function with a clear name and fill it in later. Don't optimize prematurely, and don't stay silent for minutes at a time.

Step 6: Test

Don't announce "I'm done" the moment you stop typing. Walk through your code with the small example from Step 2, line by line, tracking the variables. Then check the edge cases: empty input, one item, duplicates, the largest values. Fix any bug you find calmly; finding your own bugs is a strong positive signal. Finally, state the time and space complexity.

The Method in Action

Here is how the six steps sound in practice on Problem 5.1, *Budget Pair*: given a list of prices and a budget, find two different items whose prices add up to exactly the budget.

> **Candidate:** So I'm given a list of prices and a target budget, and I return the positions of two different items that sum exactly to the budget. Can the list be empty? And if there's no pair, what should I return? **Interviewer:** It can be empty. Return an empty list if there's no pair. **Candidate:** If several pairs work, does it matter which one I return? And roughly how long can the list be? **Interviewer:** Any pair is fine. Up to a hundred thousand prices.

The candidate writes the rules as a comment, then tries an example: prices `[12, 7, 30, 18]`, budget 25. Seven and eighteen make 25, so the answer is `[1, 3]`.

> **Candidate:** The simplest approach is to try every pair, which is $O(n^2)$. With a hundred thousand prices that's five billion pairs, too slow. But for each price, I know exactly which other price I need: the budget minus this price. If I remember the prices I've already seen in a hash map, I can check for the one I need in O(1), so the whole thing is O(n) time and O(n) extra space. Does that sound good? **Interviewer:** Go ahead.

The candidate writes the solution (it appears in Chapter 5), then traces the example: at 12, need 13, not seen; at 7, need 18, not seen; at 30, need -5; at 18, need 7, seen at index 1, so return `[1, 3]`. Then the edge cases: an empty list returns `[]`; `[5, 5]` with budget 10 works because 5 is stored only after it is checked, so an item is never paired with itself.

That is the whole method. It takes discipline to follow under pressure, which is why you should practice it on every problem in this book, out loud.

Managing Your Time

A typical 45-minute coding interview leaves about 35 minutes for the problem itself. A rough budget:

Step	Minutes
Clarify and examples	5
Brute force and optimize	5 to 10
Code	15
Test and analyze	5

If you are stuck for more than a few minutes, say what you are thinking and what is blocking you. Interviewers often give hints, and taking a hint well counts in your favor.

When You Get Stuck

- **Go back to the example.** Solve it by hand, slowly, and notice what your brain is doing. That is often the algorithm.
- **Simplify the problem.** Solve it for a sorted input, or for only two items, or without the hardest constraint. Then add the constraint back.
- **Try the pattern list.** Go through Appendix A's cheat sheet in your head: could a hash map, two pointers, a sliding window, a heap, a graph search or dynamic programming help?
- **Ask.** "I'm considering two approaches, A and B; I'm leaning to A because of this. What do you think?" is a perfectly good thing to say.

Key Takeaways

- Follow six steps every time: clarify, examples, brute force, optimize, code, test.
- Ask about the input size: it tells you how fast your solution must be.
- Always state a brute-force solution and its complexity before optimizing.
- Agree on the approach before coding, explain as you write, and test your own code with your examples and edge cases.

- When stuck, think out loud; a hint taken well is not a failure.

CHAPTER 3

Big-O Without Fear

Every interviewer will ask, "What is the time and space complexity of your solution?" Big-O notation is how you answer. It sounds mathematical, but for interviews you need only a handful of ideas, and they are all about one question: **as the input grows, how fast does the work grow?**

The Idea

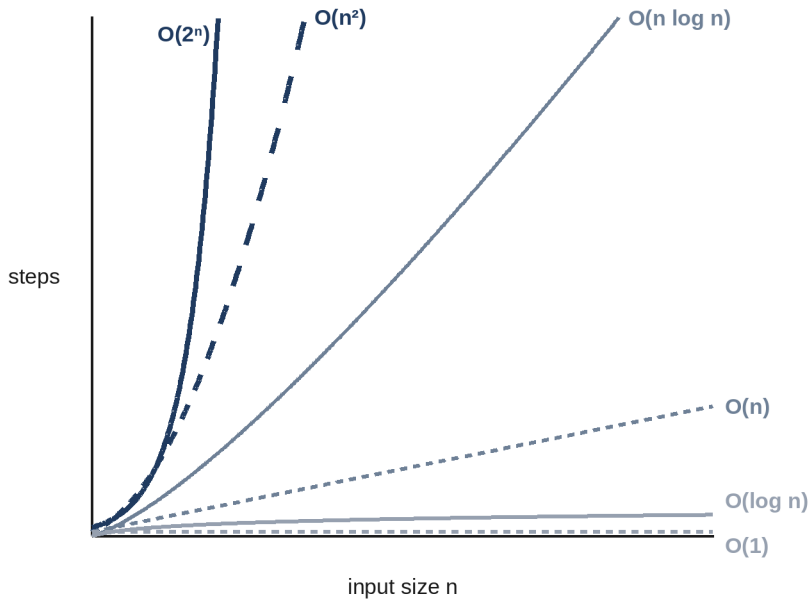
Big-O describes how the running time (or memory) of an algorithm grows with the size of its input, usually called *n*, ignoring constant factors. If doubling the input doubles the work, the algorithm is **O(n)**, "linear". If doubling the input quadruples the work, it is **$O(n^2)$**, "quadratic".

We ignore constants because they don't change the big picture. An algorithm that does 3n steps and one that does 50n steps are both O(n): for a large enough input, either will beat any $O(n^2)$ algorithm.

The Complexities You Will Meet

Big-O	Name	Typical example	Doubling n...
O(1)	Constant	Looking up a key in a hash map	changes nothing
O(log n)	Logarithmic	Binary search in a sorted list	adds one step
O(n)	Linear	One pass through a list	doubles the work

Big-O	Name	Typical example	Doubling n...
O(n log n)	Linearithmic	Sorting	slightly more than doubles it
$O(n^2)$	Quadratic	Checking every pair	quadruples it
$O(2^n)$	Exponential	Trying every subset	squares it
O(n!)	Factorial	Trying every ordering	makes it astronomically bigger



How fast different complexities grow: O(1) and O(log n) stay almost flat, O(n) and O(n log n) rise steadily, while $O(n^2)$ and $O(2^n)$ shoot upwards even for small inputs.

Reading Complexity From Code

You can usually find the complexity by looking at loops:

- A single loop over the input is **O(n)**.
- A loop inside a loop, both over the input, is **$O(n^2)$**.
- A loop that halves the remaining work each time, as binary search does, is **O(log n)**.

- Sorting first costs **O(n log n)**, which then dominates any O(n) work after it.
- Recursion that branches into two calls at each of n levels is **O(2^n)**.

Here is the brute-force solution to Problem 5.1, which checks every pair:

```
def budget_pair_brute(prices, budget):
    for i in range(len(prices)):
        for j in range(i + 1, len(prices)):
            if prices[i] + prices[j] == budget:
                return [i, j]
    return []
```

The outer loop runs n times and the inner loop up to n times, so the work is proportional to n × n: **O(n^2) time**. It uses only a few variables, so it needs **O(1) extra space**.

And here is the faster solution:

```
def budget_pair(prices, budget):
    seen = {}  # price -> index where we saw it
    for i, price in enumerate(prices):
        need = budget - price
        if need in seen:
            return [seen[need], i]
        seen[price] = i
    return []
```

One loop over the prices, with an O(1) hash-map lookup inside: **O(n) time**. The map may hold every price: **O(n) extra space**. That is the most common trade-off in interviews: use extra memory to save time.

What Counts as "Fast Enough"?

As a rough guide, Python executes on the order of ten million simple operations per second (compiled languages such as C++ manage about a hundred times more). Interview problems usually expect a solution to finish in about a second, which gives a useful rule of thumb from the input size:

Input size n	Complexity you need (roughly)
up to about 10	$O(n!)$ or $O(2^n)$ is fine: try everything
up to about 20	$O(2^n)$: all subsets
up to about 500	$O(n^3)$
up to about 5,000	$O(n^2)$
up to about 1,000,000	$O(n \log n)$ or $O(n)$
larger	$O(n)$, $O(\log n)$ or $O(1)$

That's why "how big can the input be?" is such a useful question in Step 1 of the six-step method: the answer tells you which algorithms are worth considering.

Space Complexity

Space complexity counts the **extra** memory an algorithm uses, beyond the input itself: new lists, maps, sets and the call stack. Two things are easy to forget:

- **Recursion uses memory.** Each recursive call adds a frame to the call stack, so a recursion that goes n levels deep uses O(n) space, even if it creates no lists.
- **Slices copy.** In Python, `items[1:]` creates a new list, so slicing inside a loop or a recursion can quietly add O(n) time and space per step.

Amortized and Average Cases

Some operations are usually fast but occasionally slow:

- **Appending to a Python list** is O(1) *amortized*: once in a while the list grows its storage and copies everything, but averaged over many appends each costs O(1).
- **Hash-map lookups** are O(1) *on average*: with a bad hash function or deliberately chosen keys they can degrade, but in interviews you can treat them as O(1).

When you use these operations, say "O(1) amortized" or "O(1) average"; interviewers appreciate the precision.

Several Inputs

When there are several inputs, use several letters. Merging a list of length m with a list of length n is O(m + n). Comparing every word in one list with every word in another is O(m × n). Don't collapse them into a single n unless they really are the same size.

Key Takeaways

- Big-O describes how work grows with input size, ignoring constant factors.
- Know the common classes: O(1), O(log n), O(n), O(n log n), $O(n^2)$, $O(2^n)$, O(n!).
- Loops multiply; sequential steps add, and the biggest term wins.
- Use the input size to decide how fast your solution must be.
- Count extra space too, including the recursion stack and copies made by slicing.

CHAPTER 4

Python for Interviews

Python is the most popular language for coding interviews, and for good reason: its standard library already contains most of the data structures that interview problems need, and its code is short enough to write quickly and read aloud. This chapter is a tour of the tools you will use again and again in Part II, with the operations that matter and their costs. Each snippet is a real program; the output printed below each one is what it produces.

Lists, Strings and Grids

Lists and strings support slicing, sorting and the familiar loops. Two built-ins make loops clearer: `enumerate` gives you each item's position, and `zip` walks several sequences together.

```
def demo():
    word = "Interview"
    print(word[::-1], word.lower().count("e"), "".join(sorted("cab")))
    print(ord("c") - ord("a"), chr(ord("a") + 2))

    for i, ch in enumerate("ab"):
        print(i, ch)
    print(list(zip([1, 2, 3], "xyz")))

    rows, cols = 2, 3
    grid = [[0] * cols for _ in range(rows)]  # each row is a new list
    grid[0][0] = 9
    print(grid)
    wrong = [[0] * cols] * rows  # the SAME row repeated: a classic bug
    wrong[0][0] = 9
    print(wrong)
```

```
weivretnI 2 abc
2 c
0 a
1 b
[(1, 'x'), (2, 'y'), (3, 'z')]
[[9, 0, 0], [0, 0, 0]]
[[9, 0, 0], [9, 0, 0]]
```

> **Common Mistake:** `[[0] * cols] * rows` creates one row and repeats a reference to it, so changing one cell changes every row, as the last line of output shows. Always build grids with a comprehension: `[[0] * cols for _ in range(rows)]`.

The costs to remember: indexing and appending are O(1); inserting or deleting at the front, `in` on a list, slicing and copying are O(n); sorting is O(n log n). Strings are immutable, so building a string by repeated `+=` in a loop can cost $O(n^2)$; collect pieces in a list and `"".join` them at the end.

Sets and Dictionaries

Hash-based sets and dictionaries are the single most useful tools in interviews: membership tests, insertions and lookups are all O(1) on average.

```
def demo():
    seen = {3, 1}
    seen.add(2)
    print(2 in seen, sorted(seen))
    print({1, 2, 3} & {2, 3, 4}, {1, 2} | {5}, {1, 2, 3} - {2})

    stock = {"apples": 3}
    stock["pears"] = stock.get("pears", 0) + 5
    print(stock, stock.setdefault("plums", 0), stock)
    for name, qty in sorted(stock.items()):
        print(name, qty)
```

```
True [1, 2, 3]
{2, 3} {1, 2, 5} {1, 3}
{'apples': 3, 'pears': 5, 'plums': 0} 0 {'apples': 3, 'pears': 5, 'plums':
↪ 0}
apples 3
pears 5
plums 0
```

Keys must be immutable: numbers, strings and tuples work; lists do not. To use a list as a key, convert it with `tuple(...)`, as Problem 5.3 does.

Counter and defaultdict

`Counter` counts things, and `defaultdict` creates a starting value for missing keys, which removes most "if key not in dict" checks.

```
def demo():
    tags = ["db", "api", "db", "ui", "api", "db"]
    counts = Counter(tags)
    print(counts["db"], counts["missing"])  # missing keys count as 0
    print(counts.most_common(2))

    by_length = defaultdict(list)  # a new key starts as []
    for word in ["ox", "cat", "dog", "emu", "go"]:
        by_length[len(word)].append(word)
    print(dict(by_length))
```

```
3 0
[('db', 3), ('api', 2)]
{2: ['ox', 'go'], 3: ['cat', 'dog', 'emu']}
```

deque: Fast Queues

A Python list is slow to remove from the front (O(n)), so never use `list.pop(0)` for a queue. `collections.deque` adds and removes at both ends in O(1). It is the queue for breadth-first search in Chapters 16 and 17, and with `maxlen` it keeps a fixed-size window of recent items.

```
def demo():
    queue = deque([1, 2])
    queue.append(3)  # add at the right
    queue.appendleft(0)  # add at the left
    print(queue.popleft(), queue.pop(), list(queue))

    recent = deque(maxlen=3)  # keeps only the last 3 items
    for reading in [5, 6, 7, 8]:
        recent.append(reading)
    print(list(recent))
```

```
0 3 [1, 2]
[6, 7, 8]
```

heapq: Priority Queues

`heapq` keeps a list arranged as a binary min-heap: the smallest item is always at index 0. Pushing and popping cost O(log n), and building a heap from a list

with `heapify` costs O(n). Python has no max-heap, so store negated values. Tuples are compared item by item, which makes `(priority, tie_breaker, payload)` entries a convenient way to order anything.

```
def demo():
    heap = [5, 1, 8, 3]
    heapq.heapify(heap)  # O(n)
    heapq.heappush(heap, 2)  # O(log n)
    print(heapq.heappop(heap), heap[0])  # smallest, then next smallest

    # A max-heap: store negated values.
    biggest = [-x for x in [5, 1, 8, 3]]
    heapq.heapify(biggest)
    print(-heapq.heappop(biggest))

    # Tuples compare item by item: (priority, tie-breaker, payload).
    tasks = [(2, "write"), (1, "plan"), (2, "test")]
    heapq.heapify(tasks)
    print([heapq.heappop(tasks)[1] for _ in range(3)])
    print(heapq.nlargest(2, [5, 1, 8, 3]))
```

```
1 2
8
['plan', 'test', 'write']
[8, 5]
```

> **Common Mistake:** If two heap entries have equal priorities, Python compares the next items in the tuples. If those are objects that can't be compared, such as linked-list nodes, you get a `TypeError`. Add a unique counter as a tie-breaker: `(priority, count, node)`. Problem 11.6 uses this trick.

Sorting With Keys, and bisect

`sorted` and `list.sort` accept a `key` function; returning a tuple sorts by several criteria at once, and negating a number reverses its order. The `bisect` module performs binary search on a sorted list: `bisect_left` finds the first position where a value could be inserted, `bisect_right` the last.

```
def demo():
    people = [("Ana", 31), ("Bo", 25), ("Cy", 31)]
    # Sort by age descending, then name ascending.
    print(sorted(people, key=lambda p: (-p[1], p[0])))

    scores = [10, 20, 20, 30]
    print(bisect_left(scores, 20), bisect_right(scores, 20))
```

```
    insort(scores, 25)  # insert, keeping the list sorted
    print(scores)
```

```
[('Ana', 31), ('Cy', 31), ('Bo', 25)]
1 3
[10, 20, 20, 25, 30]
```

itertools and functools

`itertools` provides combinations, permutations, Cartesian products and running totals, which are handy for brute-force solutions and for checking your answers. `functools.cache` remembers the results of a function, turning many recursive solutions into dynamic programming with one line (Chapter 18).

```
def demo():
    print(list(combinations("abc", 2)))
    print(len(list(permutations("abcd"))))
    print(list(product([0, 1], repeat=2)))
    print(list(accumulate([3, 1, 4, 1])))  # running totals

    @cache  # remember results of earlier calls
    def ways(n):
        return 1 if n <= 1 else ways(n - 1) + ways(n - 2)

    print(ways(80))
    print(float("inf") > 10**100, min([], default=-1))
```

```
[('a', 'b'), ('a', 'c'), ('b', 'c')]
24
[(0, 0), (0, 1), (1, 0), (1, 1)]
[3, 4, 8, 9]
37889062373143906
True -1
```

Python Habits That Help in Interviews

- **Use** `float("inf")` as a starting value for minimums, and `-float("inf")` for maximums.
- **Integers never overflow** in Python, which removes a whole class of bugs. Mention it if your interviewer works in Java or C++.
- **Recursion depth is limited** to about 1,000 calls by default. For inputs that could be deeper, such as a long linked list or a degenerate tree, use an

explicit stack, or say that you would raise the limit with `sys.setrecursionlimit`.

- **Don't mutate a list while looping over it.** Loop over a copy, or build a new list.
- **Don't modify the caller's input** unless the problem allows it. Several solutions in this book copy a grid before marking cells.
- **Know the edge cases of slicing:** slices never raise an error for out-of-range positions, which is convenient, but can hide bugs.

Key Takeaways

- Lists: O(1) append and index; O(n) for insertions at the front, `in`, and copying.
- Sets and dicts: O(1) average membership and lookup. Use tuples as composite keys.
- `Counter`, `defaultdict`, `deque`, `heapq`, `bisect`, `itertools` and `functools.cache` cover most of what interview problems need.
- Watch for the shared-row grid bug, uncomparable heap entries, recursion limits and accidental $O(n^2)$ string building.

PART II

The Patterns

CHAPTER 5

Arrays and Hashing

More interview problems are solved with a hash map than with any other tool. The idea is simple: instead of searching through a list again and again, remember what you have seen in a dictionary or set, and look it up in O(1). It turns many $O(n^2)$ brute-force solutions into O(n) ones.

When to Use It

Signals in the problem statement:

- "Find a pair (or element) that matches..." or "has this value been seen before?"
- "Count how many times..." or "group items that share a property".
- "Are these two things equivalent?" (same letters, same pattern, same counts).
- The brute force compares every item with every other item.

The recipe:

1. Decide what to remember about the items you have already processed, and what key will let you look it up.
2. Walk through the input once; for each item, look up what you need, then record the item.
3. Choose the right container: a **set** to remember "seen", a **dict** to remember "where" or "how many", a `Counter` for counts, a `defaultdict(list)` for groups.

The order of steps 2's "look up" and "record" matters, as the first problem shows.

5.1 Budget Pair (Easy)

A gift shop lists the prices of its items in a list. A customer has an exact budget and wants two *different* items whose prices add up to exactly that amount. Return the positions of the two items, smaller position first, or an empty list if no pair exists. If several pairs work, any one is acceptable.

```
>>> budget_pair([12, 7, 30, 18], 25)
[1, 3]
>>> budget_pair([5, 5], 10)
[0, 1]
>>> budget_pair([3, 8, 1], 100)
[]
>>> budget_pair([], 4)
[]
```

Think first: For a given price, which other price would complete the pair?

Brute force. Try every pair of positions i < j and check whether the prices add up to the budget. That is O(n²) time and O(1) space; with 100,000 prices it means about five billion checks.

The better idea. For each price, the partner it needs is fixed: `budget - price`. So walk through the list once, and for each price ask a dictionary whether its partner has already been seen. Store each price with its position *after* checking, so an item can never be paired with itself.

```
def budget_pair(prices, budget):
    seen = {}  # price -> index where we saw it
    for i, price in enumerate(prices):
        need = budget - price
        if need in seen:
            return [seen[need], i]
        seen[price] = i
    return []
```

Complexity: O(n) time, with one dictionary lookup per price; O(n) extra space for the dictionary.

> **Common Mistake:** Storing the price before checking for its partner. With prices `[5]` and a budget of 10, the item would find itself and return `[0, 0]`.

> **Follow-Up:** If the prices were sorted, could you solve it without extra memory? Yes, with two pointers (Problem 6.2). What if you needed *all* pairs, without duplicates? Then sort and use two pointers, skipping repeated values, as in Problem 6.3.

5.2 First Repeat Visitor (Easy)

A building records each visitor's badge ID in the order they enter. Return the first badge ID that appears for the second time, that is, the repeat that happens earliest in the log. Return `None` if no badge appears twice.

```
>>> first_repeat(['A7', 'B2', 'C9', 'B2', 'A7'])
'B2'
>>> first_repeat(['X1', 'X2'])
None
>>> first_repeat([])
None
>>> first_repeat(['Q', 'Q'])
'Q'
```

Think first: As you read the log, what do you need to remember in order to recognize a repeat immediately?

Brute force. For each position, check whether the badge appeared anywhere earlier. That costs $O(n^2)$ time.

The better idea. Keep a set of the badges seen so far. The first badge that is already in the set is the answer.

```
def first_repeat(badges):
    seen = set()
    for badge in badges:
        if badge in seen:
            return badge
        seen.add(badge)
    return None
```

Complexity: O(n) time, O(n) space.

> **Common Mistake:** Returning the badge whose *first* appearance is earliest among those that repeat. In `["A7", "B2", "C9", "B2", "A7"]`, A7 appears first, but B2 is the first to be *seen again*. Clarify this with the interviewer; it changes the answer.

> **Follow-Up:** If badge IDs were integers from 1 to n in a list of length n + 1, could you find a repeat in O(1) extra space without changing the list? (Yes: treat the values as "next" pointers and use the cycle detection idea of Problem 11.2.)

5.3 Anagram Shelves (Medium)

A librarian wants to shelve words together if they contain exactly the same letters, such as *listen*, *silent* and *enlist*. Given a list of lowercase words, return the groups. The order of groups and of words within a group doesn't matter.

```
>>> anagram_shelves(
...     ['listen', 'enlist', 'google', 'silent', 'elgoog', 'cat'],
... )
[['listen', 'enlist', 'silent'], ['google', 'elgoog'], ['cat']]
>>> anagram_shelves([])
[]
>>> anagram_shelves(['', ''])
[['', '']]
```

Think first: What could serve as a "label" that is identical for all anagrams of a word?

Brute force. Compare every pair of words by sorting their letters, and merge matching words into groups. That is $O(n^2 \times k \log k)$ for n words of length k.

The better idea. Give every word a key that anagrams share, and group by key with a `defaultdict(list)`. The simplest key is the word's letters in sorted order: *listen*, *silent* and *enlist* all become "eilnst".

```
def anagram_shelves(words):
    shelves = defaultdict(list)
    for word in words:
        key = "".join(sorted(word))
        shelves[key].append(word)
    return list(shelves.values())
```

An alternative key avoids sorting: count each of the 26 letters, and use the tuple of counts as the key. (A list can't be a dictionary key, but a tuple can.)

```
def anagram_shelves_counts(words):
    shelves = defaultdict(list)
    for word in words:
        counts = [0] * 26
        for ch in word:
            counts[ord(ch) - ord("a")] += 1
        shelves[tuple(counts)].append(word)
    return list(shelves.values())
```

Complexity: with sorted keys, O(n × k log k) time; with counts, O(n × k). Both use O(n × k) space for the groups.

> **Follow-Up:** What if the words could contain any Unicode characters? The count-array key no longer works; use the sorted-letters key, or a `Counter` frozen into a sorted tuple of items.

5.4 Longest Unbroken Streak (Medium)

A fitness app records the days (as integers) on which a user exercised, in no particular order and possibly with duplicates. What is the length of the longest run of consecutive days? Aim for O(n).

```
>>> longest_streak([5, 2, 99, 3, 4, 100])
4
>>> longest_streak([])
0
>>> longest_streak([7, 7, 7])
1
>>> longest_streak([-1, 0, 1])
3
```

Think first: How can you tell, in O(1), that a day is the *first* day of a streak?

The obvious idea. Sort the days, remove duplicates, and count runs of consecutive numbers. That is O(n log n), which is often acceptable, so mention it.

```
def longest_streak_sorting(days):
    if not days:
        return 0
    days = sorted(set(days))
    best = run = 1
```

```
    for a, b in zip(days, days[1:]):
        run = run + 1 if b == a + 1 else 1
        best = max(best, run)
    return best
```

The better idea. Put all days in a set. A day starts a streak only if the day before it is *not* in the set. From each start, count forward while the next day is present. Every day is counted forward at most once, because counting only starts at streak beginnings.

```
def longest_streak(days):
    present = set(days)
    best = 0
    for d in present:
        if d - 1 in present:
            continue  # not the start of a streak
        end = d
        while end + 1 in present:
            end += 1
        best = max(best, end - d + 1)
    return best
```

Complexity: O(n) time, because each day is visited a constant number of times; O(n) space.

> **Common Mistake:** Counting forward from *every* day, not just from streak starts. On a single streak of n days that does n + (n-1) + ... + 1 steps: $O(n^2)$.

5.5 Product Without Me (Medium)

For each position in a list of numbers, compute the product of all the *other* numbers, without using division. (Division fails when a value is zero, and the interviewer wants to see a different idea.)

```
>>> without_one([2, 3, 4, 5])
[60, 40, 30, 24]
>>> without_one([4, 0, 2])
[0, 8, 0]
>>> without_one([0, 0, 3])
[0, 0, 0]
>>> without_one([9])
[1]
```

Think first: The product of "everything except position i" is the product of everything to its left times everything to its right.

Brute force. For each position, multiply all the others: $O(n^2)$.

The better idea. Make two passes. Going left to right, store in each position the product of everything before it. Then go right to left with a running product of everything after it, and multiply it in.

```
def without_one(values):
    n = len(values)
    out = [1] * n
    left = 1
    for i in range(n):  # out[i] = product of everything to the left
        out[i] = left
        left *= values[i]
    right = 1
    for i in range(n - 1, -1, -1):  # times everything to the right
        out[i] *= right
        right *= values[i]
    return out
```

Complexity: O(n) time; O(1) extra space besides the output list.

> **Follow-Up:** How would you handle the same question for very large numbers where products overflow (in Java or C++)? You might work with logarithms, or with the product modulo a prime, if the interviewer allows.

5.6 Same Pattern (Easy)

Two product codes "follow the same pattern" if the letters of one can be replaced consistently to give the other: every occurrence of a letter maps to the same letter, and no two different letters map to the same letter. *moon* and *feet* follow the same pattern; *tree* and *tool* do not.

```
>>> same_pattern('moon', 'feet')
True
>>> same_pattern('tree', 'tool')
False
>>> same_pattern('ab', 'aa')
False
>>> same_pattern('', '')
True
>>> same_pattern('abc', 'ab')
False
```

Think first: What must you remember for each letter you have seen, in each direction?

Brute force. For every pair of positions, check that the letters are equal in one code exactly when they are equal in the other: $O(n^2)$.

The better idea. Keep two dictionaries, one mapping letters of `a` to letters of `b` and one mapping back. `setdefault` records a mapping the first time and returns the existing one afterward; any disagreement means the patterns differ.

```
def same_pattern(a, b):
    if len(a) != len(b):
        return False
    forward, backward = {}, {}
    for x, y in zip(a, b):
        if forward.setdefault(x, y) != y:
            return False
        if backward.setdefault(y, x) != x:
            return False
    return True
```

Complexity: O(n) time; O(k) space for k distinct letters.

> **Common Mistake:** Checking only one direction. With *ab* and *aa*, the forward map a → a, b → a looks consistent; only the backward map reveals that two letters were mapped to the same one.

Key Takeaways

- A hash map or set turns "search for it again" into an O(1) lookup, typically turning $O(n^2)$ into O(n).
- Decide what to remember, and in what order to look up and record.
- Good keys capture what matters: sorted letters or letter counts for anagrams, a value's partner for pairs.
- Sets can also tell you where a run starts, as in the streak problem.

CHAPTER 6

Two Pointers

Many problems on arrays and strings can be solved by walking two positions through the data at the same time: one from each end, moving toward each other, or both from the start at different speeds. Because each pointer only ever moves forward, the whole walk takes O(n), and usually needs no extra memory.

When to Use It

- The input is **sorted**, or sorting it first is acceptable.
- You are looking for a **pair or triple** with some property, such as a target sum.
- The problem compares the **two ends** of something: palindromes, containers, symmetric structures.
- You need to **merge** two sorted sequences, or **filter in place**.

The recipe for opposite-end pointers:

1. Start `left` at the beginning and `right` at the end.
2. Look at the two items; decide which pointer can be moved without losing any possible answer.
3. Move it, and repeat until the pointers meet.

The key step is the argument in step 2: *why* moving that pointer can't skip the answer. Say it out loud in the interview; it is the proof that your solution is correct.

6.1 Palindrome Plate (Easy)

A label reads the same forwards and backwards if you ignore everything except letters and digits, and ignore case. *Step on no pets!* qualifies. Write a function to check, using O(1) extra space.

```
>>> is_palindrome('Step on no pets!')
True
>>> is_palindrome('Race-car 7')
False
>>> is_palindrome('')
True
>>> is_palindrome(' ,. ')
True
>>> is_palindrome('A1b1a')
True
```

Think first: You could build a cleaned copy and compare it with its reverse. How can you avoid the copy?

The simple version. Filter and lower-case the characters into a list, and compare it with its reverse. It is O(n) time but O(n) space, and it is a perfectly good first answer.

```
def is_palindrome_simple(text):
    kept = [ch.lower() for ch in text if ch.isalnum()]
    return kept == kept[::-1]
```

The better idea. Compare from both ends inwards, skipping characters that don't count.

```
def is_palindrome(text):
    left, right = 0, len(text) - 1
    while left < right:
        if not text[left].isalnum():
            left += 1
        elif not text[right].isalnum():
            right -= 1
        elif text[left].lower() != text[right].lower():
            return False
        else:
            left += 1
            right -= 1
    return True
```

Complexity: O(n) time, O(1) extra space.

> **Follow-Up:** "Can it become a palindrome by deleting at most one character?" At the first mismatch, try skipping either the left or the right character, and check whether the rest is a palindrome. That is still O(n).

6.2 Sorted Pair (Easy)

A list of package weights is sorted in increasing order. Find two different packages whose weights sum to a target, using O(1) extra space. Return their positions, or an empty list.

```
>>> sorted_pair([1, 3, 4, 6, 9], 10)
[0, 4]
>>> sorted_pair([2, 2], 4)
[0, 1]
>>> sorted_pair([1, 2, 3], 7)
[]
```

Think first: If the smallest plus the largest is too small, which of the two can never be part of the answer?

The idea. Start with the lightest and heaviest packages. If their sum is too small, the lightest package can't pair with anything (even the heaviest isn't enough), so move `left` up. If the sum is too big, the heaviest package can't pair with anything, so move `right` down.

```
def sorted_pair(weights, target):
    left, right = 0, len(weights) - 1
    while left < right:
        total = weights[left] + weights[right]
        if total == target:
            return [left, right]
        if total < target:
            left += 1  # need a bigger sum
        else:
            right -= 1  # need a smaller sum
    return []
```

Complexity: O(n) time, O(1) space. Compare this with Problem 5.1, which needs O(n) space because its input isn't sorted.

6.3 Balanced Triples (Medium)

A sensor reports readings, which can be negative. Find all *distinct* triples of readings that sum to zero. Each triple should be listed in increasing order, and the same triple of values should appear only once.

```
>>> balanced_triples([-3, 1, 2, -1, 0, 1])
[[-3, 1, 2], [-1, 0, 1]]
>>> balanced_triples([0, 0, 0, 0])
[[0, 0, 0]]
>>> balanced_triples([1, 2])
[]
```

Think first: If you fix the smallest number of the triple, what problem is left?

Brute force. Check all $O(n^3)$ triples, putting sorted ones into a set to remove duplicates.

The better idea. Sort the readings. Fix each value as the first of the triple; the other two must sum to its negative, which is Problem 6.2 on the rest of the list. To avoid duplicate triples, skip a first value equal to the previous one, and after finding a triple, move `left` past any copies of the same value.

```
def balanced_triples(readings):
    nums = sorted(readings)
    result = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue  # same first value: same triples again
        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
                right -= 1
    return result
```

Complexity: $O(n^2)$ time: $O(n \log n)$ to sort, then $O(n)$ for each of n first values. $O(1)$ extra space apart from the output and the sorted copy.

> **Common Mistake:** Using a set of tuples to remove duplicates *after* finding them. It works, but interviewers usually ask you to avoid duplicates without it; the skipping logic shows that you understand where they come from.

6.4 Widest Reservoir (Medium)

Vertical walls of different heights stand at positions 0, 1, 2, and so on. Choose two walls; water fills the space between them up to the height of the shorter wall. What is the most water any pair can hold? The amount for walls i and j is `(j - i) × min(height[i], height[j])`.

```
>>> reservoir([3, 7, 2, 6, 4, 8, 1])
28
>>> reservoir([1, 1])
1
>>> reservoir([5])
0
```

Think first: Starting with the two outermost walls, the widest possible pair, which wall should you give up on?

Brute force. Try every pair: $O(n^2)$.

The better idea. Start with the outermost pair. Any pair using the *shorter* of the two walls, with some wall further in, would be narrower and no taller than this one, because the shorter wall still limits it. So the shorter wall can never do better, and you can discard it by moving its pointer inwards.

```
def reservoir(heights):
    left, right = 0, len(heights) - 1
    best = 0
    while left < right:
        width = right - left
        best = max(best, width * min(heights[left], heights[right]))
        # The shorter wall limits the water; moving the taller one
        # can never help, so move the shorter one inward.
        if heights[left] < heights[right]:
            left += 1
        else:
            right -= 1
    return best
```

Complexity: O(n) time, O(1) space.

> **Interviewer's View:** Many candidates guess the right pointer move. What distinguishes a strong answer is the one-sentence argument for *why* discarding the shorter wall is safe.

6.5 Merge Two Logs (Easy)

Two servers each produce a sorted list of event timestamps. Merge them into a single sorted list in O(m + n).

```
>>> merge_logs([1, 4, 9], [2, 3, 10, 11])
[1, 2, 3, 4, 9, 10, 11]
>>> merge_logs([], [5])
[5]
>>> merge_logs([2, 2], [2])
[2, 2, 2]
```

The idea. Keep a pointer into each list, and repeatedly take the smaller of the two current items. When one list runs out, append the rest of the other. This is the merge step of merge sort.

```
def merge_logs(a, b):
    i = j = 0
    merged = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            merged.append(a[i])
            i += 1
        else:
            merged.append(b[j])
            j += 1
    merged.extend(a[i:])  # at most one of these is non-empty
    merged.extend(b[j:])
    return merged
```

Complexity: O(m + n) time; O(m + n) space for the result. Calling `sorted(a + b)` also works, but is O(N log N) and ignores the fact that both inputs are already sorted, which is what the interviewer wants to see you use.

> **Follow-Up:** "Merge into the first list in place, if it has empty space at the end." Fill from the back: compare the largest remaining items and write them at the end, so nothing is overwritten before it is used.

6.6 Trapped Rain (Hard)

The heights of a row of blocks form a skyline. After heavy rain, water collects in the dips between taller blocks. How many units of water are trapped?

```
>>> trapped_rain([3, 0, 2, 0, 4])
7
>>> trapped_rain([2, 0, 3, 1, 0, 2, 5, 1, 2])
9
>>> trapped_rain([])
0
>>> trapped_rain([5, 4, 3])
0
```

Think first: How much water sits on top of a single block?

The key observation. The water above block `i` is `min(tallest to its left, tallest to its right) - height[i]`, counting the block itself in both directions, so the result is never negative.

Brute force. For each block, scan left and right for the tallest blocks: $O(n^2)$. An O(n) improvement precomputes two arrays of "tallest so far" from each side, using O(n) space.

```
def trapped_rain_brute(heights):
    water = 0
    for i in range(len(heights)):
        left_max = max(heights[: i + 1])
        right_max = max(heights[i:])
        water += min(left_max, right_max) - heights[i]
    return water
```

The better idea. Use two pointers and two running maxima. Whichever side currently has the lower block is the side we can settle: if `heights[left] < heights[right]`, there is a wall at least as tall as `heights[right]` on the right, so the water above `left` depends only on `left_max`.

```
def trapped_rain(heights):
    left, right = 0, len(heights) - 1
    left_max = right_max = 0
    water = 0
    while left < right:
        if heights[left] < heights[right]:
            # The right side has a wall at least this tall, so the
            # water here is limited only by left_max.
            left_max = max(left_max, heights[left])
            water += left_max - heights[left]
            left += 1
        else:
```

```
            right_max = max(right_max, heights[right])
            water += right_max - heights[right]
            right -= 1
    return water
```

Complexity: O(n) time, O(1) space.

> **Follow-Up:** The same skyline problem in two dimensions, with a grid of heights, needs a different idea: process cells from the lowest boundary inwards using a heap.

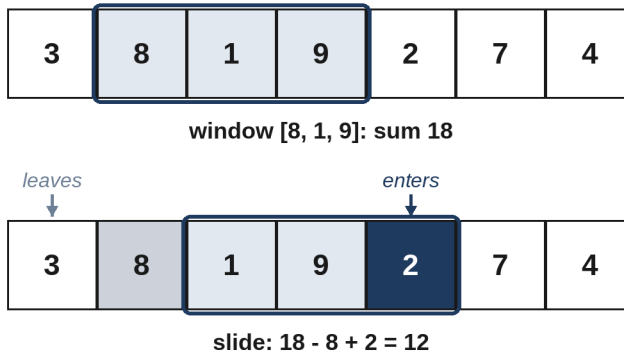
Key Takeaways

- Two pointers from opposite ends work when you can always rule out one end; say *why* the move is safe.
- On sorted input, two pointers find pairs in O(n) without extra memory; fix one value to extend this to triples.
- Two pointers moving in the same direction merge sorted lists and filter in place.
- Hard problems such as trapped rain combine two pointers with running maxima.

CHAPTER 7

Sliding Window

A sliding window is a special case of two pointers, where the two pointers mark the start and end of a block of consecutive items: a "window". As the end moves right, new items enter; as the start moves right, old items leave. If you can update what you know about the window in O(1) when one item enters or leaves, you can examine every window in O(n) instead of recomputing each from scratch.



A sliding window: the right edge advances to add new items, and the left edge advances to remove old ones, so each item enters and leaves the window at most once.

When to Use It

- The problem asks about a **consecutive** block: a substring, a subarray, a run of days.
- It asks for the **longest**, **shortest** or **best** such block, or **whether one exists**.
- Adding or removing one item changes the answer in a simple way (a sum, a count of characters).

Two kinds of windows:

- **Fixed size k.** Add the new item and remove the one that falls out, every step (Problem 7.1).
- **Variable size.** Expand the right edge every step; shrink from the left while the window breaks a rule (for "longest") or while it still meets the goal (for "shortest").

The variable-size template, in words: for each `right`, add `items[right]` to the window's state; then, while the state is invalid (or while it satisfies the goal), update the answer if appropriate and remove `items[left]`, moving `left` forward. Each item enters once and leaves at most once, so the total work is O(n).

> **Warning:** Variable-size windows rely on a property called *monotonicity*: making the window bigger can only move it toward "invalid" (or toward "goal met"). If values can be negative, a sum doesn't behave this way, and a sliding window gives wrong answers. Use prefix sums instead (Chapter 8).

7.1 Best Week (Easy)

Given daily sales, find the highest total over any k consecutive days.

```
>>> best_window([3, 8, 1, 9, 2, 7, 4], 3)
18
>>> best_window([5], 1)
5
>>> best_window([-2, -1, -3], 2)
-3
```

Brute force. Sum each window separately: O(n × k).

The better idea. Keep the current window's total. Moving one day forward, add the new day and subtract the day that left.

```
def best_window(sales, k):
    window = sum(sales[:k])
    best = window
    for i in range(k, len(sales)):
        window += sales[i] - sales[i - k]  # add the new day, drop the old
        best = max(best, window)
    return best
```

Complexity: O(n) time, O(1) space.

7.2 Longest Unique Run (Medium)

Find the length of the longest block of a string in which no character repeats.

```
>>> longest_unique('launchpad')
7
>>> longest_unique('bbbb')
1
>>> longest_unique('')
0
>>> longest_unique('dvdf')
3
>>> longest_unique('abba')
2
```

Think first: When the next character is already in the window, where must the window's new start be?

Brute force. Check every block for repeats: $O(n^3)$ as written below, or $O(n^2)$ with a running set.

The better idea. Remember the last position of each character. When the next character was last seen *inside* the current window, jump the start of the window to just after that position. Positions before the window are ignored.

```
def longest_unique(s):
    last_seen = {}  # character -> last index where it appeared
    start = 0
    best = 0
    for i, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= start:
            start = last_seen[ch] + 1  # jump past the earlier copy
        last_seen[ch] = i
        best = max(best, i - start + 1)
    return best
```

Complexity: O(n) time; O(k) space for k distinct characters.

> **Common Mistake:** Forgetting the condition `last_seen[ch] >= start`. In "abba", when the final "a" arrives, its earlier copy is already outside the window; without the check, the start would jump *backwards*, and the answer would be 3 instead of 2. The last example above tests exactly this.

7.3 Shortest Span (Medium)

Deliveries arrive with positive weights. What is the shortest run of consecutive deliveries whose total weight is at least a target? Return 0 if no run reaches it.

```
>>> shortest_span([4, 1, 1, 3, 2, 5], 8)
3
>>> shortest_span([1, 1, 1], 5)
0
>>> shortest_span([8], 7)
1
```

The idea. Expand the window to the right. As soon as the total reaches the target, record the length and try to shrink from the left, since a shorter window might still be enough. Because all weights are positive, removing an item always lowers the total, which is what makes shrinking safe.

```
def shortest_span(weights, target):
    best = len(weights) + 1
    total = 0
    left = 0
    for right, w in enumerate(weights):
        total += w
        while total >= target:  # shrink while still big enough
            best = min(best, right - left + 1)
            total -= weights[left]
            left += 1
    return best if best <= len(weights) else 0
```

Complexity: O(n) time, O(1) space.

> **Follow-Up:** With negative weights allowed, the window idea breaks. A solution uses prefix sums and a monotonic deque in O(n), or prefix sums and binary search in O(n log n).

7.4 Repaint the Fence (Medium)

A fence's boards are painted in colors, given as capital letters. You may repaint at most k boards. What is the longest run of boards of a single color you can achieve?

```
>>> repaint('RRGRBB', 1)
4
>>> repaint('ABBBAB', 1)
5
>>> repaint('', 2)
0
>>> repaint('ABC', 0)
1
```

Think first: For a given window, how many boards would you need to repaint to make it all one color?

The idea. A window can be made one color by repainting everything except its most common color: `length - most`. Keep counts of the colors in the window. If `length - most` exceeds k, slide the window: move the left edge forward by one.

```
def repaint(boards, k):
    counts = {}
    most = 0  # highest count of one color seen in a window so far
    left = 0
    best = 0
    for right, color in enumerate(boards):
        counts[color] = counts.get(color, 0) + 1
        most = max(most, counts[color])
        if right - left + 1 - most > k:  # too many boards to repaint
            counts[boards[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best
```

A subtle point: `most` is the highest count seen in *any* window so far, and is never decreased when boards leave. That's safe, because the answer can only

improve when some window has a higher count than before, and until then, the window just slides at its best length without shrinking.

Complexity: O(n) time, O(1) space for 26 colors.

7.5 Hidden Anagram (Medium)

Does `text` contain a block of consecutive characters that is a rearrangement of `word`?

```
>>> hidden_anagram('tea', 'slate')
True
>>> hidden_anagram('tea', 'tablet')
False
>>> hidden_anagram('', 'abc')
True
>>> hidden_anagram('abcd', 'abc')
False
```

The idea. Slide a fixed window of length `len(word)` across the text, keeping a `Counter` of its characters. Compare it with the word's counts after each step, adding the new character and removing the old one, and deleting a count when it reaches zero, so the comparison stays exact.

```
def hidden_anagram(word, text):
    k = len(word)
    if k > len(text):
        return False
    need = Counter(word)
    window = Counter(text[:k])
    if window == need:
        return True
    for i in range(k, len(text)):
        window[text[i]] += 1
        window[text[i - k]] -= 1
        if window[text[i - k]] == 0:
            del window[text[i - k]]
        if window == need:
            return True
    return False
```

Complexity: O(n) time for a fixed alphabet, because comparing two counters of at most 26 letters is O(1).

> **Follow-Up:** Return the starting positions of *all* the anagrams. Same window; collect `i - k + 1` at every match.

7.6 Smallest Cover (Hard)

Find the shortest block of `text` that contains every character of `needed`, including repeats. If several are equally short, return the leftmost; if none exists, return "".

```
>>> smallest_cover('PLANETARY', 'TAN')
'ANET'
>>> smallest_cover('a', 'aa')
''
>>> smallest_cover('aa', 'aa')
'aa'
>>> smallest_cover('xyz', '')
''
```

Think first: How do you know, in O(1), that the window now contains everything?

The idea. Keep `need`, a count of how many of each character the window still lacks (it can go negative when the window has extras), and `missing`, the total number of characters still lacking. Expand to the right; when `missing` reaches zero the window covers everything, so record it and shrink from the left until it no longer does.

```
def smallest_cover(text, needed):
    if not needed:
        return ""
    need = Counter(needed)
    missing = len(needed)  # characters still missing from the window
    left = 0
    best_start, best_len = 0, len(text) + 1
    for right, ch in enumerate(text):
        if need[ch] > 0:
            missing -= 1
        need[ch] -= 1
        while missing == 0:  # window covers everything: try to shrink
            if right - left + 1 < best_len:
                best_start, best_len = left, right - left + 1
            need[text[left]] += 1
            if need[text[left]] > 0:
                missing += 1
            left += 1
    if best_len > len(text):
        return ""
    return text[best_start : best_start + best_len]
```

Complexity: O(n + m) time for text length n and m needed characters; O(k) space for k distinct characters.

> **Interviewer's View:** This is a classic hard problem because of its bookkeeping. Explain the meaning of `need` and `missing` before writing the code, and test with repeated characters, such as needing "aa".

Key Takeaways

- Sliding windows examine every block of consecutive items in O(n) by updating, not recomputing.
- Fixed windows add one item and remove one each step; variable windows expand right and shrink left.
- They need a monotonic condition: with negative numbers, switch to prefix sums.
- Store just enough state to update in O(1): a sum, a count of characters, a last-seen position.

CHAPTER 8

Prefix Sums

A prefix sum is a running total: `prefix[i]` is the sum of the first i values. With it, the sum of any block of consecutive values is a single subtraction, `prefix[b + 1] - prefix[a]`. Combined with a hash map, prefix sums can also *count* blocks with a given total in one pass, even when values are negative, which is exactly where sliding windows fail.

When to Use It

- Many questions about **sums of ranges** in a list that doesn't change.
- **Counting** or finding the **longest** subarray with a given sum, especially with **negative** values.
- Conditions on sums **modulo k**, or on **balance** (more of one thing than another).
- The same ideas in two dimensions: sums of **rectangles** in a grid.

The key identity: the sum of `values[a..b]` equals `prefix[b + 1] - prefix[a]`. So "is there a block ending here with sum k?" becomes "have I seen the prefix sum `prefix - k` before?", which is a hash-map lookup.

8.1 Range Totals (Easy)

Given daily values and many queries "what is the total from day a to day b, inclusive?", answer each query in O(1).

```
>>> range_totals([4, -2, 7, 1, 3], [(0, 2), (1, 4), (3, 3)])
[9, 9, 1]
```

The idea. Precompute running totals once, starting with 0 for "no values". Then each query is one subtraction.

```
class RangeTotals:
    def __init__(self, values):
        # prefix[i] = sum of the first i values
        self.prefix = [0]
        for v in values:
            self.prefix.append(self.prefix[-1] + v)

    def total(self, a, b):
        """Sum of values[a] .. values[b], inclusive."""
        return self.prefix[b + 1] - self.prefix[a]
```

Complexity: O(n) to build, O(1) per query, O(n) space.

> **Follow-Up:** "What if values can change between queries?" Prefix sums then need O(n) per update. A *Fenwick tree* or *segment tree* supports both updates and range sums in O(log n).

8.2 Exact Total Runs (Medium)

How many blocks of consecutive values add up to exactly k? Values may be negative or zero.

```
>>> count_runs([1, 2, 3, -3, 3], 3)
5
>>> count_runs([2, 2, 2], 4)
2
>>> count_runs([], 0)
0
>>> count_runs([0, 0], 0)
3
```

Think first: A block ending at position i sums to k when the prefix sum before it equals what?

Brute force. Extend a running sum from every start: $O(n^2)$.

The better idea. Walk through the values, keeping the current prefix sum and a count of how many times each prefix sum has occurred. A block ending here sums to k exactly when an earlier prefix equals `prefix - k`, so add that

count. Start the map with `{0: 1}` for the empty prefix, so blocks starting at position 0 are counted.

```
def count_runs(values, k):
    seen = {0: 1}  # prefix sum -> how many times we've seen it
    prefix = 0
    count = 0
    for v in values:
        prefix += v
        # A run ending here sums to k when an earlier prefix was prefix-k.
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count
```

Complexity: O(n) time, O(n) space.

> **Common Mistake:** Forgetting the starting `{0: 1}`. Then a block that starts at the very beginning, such as `[1, 2]` for k = 3, is never counted.

8.3 Balance Point (Easy)

Find the leftmost position where the sum of everything to its left equals the sum of everything to its right (the item itself is in neither). Return -1 if there is none.

```
>>> balance_point([2, 3, -1, 8, 4])
3
>>> balance_point([1, 2, 3])
-1
>>> balance_point([5])
0
>>> balance_point([])
-1
```

The idea. Compute the total once. Walking left to right with a running left sum, the right sum at position i is `total - left - values[i]`.

```
def balance_point(values):
    total = sum(values)
    left = 0
    for i, v in enumerate(values):
        if left == total - left - v:
            return i
        left += v
    return -1
```

Complexity: O(n) time, O(1) space.

8.4 Even Split (Medium)

A team's results are recorded as 1 (win) and 0 (loss). Find the longest run of consecutive games with as many wins as losses.

```
>>> even_split([0, 1, 1, 0, 1, 1, 1, 0])
4
>>> even_split([1, 1, 1])
0
>>> even_split([0, 1])
2
>>> even_split([])
0
```

Think first: Count each win as +1 and each loss as -1. What is the "balance" of a block with equal wins and losses?

The idea. With the +1/-1 trick, a block is even exactly when its sum is 0, that is, when the running balance at its end equals the balance just before its start. To get the *longest* such block, remember the *first* position where each balance occurred.

```
def even_split(results):
    first_seen = {0: -1}  # balance -> first index where it occurred
    balance = 0
    best = 0
    for i, r in enumerate(results):
        balance += 1 if r == 1 else -1
        if balance in first_seen:
            best = max(best, i - first_seen[balance])
        else:
            first_seen[balance] = i
    return best
```

Complexity: O(n) time, O(n) space.

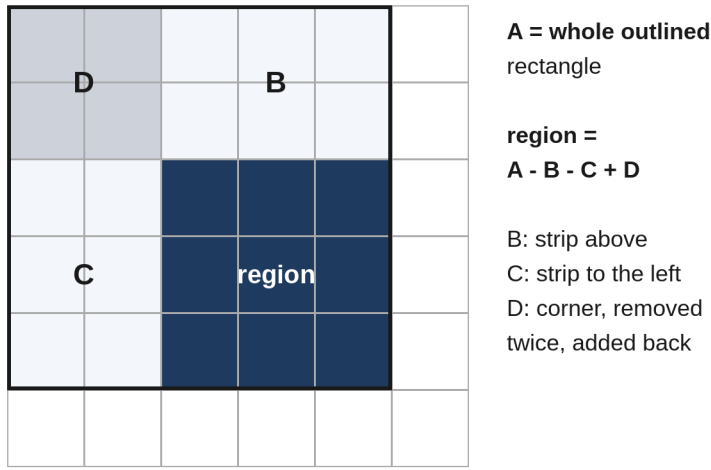
> **Common Mistake:** Updating `first_seen` every time a balance repeats. The map must keep the *earliest* position, because the longest block starts as early as possible.

8.5 Region Sums (Medium)

Given a grid of numbers, answer many queries "what is the sum of the rectangle from (r1, c1) to (r2, c2)?" in O(1) each.

```
>>> region_sums(
...     [[3, 0, 1], [5, 6, 3], [1, 2, 0]],
...     [(0, 0, 1, 1), (1, 1, 2, 2)],
... )
[14, 11]
```

The idea. Build a two-dimensional prefix table: `p[r][c]` is the sum of the rectangle from the top-left corner to cell (r-1, c-1). Each entry adds the cell to the rectangles above and to the left, and subtracts the overlap, which was counted twice. A query uses the same inclusion-exclusion in reverse.



Inclusion-exclusion for a rectangle sum: take the large rectangle to the bottom-right corner, subtract the strip above and the strip to the left, then add back the top-left corner, which was subtracted twice.

```
class RegionSums:
    def __init__(self, grid):
        rows = len(grid)
        cols = len(grid[0]) if rows else 0
        # p[r][c] = sum of the rectangle from (0, 0) to (r-1, c-1)
        self.p = [[0] * (cols + 1) for _ in range(rows + 1)]
```

```
        for r in range(rows):
            for c in range(cols):
                self.p[r + 1][c + 1] = (
                    grid[r][c]
                    + self.p[r][c + 1]
                    + self.p[r + 1][c]
                    - self.p[r][c]
                )

    def total(self, r1, c1, r2, c2):
        p = self.p
        return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1]
```

Complexity: O(rows × cols) to build, O(1) per query.

8.6 Divisible Runs (Medium)

How many blocks of consecutive values have a total divisible by k?

```
>>> divisible_runs([3, 1, 4, 1, 5, 9], 5)
6
>>> divisible_runs([5], 9)
0
>>> divisible_runs([], 3)
0
```

Think first: When is the difference of two numbers divisible by k?

The idea. A block's total is divisible by k exactly when the prefix sums at its two ends have the same remainder modulo k. So count remainders as you go, just like Problem 8.2 counts prefix sums. Python's `%` always returns a value between 0 and k-1 for positive k, even for negative sums, so no extra adjustment is needed.

```
def divisible_runs(values, k):
    seen = [0] * k  # remainder of prefix sum -> how many times
    seen[0] = 1
    prefix = 0
    count = 0
    for v in values:
        prefix = (prefix + v) % k  # Python's % is never negative here
        count += seen[prefix]
        seen[prefix] += 1
    return count
```

Complexity: O(n) time, O(k) space.

> **Common Mistake:** In Java and C++, `%` can return a negative remainder for negative numbers. If you translate this solution, use `((prefix % k) + k) % k`.

Key Takeaways

- Prefix sums turn any range sum into one subtraction.
- With a hash map of earlier prefix sums, you can count or find blocks with a given sum in O(n), even with negative values.
- Transform the problem to fit: +1/-1 for balance, remainders for divisibility.
- In two dimensions, use inclusion-exclusion: add the big rectangle, subtract two strips, add back the corner.

CHAPTER 9

Stacks and Queues

A **stack** is last-in, first-out: like a pile of plates, you add and remove at the top. A **queue** is first-in, first-out: like a line at a counter. In Python, a list is a perfect stack (`append` and `pop` are O(1)), and `collections.deque` is a perfect queue. Stacks appear whenever something must be *matched* or *undone* in reverse order; a special kind, the **monotonic stack**, answers "what is the next bigger item?" for every item in one pass.

When to Use It

- **Matching** pairs that nest: brackets, tags, function calls.
- **Undo** or **backtrack** in reverse order; evaluating expressions.
- "For each item, find the **next (or previous) greater or smaller** item": a monotonic stack.
- "The maximum (or minimum) of every **window**": a monotonic deque.
- Processing items **in arrival order**: a queue (and breadth-first search, Chapter 16).

A monotonic stack keeps items in increasing or decreasing order. When a new item would break the order, items are popped, and *popping is the moment an item finds its answer*: the new item is the first one bigger (or smaller) than it.

9.1 Balanced Brackets (Easy)

Is every bracket in the text closed in the right order? `()`, `[]` and `{}` count; other characters are ignored.

```
>>> balanced('f(a[i]) {}')
True
>>> balanced('([)]')
False
>>> balanced('((')
False
>>> balanced('')
True
>>> balanced(')(')
False
```

The idea. Push each opening bracket on a stack. A closing bracket must match the most recent unmatched opener, which is the top of the stack. At the end, the stack must be empty.

```
def balanced(s):
    closers = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in closers:
            if not stack or stack.pop() != closers[ch]:
                return False
    return not stack
```

Complexity: O(n) time, O(n) space.

> **Common Mistake:** Forgetting the final check `not stack`. "((" has no mismatches, but it isn't balanced.

9.2 Minimum Stack (Medium)

Design a stack with `push`, `pop`, `top` and `minimum`, all in O(1).

Think first: When you pop an item, how can you know the new minimum without searching?

The idea. Store, with each item, the smallest value at or below it in the stack. The minimum of the whole stack is then simply stored with the top item, and popping automatically restores the previous minimum.

```
class MinStack:
    def __init__(self):
        self.items = []  # pairs of (value, smallest value at or below)

    def push(self, value):
        smallest = min(value, self.items[-1][1]) if self.items else value
        self.items.append((value, smallest))

    def pop(self):
        return self.items.pop()[0]

    def top(self):
        return self.items[-1][0]

    def minimum(self):
        return self.items[-1][1]
```

The examples run a sequence of operations and collect what `pop`, `top` and `minimum` return:

```
>>> min_stack_session(
...     [('push', 5),
...      ('push', 2),
...      ('push', 8),
...      ('min',),
...      ('pop',),
...      ('pop',),
...      ('min',)],
... )
[2, 8, 2, 5]
```

Complexity: O(1) for every operation; O(n) space.

> **Follow-Up:** "Use less memory." Keep a second stack that receives a value only when it is less than or equal to the current minimum, and pop from it only when the popped value equals its top.

9.3 Postfix Calculator (Medium)

Evaluate an arithmetic expression written in postfix (reverse Polish) notation, where operators come after their operands: `3 4 + 2 *` means (3 + 4) × 2. Division truncates toward zero.

```
>>> postfix(['3', '4', '+', '2', '*'])
14
>>> postfix(['7', '-2', '/'])
-3
>>> postfix(['5'])
5
>>> postfix(['2', '3', '4', '*', '-'])
-10
```

The idea. Numbers are pushed onto a stack. An operator pops its two operands, the right operand first, and pushes the result.

```
def postfix(tokens):
    stack = []
    for tok in tokens:
        if tok in ("+", "-", "*", "/"):
            b = stack.pop()  # the right operand is on top
            a = stack.pop()
            if tok == "+":
                stack.append(a + b)
            elif tok == "-":
                stack.append(a - b)
            elif tok == "*":
                stack.append(a * b)
            else:
                stack.append(int(a / b))  # truncate toward zero
        else:
            stack.append(int(tok))
    return stack[0]
```

Complexity: O(n) time, O(n) space.

> **Common Mistake:** In Python, `//` rounds toward negative infinity, so `-7 // 2` is -4. "Truncate toward zero" requires `int(a / b)`, which gives -3. The example `7 / -2` checks this.

9.4 Next Warmer Day (Medium)

Given daily temperatures, for each day, how many days do you wait for a warmer one? Use 0 if no warmer day follows.

```
>>> next_warmer([21, 19, 23, 22, 20, 25])
[2, 1, 3, 2, 1, 0]
>>> next_warmer([30, 30, 30])
[0, 0, 0]
>>> next_warmer([])
[]
```

Brute force. For each day, scan forward: $O(n^2)$ in the worst case, for falling temperatures.

The better idea. Keep a stack of days still waiting for a warmer day; their temperatures decrease from bottom to top. When a new day is warmer than the top, the top day's wait is over: pop it and record the answer. Repeat, then push the new day.

```
def next_warmer(temps):
    out = [0] * len(temps)
    waiting = []  # indices of days still waiting, temps decreasing
    for i, t in enumerate(temps):
        while waiting and temps[waiting[-1]] < t:
            j = waiting.pop()
            out[j] = i - j
        waiting.append(i)
    return out
```

Complexity: O(n) time: each day is pushed once and popped at most once. O(n) space.

9.5 Largest Banner (Hard)

A skyline is made of bars of width 1. What is the area of the largest rectangle that fits entirely under the skyline?

```
>>> largest_banner([2, 4, 5, 3, 1, 3])
9
>>> largest_banner([4])
4
>>> largest_banner([])
0
>>> largest_banner([3, 3, 3])
9
```

Think first: If a bar is the *shortest* bar in the rectangle, how far can the rectangle extend left and right?

Brute force. For each starting bar, extend right while tracking the minimum height: $O(n^2)$.

The better idea. For each bar, the widest rectangle with that bar as the shortest extends until the first shorter bar on each side. A stack of bars with increasing heights finds both boundaries: when a shorter bar arrives, it is the

right boundary for every taller bar it pops, and the bar below each popped bar on the stack is its left boundary. A height-0 bar at the end flushes the stack.

```
def largest_banner(heights):
    stack = []  # indices with increasing heights
    best = 0
    for i, h in enumerate(heights + [0]):  # the 0 empties the stack
        while stack and heights[stack[-1]] >= h:
            j = stack.pop()
            # heights[j] extends left to just after the new stack top
            left = stack[-1] + 1 if stack else 0
            best = max(best, heights[j] * (i - left))
        stack.append(i)
    return best
```

Complexity: O(n) time, O(n) space.

> **Follow-Up:** "The largest rectangle of 1s in a binary grid." Treat each row as the base of a skyline whose heights count the consecutive 1s above, and apply this solution to each row: O(rows × cols).

9.6 Window Peaks (Hard)

Return the maximum of every window of k consecutive values.

```
>>> window_peaks([4, 2, 12, 3, 8, 1, 5], 3)
[12, 12, 12, 8, 8]
>>> window_peaks([4], 1)
[4]
>>> window_peaks([2, 2, 2], 2)
[2, 2]
```

Brute force. Take the maximum of each window: O(n × k). A heap gives O(n log n).

The better idea. Keep a deque of positions whose values *decrease* from front to back. A new value removes all smaller values from the back, because they can never be a window maximum again while it is in the window. The front is the current maximum; remove it when it falls out of the window.

```
def window_peaks(values, k):
    dq = deque()  # indices; their values decrease from front to back
    out = []
    for i, v in enumerate(values):
        while dq and values[dq[-1]] <= v:
            dq.pop()  # smaller values can never be a maximum again
```

```
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()  # front fell out of the window
        if i >= k - 1:
            out.append(values[dq[0]])
    return out
```

Complexity: O(n) time: each position enters and leaves the deque at most once. O(k) space.

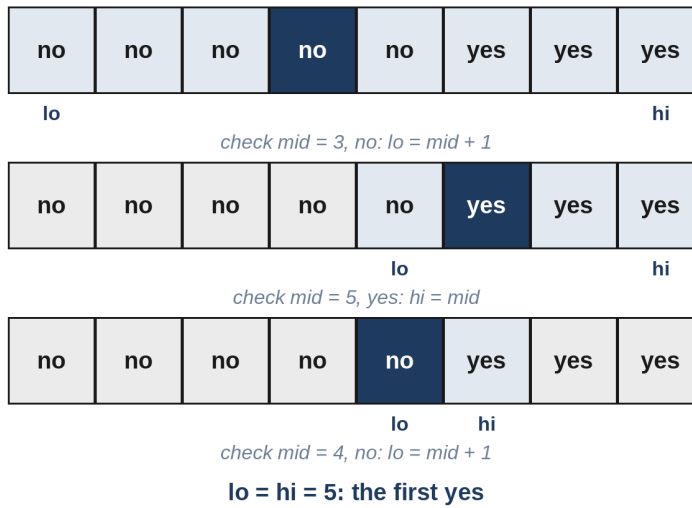
Key Takeaways

- Stacks match and undo things in reverse order: brackets, expressions, nested structures.
- A monotonic stack finds the next greater or smaller item for every position in O(n); popping is when an item gets its answer.
- A monotonic deque gives the maximum or minimum of every sliding window in O(n).
- Use a list as a stack and a `deque` as a queue.

CHAPTER 10

Binary Search

Binary search finds an item in a sorted list by repeatedly halving the range where it could be: look at the middle, decide which half the answer is in, and discard the other half. A million items need only about twenty steps. In interviews, binary search shows up in two forms: searching *in a sorted list*, and searching *for an answer*, when you can test any guess and the tests change from "no" to "yes" at some point.



Binary search for the first "yes": the range lo to hi always contains the answer; checking the middle discards half of the range each step until lo equals hi.

When to Use It

- The input is **sorted**, or sorted and **rotated**.
- You must find the **first** or **last** position where something becomes true.
- The problem asks for the **minimum** (or maximum) value that works, and you can **check a guess** efficiently: "the smallest capacity", "the earliest day", "the largest size".
- The input is huge, or the hint is "better than O(n)".

The safest template searches for the *first* position where a condition is true, on a range `[lo, hi]` that always contains the answer:

1. While `lo < hi`: take `mid = (lo + hi) // 2`.
2. If the condition holds at `mid`, the answer is at `mid` or to its left: `hi = mid`.
3. Otherwise, it is to the right: `lo = mid + 1`.
4. When the loop ends, `lo == hi` is the answer.

Because `mid` is rounded down and `lo` moves past it, the range always shrinks, so the loop never runs forever. If you need the *last* position where something is true, round `mid` up instead, as in Problem 10.5.

10.1 Find in a Sorted Catalog (Easy)

Return the position of a value in a sorted list of distinct values, or -1.

```
>>> find_sorted([2, 5, 8, 12, 16, 23], 12)
3
>>> find_sorted([2, 5, 8], 6)
-1
>>> find_sorted([], 1)
-1
```

```
def find_sorted(items, target):
    lo, hi = 0, len(items) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if items[mid] == target:
            return mid
        if items[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Complexity: O(log n) time, O(1) space.

> **Common Mistake:** Writing `while lo < hi` with this version, which searches for an exact match. With `lo < hi`, a one-item range is never examined, so the item at the edge can be missed. Match your loop condition to your template.

10.2 First Bad Build (Easy)

Builds are numbered 1 to n. Once a build is bad, every later build is bad too. You have a function `is_bad(build)`; each call is expensive. Find the first bad build (or -1 if none) with as few calls as possible.

```
>>> find_first_bad(10, 7)
7
>>> find_first_bad(1, 1)
1
>>> find_first_bad(5, 99)
-1
```

The examples use a small wrapper that makes every build from `first` onward bad.

The idea. This is exactly the "first true" template. Including n + 1 in the range as a "none" answer avoids a special case.

```
def first_bad(n, is_bad):
    lo, hi = 1, n + 1  # the answer is in [lo, hi]; n + 1 means "none"
    while lo < hi:
        mid = (lo + hi) // 2
        if is_bad(mid):
            hi = mid  # mid might be the first bad build
        else:
            lo = mid + 1  # the first bad build is after mid
    return lo if lo <= n else -1
```

Complexity: O(log n) calls. The test for this problem checks that a million builds need at most 21 calls.

10.3 Rotated Catalog (Medium)

A sorted list of distinct values was rotated at some unknown point, for example `[9, 12, 1, 4, 7]`. Find a target's position in O(log n), or return -1.

```
>>> rotated_search([9, 12, 1, 4, 7], 4)
3
>>> rotated_search([9, 12, 1, 4, 7], 10)
-1
>>> rotated_search([5], 5)
0
>>> rotated_search([], 3)
-1
```

Think first: After a rotation, at least one half of any range is still sorted. How can you tell which?

The idea. Compare the middle with the left end. If `items[lo] <= items[mid]`, the left half is sorted, and you can check directly whether the target lies in it; otherwise the right half is sorted. Search in the half that could contain the target.

```
def rotated_search(items, target):
    lo, hi = 0, len(items) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if items[mid] == target:
            return mid
        if items[lo] <= items[mid]:  # left half is sorted
            if items[lo] <= target < items[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:  # right half is sorted
            if items[mid] < target <= items[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

Complexity: O(log n) time, O(1) space.

> **Follow-Up:** "With duplicates allowed?" When `items[lo] == items[mid] == items[hi]`, you can't tell which half is sorted; shrink both ends by one. The worst case becomes O(n).

10.4 Truck Capacity (Medium)

Parcels with given weights must be shipped in order over a number of days, one truckload per day; a truckload can't exceed the truck's capacity. What is the smallest capacity that ships everything in time?

```
>>> truck_capacity([4, 8, 2, 6, 5, 3, 7, 1], 3)
13
>>> truck_capacity([5, 5, 5, 5], 2)
10
>>> truck_capacity([7], 1)
7
```

Think first: If a capacity works, does every bigger capacity work too?

The idea. Yes: more capacity never hurts. So the answer is the first capacity for which "can ship in time" is true, which is binary search *on the answer*. The capacity must be at least the heaviest parcel and at most the total weight. Each guess is checked by a greedy simulation that fills each day as much as possible.

```
def days_needed(parcels, capacity):
    days, load = 1, 0
    for p in parcels:
        if load + p > capacity:
            days += 1
            load = 0
        load += p
    return days
```

```
def truck_capacity(parcels, days):
    lo, hi = max(parcels), sum(parcels)  # the answer is in this range
    while lo < hi:
        mid = (lo + hi) // 2
        if days_needed(parcels, mid) <= days:
            hi = mid  # mid works; maybe something smaller does too
        else:
            lo = mid + 1
    return lo
```

Complexity: O(n log S), where S is the total weight: O(log S) guesses, each checked in O(n).

> **Interviewer's View:** Recognizing "binary search on the answer" is the whole problem. The tell-tale signs are "smallest capacity such that..." and a check that is easy for a given guess.

10.5 Integer Square Root (Easy)

Return the largest integer r with r × r ≤ n, without using square-root functions or powers.

```
>>> integer_root(0)
0
>>> integer_root(1)
1
>>> integer_root(8)
2
>>> integer_root(9)
3
>>> integer_root(1000000000000)
1000000
```

The idea. We want the *last* r for which `r * r <= n` is true. Use the template with `mid` rounded *up*, so that `lo = mid` always makes progress.

```
def integer_root(n):
    lo, hi = 0, n
    while lo < hi:
        mid = (lo + hi + 1) // 2  # round up so the loop always moves
        if mid * mid <= n:
            lo = mid
        else:
            hi = mid - 1
    return lo
```

Complexity: O(log n) time. Python's `math.isqrt` does the same job, and the tests compare the two on random numbers up to a billion.

> **Common Mistake:** Rounding `mid` down when the update is `lo = mid`. With `lo = 2, hi = 3`, mid would be 2 forever: an infinite loop.

10.6 Median of Two Sorted Lists (Hard)

Find the median of the combined values of two sorted lists, in O(log(min(m, n))).

```
>>> merged_median([2, 9], [4])
4.0
>>> merged_median([1, 5], [6, 8])
5.5
>>> merged_median([], [5])
5.0
>>> merged_median([4, 4], [4])
4.0
```

Think first: The median splits the combined values into a left half and a right half. If you take i values from the first list for the left half, how many must come from the second?

The idea. Binary search on i, the number of values taken from the shorter list `a`; the second list must supply `half - i`. The split is correct when everything on the left is less than or equal to everything on the right, that is, `a_left <= b_right` and `b_left <= a_right`. If `a_left` is too big, take fewer from `a`; otherwise, take more. Infinite values stand in for "nothing on that side".

```
def merged_median(a, b):
    if len(a) > len(b):
        a, b = b, a  # binary search the shorter list
    m, n = len(a), len(b)
    half = (m + n + 1) // 2  # size of the left half
    lo, hi = 0, m
    while lo <= hi:
        i = (lo + hi) // 2  # take i items from a, half - i from b
        j = half - i
        a_left = a[i - 1] if i > 0 else float("-inf")
        a_right = a[i] if i < m else float("inf")
        b_left = b[j - 1] if j > 0 else float("-inf")
        b_right = b[j] if j < n else float("inf")
        if a_left <= b_right and b_left <= a_right:
            if (m + n) % 2:
                return float(max(a_left, b_left))
            return (max(a_left, b_left) + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1  # took too many from a
        else:
            lo = i + 1  # took too few from a
    raise ValueError("inputs must be sorted and not both empty")
```

Complexity: O(log(min(m, n))) time, O(1) space.

> **Tip:** This is one of the hardest standard problems. In an interview, it's fine to start with the O(m + n) merge (Problem 6.5) and then explain the partition idea before coding it.

Key Takeaways

- Binary search halves the search range each step: O(log n).
- Use one template consistently: find the first position where a condition becomes true, with the range always containing the answer.
- Round `mid` up when searching for the last true position, to avoid infinite loops.
- "Binary search on the answer" solves many "smallest value that works" problems, combined with a fast check.

CHAPTER 11

Linked Lists

A linked list stores items in nodes, each holding a value and a reference to the next node. Unlike a Python list, it can't jump to the 50th item; you must follow the links one by one. Interviewers like linked lists because they test careful pointer manipulation: the code is short, but one link changed in the wrong order loses the rest of the list.

The Building Blocks

The problems in this chapter use a minimal node class, and two helpers that build a linked list from a Python list and read it back:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def build_list(values):
    """Build a linked list from a Python list; return its head."""
    head = None
    for v in reversed(values):
        head = ListNode(v, head)
    return head


def list_values(head, limit=10_000):
    """The values of a linked list, as a Python list."""
    out = []
    while head and len(out) < limit:
        out.append(head.val)
        head = head.next
    return out
```

Each example calls a small wrapper, such as `reversed_values`, which builds a linked list from the Python list you see, runs the solution, and returns the resulting values as a list.

When to Use the Techniques

- **A dummy head node** removes special cases when the head itself might change (Problems 11.3 and 11.4).
- **Two pointers at different speeds** ("slow and fast") find the middle or detect a cycle in one pass, with O(1) memory (Problems 11.2 and 11.5).
- **Two pointers with a gap** find the k-th node from the end in one pass (Problem 11.4).
- **Reversing in place** is a building block for many harder problems (Problems 11.1 and 11.5).

> **Tip:** Draw the nodes and arrows. Before changing any `next` pointer, ask: "Do I still have a way to reach the node it used to point to?"

11.1 Reverse a Linked List (Easy)

Reverse a linked list and return its new head.

```
>>> reversed_values([1, 2, 3, 4])
[4, 3, 2, 1]
>>> reversed_values([])
[]
>>> reversed_values([7])
[7]
```

The idea. Walk the list, turning each node's `next` pointer around to point at the previous node. Save the next node first, or you lose the rest of the list.

```
def reverse_list(head):
    prev = None
    while head:
        nxt = head.next  # remember the rest before we cut the link
        head.next = prev
        prev = head
        head = nxt
    return prev
```

Complexity: O(n) time, O(1) space.

A recursive version is elegant but uses O(n) stack space, and fails for lists longer than Python's recursion limit:

```
def reverse_list_recursive(head):
    if head is None or head.next is None:
        return head
    new_head = reverse_list_recursive(head.next)
    head.next.next = head  # the node after us now points back to us
    head.next = None
    return new_head
```

11.2 Detect a Loop (Easy)

Does following `next` pointers from the head ever reach the same node twice?

```
>>> loop_exists([5, 8, 1, 6], 2)
True
>>> loop_exists([4, 9], -1)
False
>>> loop_exists([1], 0)
True
>>> loop_exists([], -1)
False
```

In the examples, the second argument is the position the last node links back to, or -1 for no loop.

The easy idea. Store each node in a set; a repeat means a loop. O(n) space.

The better idea. Floyd's "tortoise and hare": move one pointer one step at a time and another two steps. If there is a loop, the fast pointer eventually laps the slow one and they meet; if not, the fast pointer reaches the end.

```
def has_loop(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next  # one step
        fast = fast.next.next  # two steps
        if slow is fast:
            return True
    return False
```

Complexity: O(n) time, O(1) space.

> **Follow-Up:** "Where does the loop start?" After the pointers meet, move one pointer back to the head and advance both one step at a time; they meet at the start of the loop.

11.3 Merge Two Sorted Linked Lists (Easy)

Merge two sorted linked lists into one sorted list by relinking the existing nodes.

```
>>> merged_values([1, 4, 6], [2, 4, 5])
[1, 2, 4, 4, 5, 6]
>>> merged_values([], [])
[]
```

The idea. A dummy node in front of the result means you never need a special case for the first node. Repeatedly attach the smaller current node; when one list ends, attach the rest of the other.

```
def merge_lists(a, b):
    dummy = ListNode()  # a fixed node before the head saves special cases
    tail = dummy
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

Complexity: O(m + n) time, O(1) extra space.

11.4 Remove the k-th Node From the End (Medium)

Remove the k-th node from the end of a list (k = 1 is the last node) in a single pass.

```
>>> removed_values([6, 7, 8, 9], 3)
[6, 8, 9]
>>> removed_values([1], 1)
[]
>>> removed_values([1, 2], 2)
[2]
```

The idea. Start two pointers at a dummy node and move one of them k steps ahead. Then move both until the leading one reaches the last node: the trailing one is now just before the node to remove. The dummy node handles removing the head.

```
def remove_from_end(head, k):
    dummy = ListNode(0, head)
    lead = trail = dummy
    for _ in range(k):  # put lead k nodes ahead of trail
        lead = lead.next
    while lead.next:
        lead = lead.next
        trail = trail.next
    trail.next = trail.next.next  # trail is just before the target
    return dummy.next
```

Complexity: O(n) time, O(1) space.

11.5 Interleave a List (Medium)

Reorder a list L0, L1, ..., Ln into L0, Ln, L1, Ln-1, L2, ... by relinking nodes, in O(1) extra space.

```
>>> interleaved_values([1, 2, 3, 4, 5])
[1, 5, 2, 4, 3]
>>> interleaved_values([1, 2, 3, 4])
[1, 4, 2, 3]
>>> interleaved_values([])
[]
>>> interleaved_values([9])
[9]
```

Think first: The new order alternates between the front half and the *reversed* back half.

The idea. Three steps you already know: find the middle with slow and fast pointers; reverse the second half; then weave the two halves together.

```
def interleave(head):
    if head is None:
        return head
    # 1. Find the middle.
    slow = fast = head
    while fast.next and fast.next.next:
        slow, fast = slow.next, fast.next.next
    # 2. Reverse the second half.
    prev, cur = None, slow.next
    slow.next = None
    while cur:
```

```
        cur.next, prev, cur = prev, cur, cur.next
    # 3. Weave the two halves together.
    first, second = head, prev
    while second:
        first.next, first = second, first.next
        second.next, second = first, second.next
    return head
```

Complexity: O(n) time, O(1) space.

> **Interviewer's View:** Saying "this is three problems I know: middle, reverse, merge" before coding is a strong signal. It shows you decompose problems instead of improvising pointer moves.

11.6 Merge k Sorted Linked Lists (Hard)

Merge k sorted linked lists into one sorted list.

```
>>> merged_k_values([[2, 7, 9], [1, 8], [3, 4, 10]])
[1, 2, 3, 4, 7, 8, 9, 10]
>>> merged_k_values([])
[]
>>> merged_k_values([[], [2]])
[2]
```

Brute force. Collect all values, sort them and build a new list: O(N log N) for N nodes. Merging the lists one by one is O(N × k).

The better idea. Keep a min-heap of the current head of each list. Repeatedly pop the smallest node, attach it, and push its successor. The heap never holds more than k nodes. A position `i` is stored as a tie-breaker, because nodes themselves can't be compared.

```
def merge_k(heads):
    heap = []
    for i, node in enumerate(heads):
        if node:
            heap.append((node.val, i, node))  # i breaks ties between nodes
    heapq.heapify(heap)
    dummy = tail = ListNode()
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

Complexity: O(N log k) time, O(k) extra space.

> **Follow-Up:** A divide-and-conquer approach merges the lists in pairs, then pairs of pairs, also in O(N log k), without a heap.

Key Takeaways

- Save the next node before changing a pointer; draw the arrows.
- A dummy head node removes special cases at the front of the list.
- Slow and fast pointers find middles and cycles in O(1) space; a gap of k finds the k-th from the end.
- Harder problems combine simple steps: find the middle, reverse, merge.

CHAPTER 12

Trees and Binary Search Trees

A binary tree is a set of nodes in which each node has at most two children, a left one and a right one, and every node except the root has exactly one parent. Trees model hierarchies such as file systems, organization charts and the structure of expressions, and they are the most common subject of medium-level interview questions. Almost every tree problem is solved by one of two traversals, and by asking one question: *what does each node need from its children, and what must it return to its parent?*

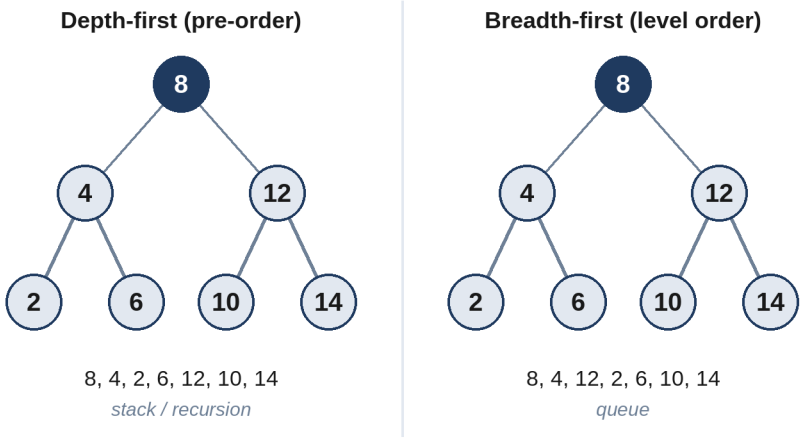
The Building Blocks

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

The examples describe trees in level order, with `None` for missing children, the format most interview platforms use. `[8, 4, 12, None, 6, 10]` is a root 8 with children 4 and 12; 4 has no left child and a right child 6; 12 has a left child 10. Two helpers, `build_tree` and `tree_values`, convert between this format and nodes.

When to Use Which Traversal

- **Depth-first search (DFS)**, usually recursive, follows one branch to the bottom before trying the next. Use it when each node's answer depends on its subtrees: heights, sums, paths, validity.
- **Breadth-first search (BFS)**, with a queue, visits the tree level by level. Use it for anything about levels: level lists, the shallowest leaf, the rightmost node of each level.
- **In-order DFS** (left, node, right) visits a binary search tree's values in sorted order.



Depth-first search goes down one branch before the next; breadth-first search visits the tree one level at a time, using a queue.

The recursive DFS recipe: handle the empty tree; ask the left and right subtrees for their answers; combine them with the current node; return the result to the parent. If the final answer isn't what you return to the parent, as in Problem 12.7, keep it in a separate variable.

12.1 Tree Depth (Easy)

How many nodes are on the longest path from the root down to a leaf?

```
>>> tree_depth([8, 4, 12, None, 6, 10])
3
>>> tree_depth([])
0
>>> tree_depth([1, None, 2, None, 3])
3
```

The idea. The depth of a tree is one more than the deeper of its two subtrees; an empty tree has depth 0.

```
def depth(root):
    if root is None:
        return 0
    return 1 + max(depth(root.left), depth(root.right))
```

Complexity: O(n) time; O(h) space for the recursion, where h is the height (O(n) for a tree shaped like a list).

A level-by-level BFS counts levels instead, and avoids recursion:

```
def depth_iterative(root):
    if root is None:
        return 0
    levels, queue = 0, deque([root])
    while queue:
        levels += 1
        for _ in range(len(queue)):  # one whole level at a time
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return levels
```

12.2 Mirror a Tree (Easy)

Swap the left and right children of every node.

```
>>> mirrored([5, 3, 9, 1, 4, 8, 11])
[5, 9, 3, 11, 8, 4, 1]
>>> mirrored([])
[]
>>> mirrored([1, 2])
[1, None, 2]
```

```
def mirror(root):
    if root:
        root.left, root.right = mirror(root.right), mirror(root.left)
    return root
```

Complexity: O(n) time, O(h) space.

> **Common Mistake:** Writing `root.left = mirror(root.right)` and then `root.right = mirror(root.left)` on separate lines. By the second line, `root.left` has already been replaced. Python's simultaneous assignment evaluates both right-hand sides first.

12.3 Level by Level (Medium)

Return the values of a tree as a list of levels, each from left to right.

```
>>> level_lists([8, 4, 12, None, 6, 10])
[[8], [4, 12], [6, 10]]
>>> level_lists([])
[]
```

The idea. BFS with a queue. At the start of each round, the queue holds exactly one level; process that many nodes, collecting their values and queueing their children.

```
def levels(root):
    if root is None:
        return []
    out, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        out.append(level)
    return out
```

Complexity: O(n) time, O(w) space, where w is the widest level.

> **Follow-Up:** "Zigzag order" (alternate directions per level) or "the rightmost value of each level" are small changes to the same loop.

12.4 Valid Search Tree (Medium)

In a binary search tree, every value in a node's left subtree is smaller than the node, and every value in its right subtree is larger. Check whether a tree satisfies this, with no duplicate values allowed.

```
>>> is_search_tree([5, 3, 8, 1, 4, 7, 9])
True
>>> is_search_tree([5, 3, 8, 1, 6])
False
>>> is_search_tree([])
True
>>> is_search_tree([2, 2])
False
```

Think first: Is it enough to check that each node is bigger than its left child and smaller than its right child?

The idea. No: in the second example, 6 is the right child of 3, which is fine locally, but 6 is in the left subtree of 5, where everything must be smaller than 5. So pass down the allowed range: going left lowers the upper bound to the node's value; going right raises the lower bound.

```
def valid_bst(root, low=float("-inf"), high=float("inf")):
    if root is None:
        return True
    if not low < root.val < high:
        return False
    return valid_bst(root.left, low, root.val) and valid_bst(
        root.right, root.val, high
    )
```

Complexity: O(n) time, O(h) space.

An equally good alternative: an in-order traversal must produce strictly increasing values.

12.5 Closest Common Manager (Medium)

In a company's reporting tree, every person has a distinct ID. Given two people, a and b, find their lowest common manager: the deepest node that has both in its subtree. A person counts as being in their own subtree.

```
>>> common_manager([6, 2, 8, 0, 4, 7, 9, None, None, 3, 5], 2, 8)
6
>>> common_manager([6, 2, 8, 0, 4, 7, 9, None, None, 3, 5], 2, 5)
2
>>> common_manager([1, 2], 1, 2)
1
```

Think first: If a is somewhere in the left subtree and b in the right subtree of a node, what is that node?

The idea. Search both subtrees. Return a node upwards when it is a or b, or when it is the meeting point. If both sides return something, the current node is the answer; otherwise pass up whichever side found something.

```
def common_ancestor(root, a, b):
    if root is None or root.val in (a, b):
        return root
    left = common_ancestor(root.left, a, b)
    right = common_ancestor(root.right, a, b)
    if left and right:
        return root  # a and b are on different sides
    return left or right
```

Complexity: O(n) time, O(h) space.

> **Follow-Up:** In a *binary search tree*, you can use the values instead: while both values are smaller than the node, go left; while both are larger, go right; otherwise the current node is the answer, in O(h).

12.6 Save and Restore a Tree (Hard)

Write `serialize`, which turns a binary tree into a string, and `deserialize`, which rebuilds the identical tree from the string.

```
>>> save_and_restore([7, 3, 9, None, 5, 8])
[7, 3, 9, None, 5, 8]
>>> save_and_restore([])
[]
>>> save_and_restore([-7])
[-7]
```

The idea. A pre-order traversal (node, left, right) with a marker such as # for every missing child describes the tree completely. Deserializing reads the tokens in the same order, and each recursive call builds exactly the subtree that was written at that point.

```
def serialize(root):
    out = []

    def walk(node):
        if node is None:
            out.append("#")
            return
        out.append(str(node.val))
        walk(node.left)
        walk(node.right)

    walk(root)
    return ",".join(out)


def deserialize(text):
    tokens = iter(text.split(","))

    def build():
        tok = next(tokens)
        if tok == "#":
            return None
        node = TreeNode(int(tok))
        node.left = build()
        node.right = build()
        return node

    return build()
```

Complexity: O(n) time and space for both.

12.7 Best Path (Hard)

A path is any sequence of nodes connected by edges, without repeating a node; it may start and end anywhere. Find the largest possible sum of node values along a path. Values may be negative.

```
>>> best_path_sum([-6, 4, 9, None, None, 7, 5])
21
>>> best_path_sum([2, -1, 3])
5
>>> best_path_sum([-3])
-3
```

Think first: A path has a highest node, where it "bends". What does the path look like below that node?

The idea. Every path bends at exactly one node, and consists of that node plus the best downward path into its left subtree, plus the best into its right subtree (or nothing, if those are negative). A recursive helper returns the best *downward* path from each node, which is what its parent can use, while the best *bending* path at each node updates a separate answer.

```
def best_path(root):
    best = float("-inf")

    def gain(node):
        """Best sum of a path that starts at node and goes down."""
        nonlocal best
        if node is None:
            return 0
        left = max(gain(node.left), 0)  # ignore negative branches
        right = max(gain(node.right), 0)
        best = max(best, node.val + left + right)  # path bends here
        return node.val + max(left, right)

    gain(root)
    return best
```

Complexity: O(n) time, O(h) space.

> **Interviewer's View:** The crucial insight is the difference between what a node *returns* (a downward path, which can extend upwards) and what it *records* (a bending path, which can't). Say it explicitly.

Key Takeaways

- Most tree problems are DFS: get answers from the subtrees, combine, return.
- Use BFS with a queue for anything about levels.

- In a binary search tree, pass down allowed ranges, or use the sorted in-order sequence.
- When the overall answer differs from what a node returns to its parent, track it separately.
- Recursion depth equals tree height; mention the O(h) space and the risk with very deep trees.

CHAPTER 13

Heaps and Top-K

A heap gives you the smallest (or largest) item of a changing collection in O(1), and lets you add or remove items in O(log n). That makes it the natural tool for "top k", "k-th largest", "merge many sorted things" and "always process the most urgent item next". Python's `heapq` module provides a min-heap stored in an ordinary list (Chapter 4).

When to Use It

- "Find the **k largest**, **k smallest**, **k most frequent** or **k closest**."
- "Find the **k-th** largest or smallest."
- **Repeatedly take the best** remaining option: scheduling, merging sorted streams, shortest paths (Chapter 17).
- Maintain a **running median** or other order statistic as data arrives.

A useful trick for top-k: to keep the k *largest* items, use a *min*-heap of size k. Its smallest item is the weakest of the current top k, and any new item that beats it replaces it. That costs O(n log k) instead of O(n log n) for sorting everything, and O(k) memory.

13.1 Kth Largest Score (Easy)

Find the k-th largest value in a list of scores (duplicates count separately).

```
>>> kth_largest([71, 88, 64, 93, 88, 50], 3)
88
>>> kth_largest([3, 2, 3, 1, 2, 4, 5, 5, 6], 4)
4
>>> kth_largest([7], 1)
7
```

The simple idea. Sort in descending order and take position k - 1: O(n log n).

The better idea. Keep a min-heap of the k largest scores seen so far; at the end, its smallest item is the k-th largest.

```
def kth_largest(scores, k):
    heap = []  # the k largest so far; heap[0] is the smallest of them
    for s in scores:
        if len(heap) < k:
            heapq.heappush(heap, s)
        elif s > heap[0]:
            heapq.heapreplace(heap, s)
    return heap[0]
```

Complexity: O(n log k) time, O(k) space.

> **Follow-Up:** "Can you do it in O(n) on average?" Yes, with *quickselect*: partition around a pivot as in quicksort, and recurse only into the side that contains position k.

13.2 Most Common Tags (Medium)

Return the k most frequent tags, most frequent first; break ties alphabetically.

```
>>> top_k(['db', 'api', 'db', 'ui', 'api', 'db'], 2)
['db', 'api']
>>> top_k(['b', 'a'], 2)
['a', 'b']
```

The idea. Count with a `Counter`, then select the k best by the key `(-count, tag)`: a higher count first, then alphabetical. `heapq.nsmallest` does this in O(n log k).

```
def top_k(tags, k):
    counts = Counter(tags)
    return heapq.nsmallest(k, counts, key=lambda t: (-counts[t], t))
```

Complexity: O(n + u log k) for n tags with u distinct values.

An alternative uses "bucket sort" by count, which avoids the log factor:

```
def top_k_buckets(tags, k):
    counts = Counter(tags)
    buckets = [[] for _ in range(len(tags) + 1)]  # count -> tags
    for tag, c in counts.items():
        buckets[c].append(tag)
    out = []
    for c in range(len(buckets) - 1, 0, -1):
        for tag in sorted(buckets[c]):
            if len(out) == k:
                return out
            out.append(tag)
    return out
```

13.3 Closest Stations (Medium)

Given stations at points (x, y), return the k closest to the origin, sorted by distance and then by coordinates.

```
>>> closest([(1, 3), (-2, 2), (5, -1)], 1)
[(-2, 2)]
>>> closest([(3, 3), (5, -1), (-2, 4)], 2)
[(3, 3), (-2, 4)]
```

The idea. Keep a max-heap of the k closest so far, by storing negated keys. If a new station is closer than the farthest one kept, replace it. Compare squared distances: no square roots needed, and no rounding errors.

```
def closest(stations, k):
    heap = []  # max-heap of the k closest, using negated keys
    for x, y in stations:
        key = (-(x * x + y * y), -x, -y)
        if len(heap) < k:
            heapq.heappush(heap, key)
        elif key > heap[0]:  # closer than the farthest kept station
            heapq.heapreplace(heap, key)
    result = [(-nx, -ny) for _, nx, ny in heap]
    return sorted(result, key=lambda p: (p[0] ** 2 + p[1] ** 2, p))
```

Complexity: O(n log k) time, O(k) space.

13.4 Running Median (Hard)

Readings arrive one at a time. After each one, report the median of all readings so far.

```
>>> running_median([5, 15, 1, 3])
[5.0, 10.0, 5.0, 4.0]
>>> running_median([])
[]
```

Think first: The median sits between the smaller half and the larger half. Which items of each half do you need quick access to?

The idea. Keep two heaps: a max-heap `low` for the smaller half and a min-heap `high` for the larger half, with `low` allowed one extra item. Each new reading passes through `low` into `high`, which keeps the halves in order, and then the sizes are rebalanced. The median is the top of `low`, or the average of both tops.

```
def running_median(readings):
    low = []  # max-heap (negated) holding the smaller half
    high = []  # min-heap holding the larger half
    out = []
    for r in readings:
        heapq.heappush(low, -r)
        heapq.heappush(high, -heapq.heappop(low))  # keep halves ordered
        if len(high) > len(low):
            heapq.heappush(low, -heapq.heappop(high))  # low may be +1
        if len(low) > len(high):
            out.append(float(-low[0]))
        else:
            out.append((-low[0] + high[0]) / 2)
    return out
```

Complexity: O(log n) per reading, O(n) space.

13.5 Cooldown Scheduler (Medium)

A machine runs tasks of several types, one task per time unit. Two tasks of the same type must be at least n units apart; the machine may sit idle. What is the minimum total time?

```
>>> cooldown(['X', 'X', 'X', 'Y', 'Z', 'Z'], 3)
9
>>> cooldown(['X', 'X', 'X', 'Y', 'Z', 'Z'], 0)
6
>>> cooldown(['A', 'B', 'C', 'A'], 3)
5
```

The simulation. Each step, run the task type with the most remaining tasks that isn't cooling down; a heap finds it, and a queue holds types waiting out their cooldown. This works and generalizes to variations.

```
def cooldown_simulate(tasks, n):
    heap = [-c for c in Counter(tasks).values()]  # max-heap of counts
    heapq.heapify(heap)
    waiting = deque()  # (time it can run again, negated remaining count)
    time = 0
    while heap or waiting:
        time += 1
        if heap:
            remaining = heapq.heappop(heap) + 1  # run the most common
            if remaining:
                waiting.append((time + n, remaining))
        if waiting and waiting[0][0] == time:
            heapq.heappush(heap, waiting.popleft()[1])
    return time
```

The formula. The most frequent type dictates the shape: with `most` tasks of that type, there are `most - 1` full blocks of length n + 1, plus a final row containing every type that ties for the maximum count. If there are so many other tasks that no idle time is needed, the answer is simply the number of tasks.

```
def cooldown(tasks, n):
    counts = Counter(tasks).values()
    most = max(counts)
    with_most = sum(1 for c in counts if c == most)
    # (most - 1) full blocks of length n + 1, then the last row.
    return max(len(tasks), (most - 1) * (n + 1) + with_most)
```

Complexity: O(n) for the formula. The tests compare the formula with the simulation on thousands of random cases.

13.6 No Two Alike (Medium)

Rearrange the letters of a string so that no two neighboring letters are equal. Return any valid arrangement, or "" if it's impossible.

```
>>> no_neighbors('ppa')
'pap'
>>> no_neighbors('pppa')
''
>>> no_neighbors('')
''
```

Think first: Which letter is most at risk of ending up next to itself?

The idea. Greedily place the most frequent remaining letter, but never the letter just placed: hold it back for one step, then return it to the heap. If the held letter can never be placed, the arrangement is impossible, which happens exactly when one letter fills more than half the positions (rounded up).

```
def no_neighbors(s):
    heap = [(-c, ch) for ch, c in Counter(s).items()]
    heapq.heapify(heap)
    out = []
    held = None  # the letter used last step can't be used right away
    while heap:
        count, ch = heapq.heappop(heap)
        out.append(ch)
        if held:
            heapq.heappush(heap, held)
        held = (count + 1, ch) if count + 1 < 0 else None
    return "" if held else "".join(out)
```

Complexity: O(n log k) for k distinct letters.

> **Tip:** When several answers are valid, the tests in this book check the *properties* of the answer, the same letters and no equal neighbors, rather than one exact string. In an interview, say how you would test it the same way.

Key Takeaways

- A heap gives the minimum in O(1) and updates in O(log n); negate values for a max-heap.
- For the top k of n items, keep a heap of size k: O(n log k).
- Two heaps maintain a running median.
- Greedy scheduling problems often mean "always take the best option a heap can give you", with a queue for items that are temporarily unavailable.

CHAPTER 14

Intervals and Greedy Choices

Interval problems are about ranges of time or space: meetings, bookings, shifts, reservations. Nearly all of them start the same way: **sort the intervals**, by start or by end, and then make one pass. Greedy problems, more generally, are those where making the locally best choice at each step leads to the best overall answer. The hard part is not the code, which is usually short, but convincing yourself, and the interviewer, that the greedy choice is safe.

When to Use It

- **Merging**, **inserting**, or **counting overlaps** of intervals.
- **Scheduling**: the most non-overlapping events, the fewest rooms, the fewest removals.
- "Can you reach the end?", "Which starting point works?": problems where a single running value (the farthest reachable point, a balance) captures everything.

Decide which end to sort by:

- Sort by **start** to merge overlapping intervals or to process them in time order.
- Sort by **end** to choose the most intervals that don't overlap: always keep the one that finishes first, since it leaves the most room for the rest.

Check the meaning of "overlap" with the interviewer: do [1, 3] and [3, 5] overlap? For closed intervals, yes; for half-open meetings like [1:00, 3:00), no.

14.1 Merge Bookings (Medium)

Combine overlapping bookings `[start, end]` into the smallest set of non-overlapping ranges. Bookings that touch, such as [1, 4] and [4, 5], are merged.

```
>>> merge_bookings([[1, 3], [8, 10], [2, 6], [15, 18]])
[[1, 6], [8, 10], [15, 18]]
>>> merge_bookings([[1, 4], [4, 5]])
[[1, 5]]
>>> merge_bookings([])
[]
```

The idea. Sort by start. Each booking either overlaps the last merged range, so extend it, or starts after it, so begin a new range.

```
def merge_bookings(bookings):
    merged = []
    for start, end in sorted(bookings):
        if merged and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)  # extend the last one
        else:
            merged.append([start, end])
    return merged
```

Complexity: O(n log n) time for sorting, O(n) space for the result.

> **Common Mistake:** Setting the merged end to the new booking's end instead of the maximum of the two. [1, 10] followed by [2, 3] must stay [1, 10].

14.2 Insert a Booking (Medium)

Insert a new booking into a sorted list of non-overlapping bookings, merging where necessary.

```
>>> insert_booking([[1, 3], [6, 9]], [2, 5])
[[1, 5], [6, 9]]
>>> insert_booking([[1, 2], [3, 5], [6, 7], [8, 10]], [4, 8])
[[1, 2], [3, 10]]
>>> insert_booking([], [4, 8])
[[4, 8]]
```

The idea. The list is already sorted, so no sorting is needed: copy the bookings that end before the new one starts, absorb every booking that overlaps it, then copy the rest.

```
def insert_booking(bookings, new):
    start, end = new
    out = []
    i = 0
    while i < len(bookings) and bookings[i][1] < start:
        out.append(bookings[i])  # entirely before the new booking
        i += 1
    while i < len(bookings) and bookings[i][0] <= end:
        start = min(start, bookings[i][0])  # overlaps: absorb it
        end = max(end, bookings[i][1])
        i += 1
    out.append([start, end])
    out.extend(bookings[i:])  # entirely after
    return out
```

Complexity: O(n) time.

14.3 Rooms Needed (Medium)

Meetings run over half-open times `[start, end)`. What is the fewest number of rooms needed so that no two meetings in the same room overlap?

```
>>> rooms_needed([[9, 12], [10, 11], [11, 13], [12, 14]])
2
>>> rooms_needed([[7, 10], [2, 4]])
1
>>> rooms_needed([[1, 5], [5, 9]])
1
>>> rooms_needed([])
0
```

The idea. Process meetings by start time, keeping a min-heap of the end times of meetings in progress. If the earliest-ending meeting has finished by the time the next one starts, reuse its room; otherwise open a new one. The heap's final size is the number of rooms.

```
def rooms_needed(meetings):
    ends = []  # end times of meetings in progress
    for start, end in sorted(meetings):
        if ends and ends[0] <= start:
            heapq.heapreplace(ends, end)  # reuse the room freed earliest
        else:
            heapq.heappush(ends, end)
    return len(ends)
```

An equally good "sweep line" turns each meeting into a +1 event at its start and a -1 at its end, and tracks the running total; sorting puts an end before a start at the same time, so back-to-back meetings share a room.

```
def rooms_sweep(meetings):
    events = []
    for s, e in meetings:
        events.append((s, 1))
        events.append((e, -1))  # -1 sorts before +1 at the same time
    best = cur = 0
    for _, change in sorted(events):
        cur += change
        best = max(best, cur)
    return best
```

Complexity: O(n log n) time for both.

14.4 Fewest Cancellations (Medium)

Cancel as few bookings `[start, end)` as possible so that the rest don't overlap.

```
>>> fewest_cancellations([[1, 2], [2, 3], [3, 4], [1, 3]])
1
>>> fewest_cancellations([[1, 2], [1, 2], [1, 2]])
2
>>> fewest_cancellations([])
0
```

Think first: Among all bookings, which one should you definitely keep?

The idea. Keep the booking that ends earliest: any solution can swap its first booking for this one without creating an overlap, so keeping it is never worse. Then repeat with the bookings that start after it. Sorting by end time makes this one pass.

```
def fewest_cancellations(bookings):
    kept_end = float("-inf")
    canceled = 0
    for start, end in sorted(bookings, key=lambda b: b[1]):
        if start >= kept_end:
            kept_end = end  # keep the booking that finishes earliest
        else:
            canceled += 1
    return canceled
```

Complexity: O(n log n) time, O(1) extra space.

> **Interviewer's View:** That one-sentence "exchange argument" (swapping in the earliest-ending booking never hurts) is the proof the interviewer is looking for. Practice saying it.

14.5 Reach the Last Stone (Medium)

Stones are in a row; from stone i you may jump forward up to `hops[i]` stones. Starting on the first stone, can you reach the last?

```
>>> reach_last([1, 3, 0, 1, 2, 0])
True
>>> reach_last([2, 1, 0, 3])
False
>>> reach_last([0])
True
```

The idea. Track the farthest stone reachable so far. If you ever stand on a stone beyond it, it can't be reached, and neither can anything after it.

```
def reach_last(hops):
    farthest = 0
    for i, h in enumerate(hops):
        if i > farthest:
            return False  # this stone can't be reached
        farthest = max(farthest, i + h)
    return True
```

Complexity: O(n) time, O(1) space, compared with $O(n^2)$ for the dynamic-programming solution used in the tests.

> **Follow-Up:** "What is the fewest number of jumps?" Treat it as a breadth-first search in levels: the stones reachable with one jump, then with two, tracking the farthest point of each level, still in O(n).

14.6 Charging Loop (Medium)

A delivery robot drives around a loop of charging stations. At station i it gains `charge[i]` and then spends `cost[i]` to reach the next station. From which station can it start, with an empty battery, and complete the loop without ever going below zero? Return the smallest such index, or -1.

```
>>> charging_loop([2, 6, 1, 4], [3, 2, 5, 2])
1
>>> charging_loop([2, 3, 4], [3, 4, 3])
-1
>>> charging_loop([5], [5])
0
```

Think first: If the robot starts at station s and first runs out of charge reaching station i + 1, could any station between s and i be a valid start?

The idea. If the total charge is less than the total cost, no start works. Otherwise, run once around: whenever the battery goes negative after station i, no station from the current start to i can be the answer (each would arrive at i with even less charge), so restart from i + 1. The last start chosen is the answer.

```
def charging_loop(charge, cost):
    if sum(charge) < sum(cost):
        return -1
    start, battery = 0, 0
    for i in range(len(charge)):
        battery += charge[i] - cost[i]
        if battery < 0:
            # No station from start to i can work; try the next one.
            start, battery = i + 1, 0
    return start
```

Complexity: O(n) time, O(1) space.

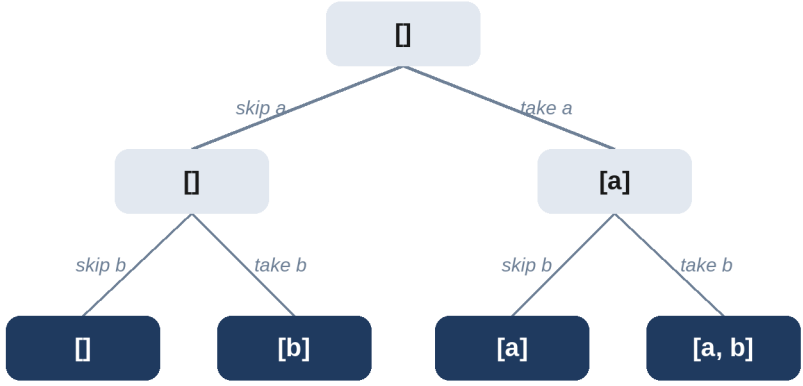
Key Takeaways

- Sort intervals first: by start to merge, by end to select non-overlapping ones.
- A min-heap of end times, or a sweep over +1/-1 events, counts simultaneous intervals.
- Greedy algorithms need a reason: an exchange argument, or a proof that skipped options can't work.
- Many "can you reach / where to start" problems reduce to one running value and one pass.

CHAPTER 15

Backtracking

Some problems ask for *every* solution: all subsets, all orderings, all ways to make change, every valid arrangement. Backtracking builds candidates one choice at a time, explores everything that follows from each choice, and then *undoes* the choice to try the next. Think of it as walking a decision tree, and climbing back up after each branch.



Each leaf is one subset. Going back up a level undoes the last choice.

A backtracking decision tree for subsets of {a, b}: at each level choose to skip or take the next item; each leaf is one subset.

When to Use It

- "Return **all** combinations, subsets, permutations, partitions or arrangements."
- "Is there **any** arrangement that satisfies these rules?" (Sudoku, queens, word search).
- The input is **small** (often 20 items or fewer), a hint that exponential time is expected.

The template has three parts:

1. **Base case:** if the current candidate is complete, record a *copy* of it.
2. **Choices:** for each option available at this point, if it's allowed, apply it.
3. **Recurse, then undo** the choice, so the next option starts from the same state.

Pruning, skipping branches that can't lead to a solution, is what makes backtracking practical: in Problem 15.3, a coin that is too big ends the loop immediately, because all later coins are bigger still.

15.1 All Bundles (Medium)

Return every subset of a list of distinct items, including the empty one.

```
>>> all_bundles(['a', 'b'])
[[], ['b'], ['a'], ['a', 'b']]
>>> all_bundles([])
[[]]
```

The idea. For each item in turn, there are two choices: skip it or take it. The recursion explores both, building the current bundle in a list and undoing each "take" with `pop`.

```
def all_bundles(items):
    out = []
    bundle = []

    def choose(i):
        if i == len(items):
            out.append(bundle[:])  # copy: bundle keeps changing
            return
        choose(i + 1)  # skip items[i]
        bundle.append(items[i])  # take items[i]
        choose(i + 1)
```

```
        bundle.pop()  # undo the choice

    choose(0)
    return out
```

Complexity: O(n × 2^n): there are 2^n subsets, and copying each takes up to n steps.

An iterative alternative treats the numbers 0 to 2^n - 1 as bit masks, where bit i says whether item i is included:

```
def all_bundles_bits(items):
    n = len(items)
    return [
        [items[i] for i in range(n) if mask >> i & 1]
        for mask in range(1 << n)
    ]
```

> **Common Mistake:** Appending `bundle` instead of `bundle[:]`. The result would contain many references to the same list, which ends up empty after all the undoing.

15.2 All Orders (Medium)

Return every ordering (permutation) of a list of distinct items.

```
>>> all_orders([1, 2, 3])
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
>>> all_orders([])
[[]]
```

The idea. Fill the order one position at a time; at each position, try every item not yet used, marking it as used while exploring.

```
def all_orders(items):
    out = []
    order = []
    used = [False] * len(items)

    def place():
        if len(order) == len(items):
            out.append(order[:])
            return
        for i, item in enumerate(items):
            if not used[i]:
                used[i] = True
                order.append(item)
```

```
                place()
                order.pop()
                used[i] = False

    place()
    return out
```

Complexity: O(n × n!).

> **Follow-Up:** "With duplicate items, return only distinct orders." Sort the items, and skip an item if it equals the previous one and the previous one is *not* currently used.

15.3 Exact Change (Medium)

Given distinct coin values, which may each be used any number of times, list all the combinations that add up to an amount, each in non-decreasing order.

```
>>> exact_change([2, 5, 3], 8)
[[2, 2, 2, 2], [2, 3, 3], [3, 5]]
>>> exact_change([2], 1)
[]
>>> exact_change([5], 0)
[[]]
```

Think first: How do you avoid listing both [2, 3, 3] and [3, 2, 3]?

The idea. Only allow coins in non-decreasing order: after choosing coin i, the next choice starts from i again (reuse is allowed), never from an earlier coin. Sorting the coins lets the loop stop as soon as a coin is too big.

```
def exact_change(coins, amount):
    coins = sorted(coins)
    out, combo = [], []

    def pick(start, remaining):
        if remaining == 0:
            out.append(combo[:])
            return
        for i in range(start, len(coins)):
            if coins[i] > remaining:
                break  # sorted, so every later coin is too big as well
            combo.append(coins[i])
            pick(i, remaining - coins[i])  # i again: reuse allowed
            combo.pop()

    pick(0, amount)
    return out
```

Complexity: exponential in the worst case, but pruning keeps it fast for realistic amounts.

> **Follow-Up:** "How many combinations are there?" If you only need the count, don't list them: dynamic programming counts them in O(amount × coins) (Chapter 18).

15.4 Grid Word (Medium)

Can a word be spelled by moving up, down, left or right through a grid of letters, using each cell at most once?

```
>>> grid_word(
...     [['S', 'T', 'A', 'R'], ['P', 'O', 'O', 'L'], ['E', 'N', 'D', 'S']],
...     'STOP',
... )
True
>>> grid_word(
...     [['S', 'T', 'A', 'R'], ['P', 'O', 'O', 'L'], ['E', 'N', 'D', 'S']],
...     'POOLS',
... )
True
>>> grid_word(
...     [['S', 'T', 'A', 'R'], ['P', 'O', 'O', 'L'], ['E', 'N', 'D', 'S']],
...     'POOP',
... )
False
```

The idea. Try every cell as a starting point. From each, explore the four neighbors that match the next letter. Mark a cell as used by temporarily replacing its letter with `#`, and restore it on the way back, so other paths can use it.

```
def grid_word(grid, word):
    rows, cols = len(grid), len(grid[0])

    def found(r, c, i):
        if i == len(word):
            return True
        if not (0 <= r < rows and 0 <= c < cols):
            return False
        if grid[r][c] != word[i]:
            return False
        saved, grid[r][c] = grid[r][c], "#"  # mark as used
        ok = (
            found(r + 1, c, i + 1)
            or found(r - 1, c, i + 1)
```

```
            or found(r, c + 1, i + 1)
            or found(r, c - 1, i + 1)
        )
        grid[r][c] = saved  # unmark on the way back
        return ok

    return any(found(r, c, 0) for r in range(rows) for c in range(cols))
```

Complexity: $O(r \times c \times 3^L)$ for a word of length L: from each step there are at most three new directions.

> **Tip:** Marking cells in the grid itself avoids a separate set, but changes the input while the search runs. Mention it, and restore every cell, as this solution does.

15.5 Safe Queens (Hard)

How many ways can n queens be placed on an n × n chessboard so that no two attack each other (no shared row, column or diagonal)?

```
>>> safe_queens(1)
1
>>> safe_queens(2)
0
>>> safe_queens(4)
2
>>> safe_queens(6)
4
>>> safe_queens(8)
92
```

The idea. Place one queen per row. A queen at (row, col) occupies column `col`, the diagonal `row - col` and the anti-diagonal `row + col`; three sets make each safety check O(1).

```
def safe_queens(n):
    cols, diag, anti = set(), set(), set()

    def place(row):
        if row == n:
            return 1
        count = 0
        for c in range(n):
            if c in cols or row - c in diag or row + c in anti:
                continue
            cols.add(c)
            diag.add(row - c)
            anti.add(row + c)
```

```
            count += place(row + 1)
            cols.remove(c)
            diag.remove(row - c)
            anti.remove(row + c)
        return count

    return place(0)
```

Complexity: roughly O(n!) in the worst case, with heavy pruning in practice. The tests check n = 1 to 10 against the known counts (1, 0, 0, 2, 10, 4, 40, 92, 352, 724), using a brute force over permutations for small n.

15.6 Bracket Sets (Medium)

Generate all strings of n pairs of correctly nested parentheses.

```
>>> bracket_sets(1)
['()']
>>> bracket_sets(3)
['((()))', '(()())', '(())()', '()(())', '()()()']
>>> bracket_sets(0)
['']
```

The idea. Build the string one character at a time. You may add "(" while fewer than n have been opened, and ")" while fewer have been closed than opened. These two rules guarantee that every string produced is valid, so no checking is needed at the end.

```
def bracket_sets(n):
    out = []

    def build(text, opened, closed):
        if len(text) == 2 * n:
            out.append(text)
            return
        if opened < n:
            build(text + "(", opened + 1, closed)
        if closed < opened:  # only close what is open
            build(text + ")", opened, closed + 1)

    build("", 0, 0)
    return out
```

Complexity: proportional to the number of results, the n-th Catalan number, which grows roughly like 4^n.

Key Takeaways

- Backtracking explores a decision tree: choose, recurse, undo.
- Record copies of the current candidate, never the candidate itself.
- Avoid duplicate results by controlling the order of choices, for example "only equal or later coins".
- Prune early: invalid partial candidates should never be extended.
- Small input limits are a hint that exponential solutions are expected.

CHAPTER 16

Graphs: BFS and DFS

A graph is a set of nodes connected by edges: cities and roads, people and friendships, web pages and links, cells of a grid and their neighbors. Interview graph problems rarely say "graph"; recognizing one hidden in a grid, a word list or a set of rules is half the work. Once you see it, two searches cover most problems: **depth-first search** to explore everything connected, and **breadth-first search** to find shortest paths when every step costs the same.

Representing a Graph

- An **adjacency list** maps each node to its neighbors: `graph[node] = [neighbor, ...]`. Build it from a list of edges with a loop. It is the right default.
- A **grid** is already a graph: each cell's neighbors are the cells up, down, left and right of it, which you compute on the fly.
- An **implicit graph** has nodes you generate as you go, such as all words one letter away from the current word (Problem 16.5).

When to Use Which Search

- **DFS** (recursion or a stack): count or label connected regions, check whether something is reachable, detect cycles, check two-colorability.
- **BFS** (a queue): the **shortest path** in steps; anything spreading **in waves**, one step per minute; levels of distance.

Both visit each node and edge once: O(V + E) time. And both need a **visited set**, or a mark in the grid, to avoid going round in circles.

> **Warning:** Mark a node as visited when you *add* it to the BFS queue, not when you take it out. Otherwise, the same node can be added many times before it's processed, which wastes time and, on large grids, memory.

16.1 Count Islands (Medium)

A map is a grid of "1" (land) and "0" (water). An island is a group of land cells connected up, down, left or right. How many islands are there?

```
>>> count_islands(
...     [['1', '1', '0', '0', '0'],
...      ['1', '1', '0', '0', '0'],
...      ['0', '0', '1', '0', '0'],
...      ['0', '0', '0', '1', '1']],
... )
3
>>> count_islands([['0', '0', '0']])
0
>>> count_islands([])
0
```

The idea. Scan the grid. Each time you find unvisited land, that's a new island: count it, then "sink" all its connected land with a DFS, so it is never counted again.

```
def count_islands(grid):
    grid = [row[:] for row in grid]  # don't change the caller's grid
    rows, cols = len(grid), len(grid[0]) if grid else 0

    def sink(r, c):
        if 0 <= r < rows and 0 <= c < cols and grid[r][c] == "1":
            grid[r][c] = "0"
            sink(r + 1, c)
            sink(r - 1, c)
            sink(r, c + 1)
            sink(r, c - 1)

    islands = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":
                islands += 1
                sink(r, c)
    return islands
```

Complexity: O(r × c) time; O(r × c) space for the copy and the recursion in the worst case. For very large grids, the BFS version in the code files avoids Python's recursion limit.

16.2 Paint Bucket (Easy)

Recolor the region of same-colored cells containing a given cell, like the fill tool in a paint program.

```
>>> flood_fill([[1, 1, 1], [1, 1, 0], [1, 0, 1]], 1, 1, 2)
[[2, 2, 2], [2, 2, 0], [2, 0, 1]]
>>> flood_fill([[0, 0]], 0, 0, 0)
[[0, 0]]
```

The idea. An iterative DFS with an explicit stack: recolor a cell if it has the old color, then push its neighbors.

```
def flood_fill(image, r, c, color):
    image = [row[:] for row in image]
    old = image[r][c]
    if old == color:
        return image
    stack = [(r, c)]
    while stack:
        r, c = stack.pop()
        if (
            0 <= r < len(image)
            and 0 <= c < len(image[0])
            and image[r][c] == old
        ):
            image[r][c] = color
            stack.extend([(r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)])
    return image
```

Complexity: O(r × c) time and space.

> **Common Mistake:** Forgetting the case where the new color equals the old color. The search then never stops, because recolored cells still match. The early return handles it.

16.3 Warehouse Route (Medium)

A robot moves through a warehouse grid of open cells (0) and shelves (1), one step at a time up, down, left or right. What is the fewest number of steps from the top-left corner to the bottom-right corner? Return -1 if it is impossible.

```
>>> warehouse_route([[0, 0, 0], [1, 1, 0], [0, 0, 0]])
4
>>> warehouse_route([[0, 1], [1, 0]])
-1
>>> warehouse_route([[0]])
0
```

The idea. BFS explores cells in order of distance: all cells one step away, then two, and so on. The first time it reaches the goal, that distance is the shortest.

```
def warehouse_route(grid):
    rows, cols = len(grid), len(grid[0])
    if grid[0][0] or grid[rows - 1][cols - 1]:
        return -1
    dist = {(0, 0): 0}
    queue = deque([(0, 0)])
    while queue:
        r, c = queue.popleft()
        if (r, c) == (rows - 1, cols - 1):
            return dist[(r, c)]
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if (
                0 <= nr < rows
                and 0 <= nc < cols
                and grid[nr][nc] == 0
                and (nr, nc) not in dist
            ):
                dist[(nr, nc)] = dist[(r, c)] + 1
                queue.append((nr, nc))
    return -1
```

Complexity: O(r × c) time and space.

> **Interviewer's View:** Why BFS and not DFS? Because DFS finds *a* path, not the shortest one. Saying this before coding shows you chose the algorithm deliberately.

16.4 Spreading Mold (Medium)

A tray holds bread: 0 is empty, 1 is fresh bread, 2 is moldy bread. Every minute, mold spreads from each moldy piece to fresh pieces next to it. How many minutes until no fresh bread is left? Return -1 if some bread can never go moldy.

```
>>> spreading_mold([[1, 1, 0, 2], [0, 1, 1, 1], [0, 0, 0, 1]])
5
>>> spreading_mold([[2, 1, 0], [0, 0, 1]])
-1
>>> spreading_mold([[0, 2]])
0
```

Think first: Mold starts in several places at once. How do you run BFS from many starting points?

The idea. A *multi-source* BFS: put every moldy cell in the queue at time 0. The queue then processes the tray in waves, minute by minute. Count fresh pieces at the start, and check whether any are left at the end.

```
def spreading_mold(tray):
    rows, cols = len(tray), len(tray[0])
    tray = [row[:] for row in tray]
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if tray[r][c] == 2:
                queue.append((r, c, 0))  # every moldy cell starts at once
            elif tray[r][c] == 1:
                fresh += 1
    minutes = 0
    while queue:
        r, c, t = queue.popleft()
        minutes = max(minutes, t)
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and tray[nr][nc] == 1:
                tray[nr][nc] = 2
                fresh -= 1
                queue.append((nr, nc, t + 1))
    return minutes if fresh == 0 else -1
```

Complexity: O(r × c) time and space. A minute-by-minute simulation is simpler but O((r × c)²) in the worst case; the tests use it as a check.

16.5 Word Steps (Hard)

Change a start word into a goal word one letter at a time; every intermediate word must be in a dictionary. How many words are in the shortest such chain, including both ends? Return 0 if no chain exists.

```
>>> word_steps('cat', 'dog', ['cot', 'cut', 'cog', 'dot', 'dog'])
4
>>> word_steps('cat', 'dog', ['cut', 'cog', 'dot', 'dog'])
0
>>> word_steps('aa', 'aa', ['aa'])
1
```

Think first: What are the nodes and edges of this graph?

The idea. Words are nodes, and two words are connected if they differ in exactly one letter. Instead of comparing every pair of words, generate each word's neighbors directly: change each position to each letter of the alphabet, and keep the results that are in the dictionary. Then BFS from the start word.

```
def word_steps(start, goal, words):
    words = set(words)
    if goal not in words:
        return 0
    queue = deque([(start, 1)])
    seen = {start}
    while queue:
        word, length = queue.popleft()
        if word == goal:
            return length
        for i in range(len(word)):
            for ch in ascii_lowercase:
                nxt = word[:i] + ch + word[i + 1 :]
                if nxt in words and nxt not in seen:
                    seen.add(nxt)
                    queue.append((nxt, length + 1))
    return 0
```

Complexity: O(n × L × 26 × L) for n words of length L: each word generates 26L candidates, and building or hashing each takes O(L).

> **Follow-Up:** A *bidirectional* BFS from both ends, always expanding the smaller frontier, explores far fewer words on large dictionaries.

16.6 Two Teams (Medium)

People 0 to n-1 must be split into two teams, and certain pairs refuse to be on the same team. Is it possible?

```
>>> two_teams(4, [(0, 1), (1, 2), (2, 3), (3, 0)])
True
>>> two_teams(3, [(0, 1), (1, 2), (2, 0)])
False
>>> two_teams(1, [])
True
```

The idea. This asks whether the graph is *bipartite*: two-colorable. Give the first person of each connected group team 0, and each rival the opposite team of the person who reached them. A conflict between two people on the same team means it's impossible, which happens exactly when the graph has a cycle of odd length.

```
def two_teams(n, conflicts):
    graph = [[] for _ in range(n)]
    for a, b in conflicts:
        graph[a].append(b)
        graph[b].append(a)
    team = [None] * n
    for person in range(n):
        if team[person] is not None:
            continue
        team[person] = 0
        stack = [person]
        while stack:
            p = stack.pop()
            for q in graph[p]:
                if team[q] is None:
                    team[q] = 1 - team[p]  # rivals go on the other team
                    stack.append(q)
                elif team[q] == team[p]:
                    return False
    return True
```

Complexity: O(n + e) time and space.

Key Takeaways

- Look for the hidden graph: grids, words, rules and dependencies are all graphs.
- DFS explores and labels connected regions; BFS finds shortest paths in unweighted graphs.

- Multi-source BFS models anything spreading from several places at once.
- Always track visited nodes, marking them when they are queued.
- Build neighbors on the fly when the graph is implicit.

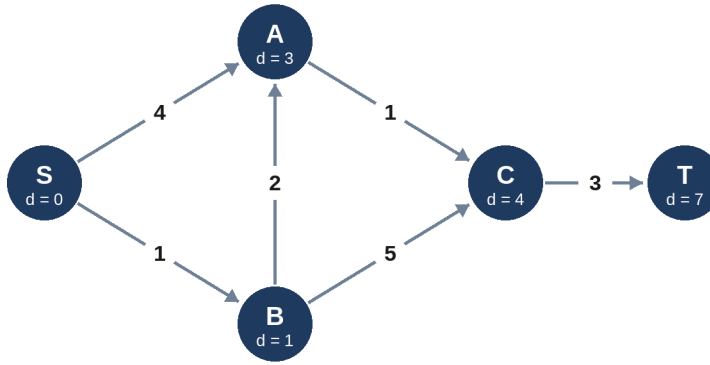
CHAPTER 17

Advanced Graphs

This chapter covers the graph algorithms that come up when plain BFS and DFS aren't enough: **topological sorting** for dependencies, **union-find** for merging groups, **Dijkstra's algorithm** for shortest paths with different edge costs, a limited **Bellman-Ford** for paths with a maximum number of steps, and **minimum spanning trees** for connecting everything as cheaply as possible.

When to Use Which

Problem shape	Algorithm	Time
Tasks with prerequisites; "an order that respects dependencies"; detect a cycle in a directed graph	Topological sort (Kahn's algorithm, or DFS)	O(V + E)
Connecting things and asking "are these already connected?"	Union-find (disjoint sets)	nearly O(1) per operation
Shortest path with non-negative edge weights	Dijkstra with a heap	O(E log V)
Shortest path with at most k edges, or negative weights	Bellman-Ford, k rounds	O(k × E)
Cheapest way to connect all nodes	Minimum spanning tree (Prim or Kruskal)	O(E log V)



Settled in order: S, B, A, C, T. A is reached via B (1 + 2 = 3), not directly (4).

Dijkstra's algorithm: a heap always expands the closest unsettled node next; once a node is taken from the heap, its distance is final.

17.1 Can I Graduate? (Medium)

A degree has courses numbered 0 to n-1. A pair (a, b) means course b must be completed before course a. Can every course be completed?

```
>>> can_finish(2, [(1, 0)])
True
>>> can_finish(2, [(1, 0), (0, 1)])
False
>>> can_finish(3, [])
True
```

Think first: When is it impossible?

The idea. It's impossible exactly when the prerequisites form a cycle. Kahn's algorithm finds out: count each course's unmet prerequisites; start with the courses that have none; "take" a course and reduce the count of every course that depends on it, adding a course to the ready queue when its count reaches zero. If some courses are never taken, they are stuck in a cycle.

```
def can_finish(n, prereqs):
    after = [[] for _ in range(n)]
    waiting_on = [0] * n  # unmet prerequisites per course
    for course, before in prereqs:
        after[before].append(course)
        waiting_on[course] += 1
    ready = deque(c for c in range(n) if waiting_on[c] == 0)
    taken = 0
    while ready:
        c = ready.popleft()
        taken += 1
        for nxt in after[c]:
            waiting_on[nxt] -= 1
            if waiting_on[nxt] == 0:
                ready.append(nxt)
    return taken == n  # anything left is stuck in a cycle
```

Complexity: O(n + p) for p prerequisites.

The DFS alternative marks nodes as "on the current path"; meeting such a node again means a cycle:

```
def can_finish_dfs(n, prereqs):
    graph = [[] for _ in range(n)]
    for course, before in prereqs:
        graph[before].append(course)
    state = [0] * n  # 0 unvisited, 1 on the current path, 2 done

    def has_cycle(c):
        if state[c] == 1:
            return True
        if state[c] == 2:
            return False
        state[c] = 1
        if any(has_cycle(nxt) for nxt in graph[c]):
            return True
        state[c] = 2
        return False

    return not any(has_cycle(c) for c in range(n))
```

17.2 Build Order (Medium)

Packages 0 to n-1 have build dependencies: (a, b) means b must be built before a. Return a valid build order, choosing the smallest-numbered package whenever several are ready, or an empty list if there is a cycle.

```
>>> build_order(4, [(1, 0), (2, 0), (3, 1), (3, 2)])
[0, 1, 2, 3]
>>> build_order(2, [(0, 1), (1, 0)])
[]
>>> build_order(3, [(0, 2)])
[1, 2, 0]
```

The idea. Kahn's algorithm again, recording the order. Using a heap instead of a queue for the ready packages makes the order deterministic, which interviewers sometimes require.

```
def build_order(n, deps):
    after = [[] for _ in range(n)]
    waiting_on = [0] * n
    for pkg, before in deps:
        after[before].append(pkg)
        waiting_on[pkg] += 1
    ready = [p for p in range(n) if waiting_on[p] == 0]
    heapq.heapify(ready)  # smallest ready package first
    order = []
    while ready:
        p = heapq.heappop(ready)
        order.append(p)
        for nxt in after[p]:
            waiting_on[nxt] -= 1
            if waiting_on[nxt] == 0:
                heapq.heappush(ready, nxt)
    return order if len(order) == n else []
```

Complexity: O((n + d) log n) with the heap; O(n + d) with a plain queue.

17.3 The Extra Cable (Medium)

n computers, numbered 1 to n, were connected by cables into a tree (every computer reachable, no loops). Then one extra cable was added. Return a cable whose removal restores a tree; if several qualify, return the one listed last.

```
>>> extra_cable([[1, 2], [2, 3], [1, 3]])
[1, 3]
>>> extra_cable([[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]])
[1, 4]
```

The idea. Union-find keeps track of which computers are already connected. Add cables in order; the first cable whose two ends are already connected creates the loop. It is also the last cable of the loop in the listed order, which matches "the one listed last".

```
def extra_cable(cables):
    parent = list(range(len(cables) + 1))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]  # shorten the path as we go
            x = parent[x]
        return x

    for a, b in cables:
        ra, rb = find(a), find(b)
        if ra == rb:
            return [a, b]  # a and b were already connected
        parent[ra] = rb
    return []
```

Complexity: nearly O(n). *Path compression*, the line that shortens paths as it walks them, keeps the trees of the union-find structure almost flat.

17.4 Signal Time (Medium)

A network has n nodes, 1 to n. A link (u, v, t) carries a signal from u to v in t milliseconds. A signal starts at node k. How long until every node has received it? Return -1 if some node never does.

```
>>> signal_time(4, [(2, 1, 1), (2, 3, 1), (3, 4, 1)], 2)
2
>>> signal_time(2, [(1, 2, 1)], 2)
-1
>>> signal_time(1, [], 1)
0
```

The idea. Dijkstra's algorithm: a heap always gives the node with the earliest arrival time not yet settled. When a node comes out of the heap for the first time, its arrival time is final, because every other route would arrive later. The answer is the latest arrival time.

```
def signal_time(n, links, k):
    graph = {u: [] for u in range(1, n + 1)}
    for u, v, t in links:
        graph[u].append((v, t))
    arrival = {}
    heap = [(0, k)]
    while heap:
        time, u = heapq.heappop(heap)
        if u in arrival:
            continue  # already reached by a faster route
        arrival[u] = time
        for v, t in graph[u]:
            if v not in arrival:
```

```
                heapq.heappush(heap, (time + t, v))
    return max(arrival.values()) if len(arrival) == n else -1
```

Complexity: O(E log E) time, O(V + E) space.

> **Warning:** Dijkstra's algorithm requires non-negative weights. With negative weights, a node settled early could later be reached more cheaply. Use Bellman-Ford instead.

17.5 Cheapest Trip With Few Stops (Medium)

Flights (u, v, price) connect n cities. What is the cheapest way from src to dst with at most k stops, that is, at most k + 1 flights? Return -1 if impossible.

```
>>> cheap_flights(
...     4,
...     [(0, 1, 100), (1, 2, 100), (2, 0, 100), (1, 3, 600), (2, 3, 200)],
...     0,
...     3,
...     1,
... )
700
>>> cheap_flights(3, [(0, 1, 100), (1, 2, 100), (0, 2, 500)], 0, 2, 1)
200
>>> cheap_flights(3, [(0, 1, 100), (1, 2, 100), (0, 2, 500)], 0, 2, 0)
500
```

Think first: Why doesn't plain Dijkstra work here?

The idea. Dijkstra finds the cheapest route regardless of the number of flights, and may discard a slightly more expensive route that uses fewer flights. Bellman-Ford limited to k + 1 rounds solves it directly: round i allows routes with up to i flights. Each round reads the previous round's prices, so a round can't chain two flights together.

```
def cheap_flights(n, flights, src, dst, k):
    inf = float("inf")
    price = [inf] * n
    price[src] = 0
    for _ in range(k + 1):  # each round allows one more flight
        new = price[:]  # read last round's prices only
        for u, v, p in flights:
            if price[u] + p < new[v]:
                new[v] = price[u] + p
        price = new
    return -1 if price[dst] == inf else price[dst]
```

Complexity: O(k × F) for F flights.

> **Common Mistake:** Updating `price` in place within a round. Then a single round can use several flights, and the limit on stops is silently broken. The copy `new = price[:]` prevents this.

17.6 Cheapest Wiring (Medium)

Sensors sit at points on a grid. A cable between two sensors costs their Manhattan distance, `|x1 - x2| + |y1 - y2|`. What is the cheapest way to connect all the sensors?

```
>>> cheapest_wiring([(0, 0), (2, 2), (3, 10), (5, 2), (7, 0)])
20
>>> cheapest_wiring([(1, 1)])
0
```

The idea. This is a *minimum spanning tree*. Prim's algorithm grows the tree from one sensor, always adding the cheapest cable that reaches a new sensor, which a heap provides.

```
def cheapest_wiring(points):
    n = len(points)
    connected = [False] * n
    heap = [(0, 0)]  # (cost to connect, point index)
    total = 0
    joined = 0
    while joined < n:
        cost, i = heapq.heappop(heap)
        if connected[i]:
            continue
        connected[i] = True
        total += cost
        joined += 1
        xi, yi = points[i]
        for j in range(n):
            if not connected[j]:
                xj, yj = points[j]
                heapq.heappush(heap, (abs(xi - xj) + abs(yi - yj), j))
    return total
```

Complexity: O(n² log n) here, because every pair of sensors is a possible cable. Kruskal's algorithm, which sorts all cables and joins them with union-find unless they would form a loop, gives the same total; the tests compare the two.

Key Takeaways

- Dependencies are directed graphs: topological sort orders them and detects cycles.
- Union-find answers "already connected?" almost in O(1), ideal for finding redundant edges.
- Dijkstra finds shortest paths with non-negative weights; a node's distance is final when it leaves the heap.
- Limit Bellman-Ford to k rounds for "at most k steps" constraints.
- Minimum spanning trees connect everything cheaply: Prim with a heap, or Kruskal with union-find.

CHAPTER 18

Dynamic Programming I: One Dimension

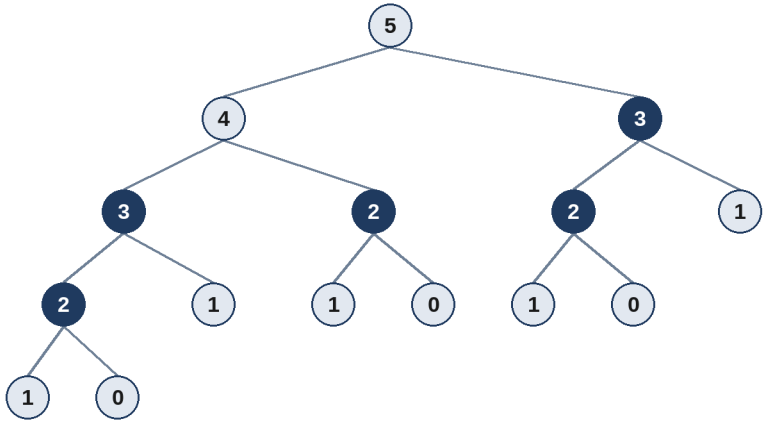
Dynamic programming (DP) has a reputation as the hardest interview topic. Most of that reputation comes from trying to guess the final, compact solution directly. Approached step by step, it is a mechanical process: write the plain recursive solution, notice that it solves the same smaller problems again and again, and store their answers. This chapter covers DP problems where the state is a single number, such as a position in a list or an amount of money.

The Four-Step Recipe

1. **Define the state** in words: "`ways(i)` is the number of ways to reach step i." Most of the difficulty is here.
2. **Write the recurrence**: how does the answer for a state follow from smaller states? "`ways(i) = ways(i - 1) + ways(i - 2)`."
3. **Choose the base cases**: the smallest states, whose answers you know directly.
4. **Choose the order**: either recursion with a cache (*top-down*, or *memoization*) or a loop that fills a table from the smallest state upward (*bottom-up*, or *tabulation*).

Top-down is usually quicker to write correctly: in Python, `functools.cache` turns a recursive function into a memoized one with one line. Bottom-up avoids Python's recursion limit and often allows keeping only the last few

values, cutting the memory from O(n) to O(1). In an interview, start with whichever you can write confidently, and mention the other.



Without a cache, ways(3) is computed 2 times and ways(2) 3 times.
With a cache or a table, each state is computed once.

From recursion to DP: the plain recursion tree for stair_ways(5) recomputes the same states many times; a cache or a table computes each state once.

When to Use It

- "**How many ways**...", "**fewest / most / longest**...", "**is it possible**..." over sequences of choices.
- A brute force that tries every combination of choices, where different combinations reach the **same situation** (the same remaining amount, the same position).
- Choices made now affect only what is possible *later*, not what happened before.

If a greedy rule works (Chapter 14), it is simpler and faster. DP is for when no single rule is safe and you must compare options.

18.1 Stair Ways (Easy)

How many different ways are there to climb n steps, taking 1 or 2 steps at a time?

```
>>> stair_ways(1)
1
>>> stair_ways(2)
2
>>> stair_ways(5)
8
>>> stair_ways(0)
1
```

Brute force. The last move onto step n came from step n - 1 or from step n - 2, so the answer is the sum of the two smaller answers:

```
def stair_ways_brute(n):
    if n <= 1:
        return 1
    return stair_ways_brute(n - 1) + stair_ways_brute(n - 2)
```

That's correct but takes exponential time: `stair_ways_brute(40)` makes over 300 million calls and takes many seconds, and each extra step multiplies the time by about 1.6, because the same values are recomputed an enormous number of times.

With a cache. Adding `@cache` stores each answer the first time it's computed, so each of the n states is computed once:

```
@cache
def stair_ways_memo(n):
    if n <= 1:
        return 1
    return stair_ways_memo(n - 1) + stair_ways_memo(n - 2)
```

Bottom-up with O(1) memory. Each answer depends only on the previous two, so two variables are enough:

```
def stair_ways(n):
    a, b = 1, 1  # ways to reach steps i - 1 and i
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

Complexity: O(n) time and O(1) space. The answers are the Fibonacci numbers.

> **Interviewer's View:** Showing all three versions, and explaining why each is faster than the last, is a complete answer to this problem, and a model for every DP problem that follows.

18.2 Quiet Street (Medium)

Volunteers collect donations along a street of houses in a row. To avoid annoying the neighbors, they never visit two neighboring houses. Given the donation each house would give, what is the largest total they can collect?

```
>>> quiet_street([4, 1, 2, 7, 5, 3, 1])
14
>>> quiet_street([])
0
>>> quiet_street([5, 1, 1, 5])
10
```

Think first: For the last house, there are two choices. What does each choice leave?

The idea. Process the houses left to right, tracking two numbers: the best total if the house just seen was *taken*, and the best total if it was *skipped*. Taking the current house is only allowed after a skip; skipping it keeps the better of the two.

```
def quiet_street(gifts):
    take, skip = 0, 0  # best totals if the last house was taken/skipped
    for g in gifts:
        take, skip = skip + g, max(take, skip)
    return max(take, skip)
```

Complexity: O(n) time, O(1) space.

> **Follow-Up:** "The houses are arranged in a circle, so the first and last are neighbors." Solve the row twice, once without the first house and once without the last, and take the better result.

18.3 Fewest Coins (Medium)

Given coin values, with an unlimited supply of each, what is the fewest number of coins that adds up to exactly an amount? Return -1 if it's impossible.

```
>>> fewest_coins([1, 5, 10, 25], 63)
6
>>> fewest_coins([2], 3)
-1
>>> fewest_coins([1], 0)
0
>>> fewest_coins([1, 3, 4], 6)
2
```

Think first: Why does "always take the largest coin that fits" fail for coins [1, 3, 4] and amount 6?

The idea. Greedy takes 4 + 1 + 1, three coins, but 3 + 3 uses two. Instead, define `best[a]` as the fewest coins for amount a. The last coin used is one of the coin values c, so `best[a] = 1 + min(best[a - c])` over all coins that fit. Fill the table from 0 upward.

```
def fewest_coins(coins, amount):
    inf = float("inf")
    best = [0] + [inf] * amount  # best[a] = fewest coins for amount a
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
    return -1 if best[amount] == inf else best[amount]
```

Complexity: O(amount × coins) time, O(amount) space.

The top-down version expresses the same recurrence directly:

```
def fewest_coins_memo(coins, amount):
    @cache
    def need(a):
        if a == 0:
            return 0
        options = [need(a - c) for c in coins if c <= a]
        options = [o for o in options if o >= 0]
        return 1 + min(options) if options else -1

    return need(amount)
```

> **Common Mistake:** Using 0 or -1 for "impossible" inside the table, where it then wins every `min`. Use infinity inside the computation and convert to -1 only at the end.

18.4 Longest Rising Sequence (Medium)

Find the length of the longest strictly increasing subsequence: values in their original order, but not necessarily next to each other.

```
>>> rising([3, 8, 4, 5, 1, 9, 6, 7])
5
>>> rising([7, 7, 7])
1
>>> rising([])
0
```

The O(n²) DP. Let `best[i]` be the length of the longest rising sequence that *ends* at position i. It extends the best sequence ending at any earlier, smaller value.

```
def rising_quadratic(values):
    best = [1] * len(values)  # best[i]: longest ending at values[i]
    for i in range(len(values)):
        for j in range(i):
            if values[j] < values[i]:
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)
```

The O(n log n) idea. Keep `tails[k]`, the smallest value that can end a rising sequence of length k + 1. The list is always sorted, so binary search finds where each new value goes: past the end, it extends the longest sequence; otherwise it replaces a larger tail, which can only make later extensions easier.

```
def rising(values):
    tails = []  # tails[k]: smallest tail of a rising run of length k+1
    for v in values:
        k = bisect_left(tails, v)
        if k == len(tails):
            tails.append(v)
        else:
            tails[k] = v  # a smaller tail is easier to extend later
    return len(tails)
```

Complexity: O(n log n) time, O(n) space.

> **Warning:** `tails` is not itself a rising subsequence of the input; only its length is meaningful. If the interviewer asks for the sequence itself, store a link from each value to its predecessor.

18.5 Split Into Words (Medium)

Can a string without spaces be split into words from a dictionary? Words may be used more than once.

```
>>> split_words('sunflowerseed', ['sun', 'flower', 'flowers', 'seed'])
True
>>> split_words('parkingspace', ['park', 'parking', 'king', 'spa'])
False
>>> split_words('', ['a'])
True
```

Think first: In "parkingspace", both "park" and "parking" match at the start. Why can't you simply take the longest match?

The idea. Let `ok[i]` mean "the first i characters can be split". Then `ok[i]` is true if some earlier `ok[j]` is true and `text[j:i]` is a word. Only the last `longest` positions need checking, where `longest` is the length of the longest word.

```
def split_words(text, words):
    words = set(words)
    longest = max(map(len, words), default=0)
    ok = [True] + [False] * len(text)  # ok[i]: text[:i] can be split
    for i in range(1, len(text) + 1):
        for j in range(max(0, i - longest), i):
            if ok[j] and text[j:i] in words:
                ok[i] = True
                break
    return ok[len(text)]
```

Complexity: O(n × L × L) for text of length n and words up to length L: n positions, L split points each, and O(L) to slice and hash each piece.

> **Follow-Up:** "Return every possible sentence." Then you need backtracking (Chapter 15), and the output itself can be exponential in size. Memoize the list of sentences for each starting position.

18.6 Decode Ways (Medium)

A message of capital letters was encoded as numbers, A = 1 to Z = 26, and the spaces between the numbers were lost. In how many ways can a digit string be read back as letters?

```
>>> decode_ways('12')
2
>>> decode_ways('2611')
4
>>> decode_ways('06')
0
>>> decode_ways('10')
1
>>> decode_ways('')
0
```

Think first: What makes a single digit or a pair of digits a valid letter?

The idea. It's Stair Ways with rules: the last letter used either one digit, which must not be "0", or two digits, which must form a number from 10 to 26. So the number of ways for a prefix is the number for the prefix one shorter (if the last digit is valid alone) plus the number for the prefix two shorter (if the last pair is valid).

```
def decode_ways(digits):
    prev2, prev1 = 0, 1  # ways for the prefixes two and one shorter
    for i, d in enumerate(digits):
        cur = 0
        if d != "0":
            cur += prev1  # read d as a single letter
        if i > 0 and "10" <= digits[i - 1 : i + 1] <= "26":
            cur += prev2  # read the last two digits as one letter
        prev2, prev1 = prev1, cur
    return prev1 if digits else 0
```

Complexity: O(n) time, O(1) space.

> **Common Mistake:** Treating "06" as a valid two-digit letter. Comparing the strings with `"10" <= pair <= "26"` excludes a leading zero, because "06" sorts before "10".

Key Takeaways

- DP is recursion plus remembering: write the brute force, find the repeated states, and cache them.
- Define the state in one sentence before writing code.
- Top-down with `@cache` is fast to write; bottom-up avoids the recursion limit and can save memory.
- When each state depends only on the last few states, keep only those few values.
- Greedy rules often fail on "fewest coins"-style problems; test them on a small counterexample.

CHAPTER 19

Dynamic Programming II: Grids, Strings and Knapsacks

The second DP chapter moves to states described by two numbers: a cell in a grid, a position in each of two strings, an item and a remaining capacity. The recipe from Chapter 18 does not change. What changes is that the table becomes two-dimensional, and choosing the filling order needs a moment's thought: every entry must be computed after the entries it depends on.

Recognizing Two-Dimensional DP

Problem shape	State	Typical recurrence
Paths through a grid	`t[r][c]`: the answer for cell (r, c)	from the cell above and the cell to the left
Two strings	`t[i][j]`: the answer for `a[:i]` and `b[:j]`	from `t[i-1][j-1]`, `t[i-1][j]`, `t[i][j-1]`
Choose items under a limit	`best[c]`: the best value within capacity c, after the items so far	take or skip each item
One string, all substrings	`t[i][j]`: the answer for `s[i:j+1]`	from the shorter substring inside it

	""	d	r	a	m	a
""	0	0	0	0	0	0
d	0	1	1	1	1	1
y	0	1	1	1	1	1
n	0	1	1	1	1	1
a	0	1	1	2	2	2
m	0	1	1	2	3	3
i	0	1	1	2	3	3
c	0	1	1	2	3	3

Dark cells: matching characters (diagonal + 1). Answer: 3.

The table for the longest shared subsequence of "dynamic" and "drama": matching characters extend the diagonal; otherwise take the better of the cell above and the cell to the left.

> **Tip:** When each row of the table depends only on the row above, keep just one or two rows. That turns O(m × n) memory into O(n), a common follow-up question.

19.1 Grid Paths (Medium)

A robot moves only right or down across a grid, from the top-left cell to the bottom-right cell. Some cells are blocked (1). How many different routes are there?

```
>>> grid_paths([[0, 0, 0], [0, 1, 0], [0, 0, 0]])
2
>>> grid_paths(
...     [[0, 0, 0, 0, 0, 0, 0],
...      [0, 0, 0, 0, 0, 0, 0],
...      [0, 0, 0, 0, 0, 0, 0]],
... )
28
```

```
>>> grid_paths([[1]])
0
```

The idea. The number of routes into a cell is the number into the cell above plus the number into the cell to the left; a blocked cell has none. Filling the grid row by row, a single list is enough: before the update, `ways[c]` still holds the count from the row above, and `ways[c - 1]` already holds the new count from the left.

```
def grid_paths(grid):
    rows, cols = len(grid), len(grid[0])
    ways = [0] * cols  # one row of the table at a time
    ways[0] = 1 if grid[0][0] == 0 else 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                ways[c] = 0
            elif c > 0:
                ways[c] += ways[c - 1]  # from above (old) + from left
    return ways[-1]
```

Complexity: O(r × c) time, O(c) space.

> **Follow-Up:** "No blocked cells: is there a formula?" Every route consists of r - 1 moves down and c - 1 moves right in some order, so the count is the binomial coefficient `math.comb(r + c - 2, r - 1)`. For the 3 × 7 grid, that's `comb(8, 2) = 28`.

19.2 Longest Shared Subsequence (Medium)

Find the length of the longest sequence of characters that appears, in order but not necessarily together, in both of two strings.

```
>>> shared('dynamic', 'drama')
3
>>> shared('abc', 'def')
0
>>> shared('', 'x')
0
```

Think first: Compare the first characters of both strings. What are the cases?

The idea. If the first characters match, use them both and continue with the rest of each string. If they don't, one of them is not part of the answer: try dropping each, and keep the better result. Top-down, that's three lines of logic:

```
def shared_memo(a, b):
    @cache
    def best(i, j):
        if i == len(a) or j == len(b):
            return 0
        if a[i] == b[j]:
            return 1 + best(i + 1, j + 1)
        return max(best(i + 1, j), best(i, j + 1))

    return best(0, 0)
```

Bottom-up, `t[i][j]` holds the answer for the first i characters of a and the first j characters of b:

```
def shared(a, b):
    m, n = len(a), len(b)
    t = [[0] * (n + 1) for _ in range(m + 1)]  # t[i][j]: a[:i] vs b[:j]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                t[i][j] = t[i - 1][j - 1] + 1
            else:
                t[i][j] = max(t[i - 1][j], t[i][j - 1])
    return t[m][n]
```

Complexity: O(m × n) time and space.

> **Interviewer's View:** "Longest common subsequence" is the ancestor of many string problems, including file comparison tools that show which lines changed. Interviewers like it because the recurrence is explainable in two sentences.

19.3 Typo Distance (Hard)

What is the fewest single-character insertions, deletions or replacements that turn one string into another? This is the *edit distance*, or *Levenshtein distance*, used by spelling correctors.

```
>>> typo_distance('rocket', 'pocket')
1
>>> typo_distance('launch', 'lunches')
3
>>> typo_distance('', 'abc')
3
>>> typo_distance('same', 'same')
0
```

The idea. Compare the last characters of the two prefixes. If they match, no edit is needed for them. Otherwise, the last edit was one of three, and each leaves a smaller problem:

- **delete** the last character of a: the cell above;
- **insert** the last character of b: the cell to the left;
- **replace** one with the other: the cell diagonally up and to the left.

The first row and column are the costs of building a prefix from nothing, or erasing it.

```
def typo_distance(a, b):
    prev = list(range(len(b) + 1))  # distances from "" to b[:j]
    for i in range(1, len(a) + 1):
        cur = [i] + [0] * len(b)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                cur[j] = prev[j - 1]
            else:
                cur[j] = 1 + min(
                    prev[j],  # delete a[i-1]
                    cur[j - 1],  # insert b[j-1]
                    prev[j - 1],  # replace
                )
        prev = cur
    return prev[len(b)]
```

Complexity: O(m × n) time, O(n) space with two rows.

The top-down version is a direct translation of the three cases:

```
def typo_distance_memo(a, b):
    @cache
    def d(i, j):
        if i == len(a):
            return len(b) - j
        if j == len(b):
            return len(a) - i
        if a[i] == b[j]:
            return d(i + 1, j + 1)
        return 1 + min(d(i + 1, j), d(i, j + 1), d(i + 1, j + 1))
```

```
    return d(0, 0)
```

19.4 Backpack (Medium)

Items have weights and values, and each item can be packed at most once. What is the most value that fits within a weight capacity? This is the classic *0/1 knapsack* problem.

```
>>> backpack([1, 3, 4, 5], [1, 4, 5, 7], 7)
9
>>> backpack([], [], 5)
0
>>> backpack([6], [10], 5)
0
```

Think first: Why can't you simply sort by value per unit of weight?

The idea. Ratio-greedy fails because items can't be split: a good-ratio item can waste capacity that a pair of other items would fill exactly. Instead, for each item, decide for every capacity c whether to take it: `best[c] = max(best[c], best[c - w] + v)`.

```
def backpack(weights, values, capacity):
    best = [0] * (capacity + 1)  # best[c]: most value within capacity c
    for w, v in zip(weights, values):
        for c in range(capacity, w - 1, -1):  # backwards: use item once
            best[c] = max(best[c], best[c - w] + v)
    return best[capacity]
```

Complexity: O(n × capacity) time, O(capacity) space.

> **Common Mistake:** Looping capacities upward in the one-list version. Then `best[c - w]` may already include the current item, so the item gets packed twice. The backward loop reads only values from before this item. (Looping upward is exactly right for the *unlimited* version, as in Fewest Coins.)

19.5 Fair Split (Medium)

Can a set of parcels be divided into two groups of exactly equal total weight?

```
>>> fair_split([3, 1, 4, 2, 2])
True
>>> fair_split([1, 2, 4, 9])
False
>>> fair_split([])
True
```

Think first: What does the total weight tell you, and what single question is left?

The idea. An odd total can't be split. Otherwise, the question becomes: can some group of parcels weigh exactly half the total? Track the set of reachable sums, adding each parcel in turn.

```
def fair_split(weights):
    total = sum(weights)
    if total % 2:
        return False
    reachable = {0}  # sums that some group of parcels can make
    for w in weights:
        reachable |= {s + w for s in reachable}
    return total // 2 in reachable
```

Complexity: O(n × total) time and O(total) space.

Python's unlimited-size integers allow a compact trick: bit s of `sums` records whether sum s is reachable, and one shift and OR adds a parcel to every reachable sum at once.

```
def fair_split_bits(weights):
    total = sum(weights)
    if total % 2:
        return False
    sums = 1  # bit s is set when sum s is reachable
    for w in weights:
        sums |= sums << w
    return bool(sums >> (total // 2) & 1)
```

> **Tip:** The bit version is fast in practice, but explain the set version first. The interviewer is grading whether you see the "subset sum" structure, not the trick.

19.6 Longest Mirror (Medium)

Find the longest block of consecutive characters that reads the same backwards (a palindrome). If several are equally long, return the leftmost.

```
>>> longest_mirror('noonday')
'noon'
>>> longest_mirror('xyz')
'x'
>>> longest_mirror('')
''
>>> longest_mirror('a')
'a'
```

The DP view. `t[i][j]` is true when `text[i:j+1]` is a palindrome: its two end characters match, and the inside is a palindrome too. That takes $O(n^2)$ time and $O(n^2)$ space.

The better idea. Every palindrome has a center: a character (odd length) or the gap between two characters (even length). Expand outward from each of the 2n - 1 centers while the characters match. Same time, no table.

```
def longest_mirror(text):
    best_start, best_len = 0, 0
    for center in range(len(text)):
        for left, right in ((center, center), (center, center + 1)):
            while (
                left >= 0
                and right < len(text)
                and text[left] == text[right]
            ):
                left -= 1
                right += 1
            length = right - left - 1
            start = left + 1
            if length > best_len or (
                length == best_len and start < best_start
            ):
                best_start, best_len = start, length
    return text[best_start : best_start + best_len]
```

Complexity: $O(n^2)$ time, O(1) extra space.

> **Follow-Up:** Manacher's algorithm finds the answer in O(n), reusing information from palindromes already found. It is rarely expected in interviews; knowing that it exists is enough.

Key Takeaways

- Two-string problems use `t[i][j]` for the prefixes `a[:i]` and `b[:j]`, with an extra row and column for empty prefixes.
- Grid problems combine the cell above and the cell to the left; one row of memory is often enough.
- 0/1 knapsack loops capacities backwards; the unlimited version loops forwards.
- "Split into two equal groups" is subset sum in disguise.
- Before building a table, check whether a simpler structure, such as expanding from centers, does the same job.

CHAPTER 20

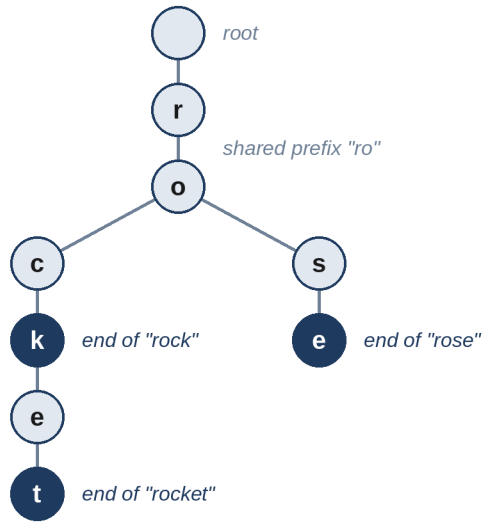
Tries and String Techniques

Strings appear in nearly every interview, usually combined with a pattern from earlier chapters: a hash map for anagrams, a sliding window for substrings, DP for edit distance. This chapter adds the tools specific to strings: the **trie**, or prefix tree, for searching many words by prefix; a few techniques for prefixes and encodings; and a linear-time algorithm for finding a pattern in text.

When to Use a Trie

- **Prefix queries** over a set of words: autocomplete, "does any word start with...?"
- **Searching for many words at once**, for example in a letter grid, sharing the work for words with a common start.
- Problems where a sorted list with binary search would also work. Mention both; a trie is easier to extend, a sorted list is simpler.

A trie stores words letter by letter in a tree. Each node maps a letter to a child node, and a marker records where a whole word ends. Words with the same prefix share the same path, so looking up a word or prefix of length L takes O(L) time, however many words are stored.



A trie holding "rock", "rocket" and "rose": the shared prefix "ro" is stored once, and markers show where whole words end.

20.1 Prefix Tree (Medium)

Implement a structure that adds words and answers two questions: is this exact word stored, and does any stored word start with this prefix?

```
>>> trie_session(
...     [('add', 'rocket'),
...      ('word', 'rocket'),
...      ('word', 'rock'),
...      ('prefix', 'rock'),
...      ('add', 'rock'),
...      ('word', 'rock')],
... )
[True, False, True, True]
```

The examples replay a session of operations: `('add', w)` stores w, `('word', w)` asks whether w is stored, and `('prefix', p)` asks whether any word starts with p. The output lists the answers to the questions.

The idea. Nested dictionaries make a compact trie in Python: each node is a dict from letter to child, and the key `"$"` marks the end of a word. Both

questions walk down the tree letter by letter; they differ only in what they check at the end.

```
class Trie:
    def __init__(self):
        self.root = {}  # each node is a dict: letter -> child node

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.setdefault(ch, {})
        node["$"] = True  # marks the end of a whole word

    def _walk(self, text):
        node = self.root
        for ch in text:
            if ch not in node:
                return None
            node = node[ch]
        return node

    def contains(self, word):
        node = self._walk(word)
        return node is not None and "$" in node

    def has_prefix(self, prefix):
        node = self._walk(prefix)
        return bool(node)  # an empty node means no word goes this way
```

Complexity: O(L) per operation for a word of length L; space proportional to the total length of the stored words.

> **Common Mistake:** In an early version of this solution, `has_prefix` returned `node is not None`. That looks right, but for the empty prefix the walk stops at the root, which exists even when the trie is empty, so `has_prefix("")` claimed a word existed in an empty trie. The random tests against a simple set-based version found the case. Returning `bool(node)` fixes it, because an empty root is an empty dict. Edge cases like the empty string are exactly where handwritten tests miss bugs and comparisons with a brute force find them.

20.2 Autocomplete (Medium)

After each character typed into a search box, suggest up to three product names that start with the text typed so far, in alphabetical order.

```
>>> autocomplete(
...     ['stapler', 'stand', 'stamp', 'stool', 'star', 'stack'],
...     'sta',
... )
[['stack', 'stamp', 'stand'],
 ['stack', 'stamp', 'stand'],
 ['stack', 'stamp', 'stand']]
>>> autocomplete(
...     ['stapler', 'stand', 'stamp', 'stool', 'star', 'stack'],
...     'stap',
... )
[['stack', 'stamp', 'stand'],
 ['stack', 'stamp', 'stand'],
 ['stack', 'stamp', 'stand'],
 ['stapler']]
>>> autocomplete(['a'], 'b')
[[]]
```

Think first: Once the names are sorted, where are all the names that start with a given prefix?

The idea. In a sorted list, all names with the same prefix sit next to each other, starting at the first name that is greater than or equal to the prefix. Binary search finds that position, and the next three names are the candidates, as long as they still start with the prefix.

```
def autocomplete(products, typed):
    products = sorted(products)
    out = []
    prefix = ""
    for ch in typed:
        prefix += ch
        i = bisect_left(products, prefix)  # first name >= prefix
        out.append([p for p in products[i : i + 3] if p.startswith(prefix)])
    return out
```

Complexity: O(n log n × L) to sort, then O(log n × L) per character typed.

> **Follow-Up:** A trie works too: store at each node the three best suggestions below it, so each keystroke is answered in O(1) after walking one level down. That's how you would scale it to rank by popularity rather than alphabetically.

20.3 Board Words (Hard)

Which words from a list can be traced on a letter grid, moving up, down, left or right, without using a cell twice in the same word?

```
>>> board_words(
...     [['c', 'a', 'r', 't'], ['a', 'o', 'e', 'n'], ['t', 'p', 'i', 's']],
...     ['cat', 'cap', 'pie', 'rent', 'spin'],
... )
['cat', 'pie', 'rent']
```

Brute force. Run the Grid Word search from Problem 15.4 for each word. With many words, that repeats the same work: words that start the same way explore the same paths.

The idea. Put all the words into a trie, then run one backtracking search from each cell, following the trie as you go. A path stops as soon as no word continues with the next letter, and every word found along the way is recorded.

```
def board_words(board, words):
    trie = {}
    for w in words:
        node = trie
        for ch in w:
            node = node.setdefault(ch, {})
        node["$"] = w
    rows, cols = len(board), len(board[0])
    found = set()

    def explore(r, c, node):
        ch = board[r][c]
        if ch not in node:
            return
        node = node[ch]
        if "$" in node:
            found.add(node["$"])
        board[r][c] = "#"  # in use
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != "#":
                explore(nr, nc, node)
        board[r][c] = ch

    for r in range(rows):
        for c in range(cols):
            explore(r, c, trie)
    return sorted(found)
```

Complexity: $O(r \times c \times 3^L)$ in the worst case for words up to length L, but the trie prunes most paths early, and the time hardly depends on the number of words.

> **Tip:** Storing the whole word at its end marker (`node["$"] = w`) avoids rebuilding the word from the path. A further optimization removes a word's marker once found, so it isn't reported twice, and removes trie branches that have no words left.

20.4 Shared Start (Easy)

Find the longest prefix shared by all the strings in a list.

```
>>> shared_start(['interview', 'internet', 'interval'])
'inter'
>>> shared_start(['cat', 'dog'])
''
>>> shared_start([])
''
>>> shared_start(['solo'])
'solo'
```

The idea. Compare the strings position by position against the first one, and stop at the first position where any string is too short or has a different character.

```
def shared_start(strings):
    if not strings:
        return ""
    first = strings[0]
    for i, ch in enumerate(first):
        for s in strings[1:]:
            if i == len(s) or s[i] != ch:
                return first[:i]
    return first
```

Complexity: O(total length) time, O(1) extra space.

A neat alternative: the alphabetically smallest and largest strings differ the most, so the prefix they share is shared by all.

```
def shared_start_minmax(strings):
    if not strings:
        return ""
    lo, hi = min(strings), max(strings)  # most different pair
    i = 0
    while i < len(lo) and lo[i] == hi[i]:
        i += 1
    return lo[:i]
```

20.5 Pack and Unpack (Medium)

Write two functions: one that turns a list of strings into a single string, and one that turns it back. The strings may contain any characters, including any separator you might choose.

```
>>> pack_and_unpack(['hello', '', 'a:b', '12:'])
['hello', '', 'a:b', '12:']
>>> pack_and_unpack([])
[]
```

Think first: Why does joining with a comma, or any other special character, fail?

The idea. Any separator might appear inside a string. Instead, write each string's *length* before it, followed by a colon: `"5:hello0:3:a:b"`. Reading it back, the number before the first colon says exactly how many characters to take, so colons inside the strings don't matter.

```
def pack(strings):
    return "".join(f"{len(s)}:{s}" for s in strings)  # length prefixes


def unpack(packed):
    out, i = [], 0
    while i < len(packed):
        colon = packed.index(":", i)
        length = int(packed[i:colon])
        start = colon + 1
        out.append(packed[start : start + length])
        i = start + length
    return out
```

Complexity: O(total length) for both.

> **Interviewer's View:** This is how many real network protocols frame messages: a length header, then the data. Saying so connects a puzzle to engineering practice, which interviewers value.

20.6 Find Every Match (Hard)

Return every position where a pattern occurs in a text, including overlapping matches, in O(n + m) time for text length n and pattern length m.

```
>>> find_all('abababa', 'aba')
[0, 2, 4]
>>> find_all('aaaa', 'aa')
[0, 1, 2]
>>> find_all('abc', 'd')
[]
>>> find_all('ab', '')
[0, 1, 2]
```

Brute force. Compare the pattern at every position: O(n × m) in the worst case, for example searching "aaab" in "aaaa...a".

The idea. The Knuth-Morris-Pratt (KMP) algorithm never moves backwards in the text. When a mismatch happens after k matching characters, it already knows those k characters, because they equal the pattern's first k. A table computed from the pattern alone, `fail`, says how much of the pattern is still matched after the mismatch: the longest prefix of the pattern that is also a suffix of what was matched.

```
def find_all(text, pattern):
    if not pattern:
        return list(range(len(text) + 1))
    # fail[i]: length of the longest proper prefix of pattern[:i+1]
    # that is also a suffix of it (the Knuth-Morris-Pratt table).
    fail = [0] * len(pattern)
    k = 0
    for i in range(1, len(pattern)):
        while k and pattern[i] != pattern[k]:
            k = fail[k - 1]
        if pattern[i] == pattern[k]:
            k += 1
        fail[i] = k
    out, k = [], 0
    for i, ch in enumerate(text):
        while k and ch != pattern[k]:
            k = fail[k - 1]  # fall back without moving i
        if ch == pattern[k]:
            k += 1
        if k == len(pattern):
            out.append(i - k + 1)
            k = fail[k - 1]
    return out
```

Complexity: O(n + m) time, O(m) space.

> **Tip:** In Python's own code, `text.find(pattern, start)` in a loop is the practical choice. Explain KMP when the interviewer asks for a guaranteed linear time, and ask whether they want you to implement it; many are content with a clear explanation of the `fail` table.

Key Takeaways

- A trie answers word and prefix questions in O(L), and lets one search look for many words at once.
- In a sorted list, all strings sharing a prefix are adjacent, so binary search finds them.
- Length-prefixed encoding works for any content; separator-based encoding doesn't.
- KMP finds all matches in linear time by never re-reading the text.
- Test strings with the empty string, one character, and repeated characters: that's where bugs hide.

CHAPTER 21

Bit Manipulation and Math

A few interview problems hinge on a small mathematical fact or a bit trick: XOR cancels pairs, `n & (n - 1)` clears the lowest set bit, a sum formula finds what's missing. These problems are short, and the insight is the whole answer, so the best preparation is to know the handful of facts that keep coming up.

The Essential Bit Operations

Expression	Meaning	Example (in binary)
`a & b`	AND: bits set in both	`1100 & 1010 = 1000`
`a | b`	OR: bits set in either	`1100 | 1010 = 1110`
`a ^ b`	XOR: bits set in exactly one	`1100 ^ 1010 = 0110`
`x << k`	shift left: multiply by 2^k	`0011 << 2 = 1100`
`x >> k`	shift right: divide by 2^k, rounding down	`1100 >> 2 = 0011`
`x & 1`	the lowest bit: 1 if x is odd	`0101 & 1 = 1`
`x & (x - 1)`	x with its lowest set bit cleared	`1100 → 1000`
`x & -x`	only the lowest set bit of x	`1100 → 0100`

Three facts about XOR solve a surprising number of problems: `x ^ x == 0`, `x ^ 0 == x`, and the order of XORs doesn't matter.

> **Warning:** Python integers have unlimited size, and negative numbers behave as if they had infinitely many leading 1 bits. Problems written for 32-bit integers in other languages sometimes need a mask, `x & 0xFFFFFFFF`, to get the same results in Python. Mention it if the problem talks about 32-bit values.

21.1 The Lonely Number (Easy)

In a list, every value appears exactly twice except one, which appears once. Find it in O(n) time and O(1) extra space.

```
>>> lonely([6, 3, 9, 3, 6])
9
>>> lonely([7])
7
>>> lonely([-3, 5, 5])
-3
```

Brute force. Count with a `Counter`: O(n) time, but O(n) space.

The idea. XOR all the values together. Each pair cancels to zero, whatever order the values come in, and the lonely value is all that remains.

```
def lonely(values):
    result = 0
    for v in values:
        result ^= v  # pairs cancel out: x ^ x == 0
    return result
```

Complexity: O(n) time, O(1) space.

> **Follow-Up:** "Now two values appear once." XOR everything to get `a ^ b`. Its lowest set bit is a bit where a and b differ; split all the values into two groups by that bit, and XOR each group separately.

21.2 Count the Ones (Easy)

For every number from 0 to n, how many 1 bits does its binary form contain?

```
>>> count_ones(5)
[0, 1, 1, 2, 1, 2]
>>> count_ones(0)
[0]
```

The idea. Shifting i right by one bit gives i // 2, which has the same bits as i except the last one. So the count for i is the count for i // 2, already computed, plus i's last bit. That's dynamic programming on the bits.

```
def count_ones(n):
    ones = [0] * (n + 1)
    for i in range(1, n + 1):
        ones[i] = ones[i >> 1] + (i & 1)  # i/2 has the same bits, shifted
    return ones
```

Complexity: O(n) time.

For a single number, Brian Kernighan's trick counts the 1 bits by clearing the lowest one until nothing is left, taking one step per 1 bit:

```
def count_ones_kernighan(n):
    out = []
    for i in range(n + 1):
        c = 0
        while i:
            i &= i - 1  # clears the lowest 1 bit
            c += 1
        out.append(c)
    return out
```

> **Tip:** In everyday Python, `bin(i).count("1")` or, since Python 3.10, `i.bit_count()` is the practical answer. Mention it, then show the technique if asked.

21.3 The Missing Ticket (Easy)

Tickets numbered 0 to n were issued, and n of them came back: one is missing. Which one?

```
>>> missing_ticket([4, 0, 1, 3, 5])
2
>>> missing_ticket([0])
1
>>> missing_ticket([])
0
```

The idea. The numbers 0 to n add up to n(n + 1)/2. Subtract the sum of the returned tickets, and the difference is the missing number.

```
def missing_ticket(tickets):
    n = len(tickets)
    return n * (n + 1) // 2 - sum(tickets)  # expected total minus actual
```

Complexity: O(n) time, O(1) space.

An XOR version avoids any concern about large sums in languages with fixed-size integers: XOR every index and every ticket together, and all numbers except the missing one cancel in pairs.

```
def missing_ticket_xor(tickets):
    result = len(tickets)
    for i, t in enumerate(tickets):
        result ^= i ^ t
    return result
```

21.4 Power of Two (Easy)

Is n a power of two? And, as a related warm-up: reverse the bits of a 32-bit unsigned number.

```
>>> is_power_of_two(1)
True
>>> is_power_of_two(16)
True
>>> is_power_of_two(18)
False
>>> is_power_of_two(0)
False
>>> is_power_of_two(-8)
False
```

The idea. A power of two has exactly one 1 bit. Clearing the lowest set bit with `n & (n - 1)` leaves zero exactly in that case. Zero and negative numbers must be excluded first.

```
def is_power_of_two(n):
    return n > 0 and n & (n - 1) == 0  # exactly one bit set
```

Complexity: O(1).

To reverse 32 bits, take the lowest bit of x 32 times, and shift each one into the result from the right:

```
def reverse_bits(x):
    result = 0
    for _ in range(32):
        result = (result << 1) | (x & 1)  # move x's lowest bit across
        x >>= 1
    return result
```

21.5 Fast Power (Medium)

Compute `base ** exp % mod` for very large exponents, using O(log exp) multiplications.

```
>>> fast_power(2, 10, 1000)
24
>>> fast_power(3, 0, 7)
1
>>> fast_power(5, 3, 1)
0
```

Think first: How can you compute x^{16} with only four multiplications?

The idea. Square repeatedly: x, x^2, x^4, x^8, x^{16}. Any exponent is a sum of powers of two, its binary digits, so multiply together the squares that correspond to its 1 bits. Taking the remainder after each step keeps the numbers small.

```
def fast_power(base, exp, mod):
    result = 1
    base %= mod
    while exp:
        if exp & 1:  # this bit of the exponent is set
            result = result * base % mod
        base = base * base % mod  # base, base^2, base^4, ...
        exp >>= 1
    return result % mod
```

Complexity: O(log exp) multiplications. Python's built-in `pow(base, exp, mod)` does the same, and the tests check that both agree.

> **Common Mistake:** Forgetting the case `mod == 1`, where every answer is 0. With exponent 0, the loop never runs and `result` stays 1, so the final `% mod` is needed.

21.6 Primes Below n (Medium)

List all prime numbers less than n.

```
>>> primes_below(10)
[2, 3, 5, 7]
>>> primes_below(2)
[]
>>> primes_below(3)
[2]
```

Brute force. Test each number by trying divisors up to its square root: about $O(n\sqrt{n})$ in total.

The idea. The Sieve of Eratosthenes, more than 2,000 years old, crosses out multiples instead. For each prime p, cross out its multiples from p^2 upward; smaller multiples, such as 2p, were already crossed out by smaller primes. The numbers left standing are prime.

```
def primes_below(n):
    if n < 3:
        return []
    is_prime = [True] * n
    is_prime[0] = is_prime[1] = False
    p = 2
    while p * p < n:
        if is_prime[p]:
            # Smaller multiples were already crossed out by smaller primes.
            for m in range(p * p, n, p):
                is_prime[m] = False
        p += 1
    return [i for i in range(n) if is_prime[i]]
```

Complexity: O(n log log n) time, which is nearly linear, and O(n) space.

Key Takeaways

- XOR cancels pairs: use it to find the odd one out.

- `n & (n - 1)` clears the lowest set bit: it counts bits and tests for powers of two.
- Sum formulas and XOR find a single missing number in O(1) space.
- Repeated squaring computes large powers in O(log n) steps.
- The sieve lists primes far faster than testing each number.
- Remember that Python integers are unbounded; mask when a problem assumes 32 bits.

CHAPTER 22

Designing Data Structures

"Design a cache", "design a hit counter": these problems ask you to build a small class whose methods meet stated time limits, usually O(1) or O(log n) per operation. They test whether you can **combine** basic structures, a hash map plus a linked list, a hash map plus a list, so that each covers the other's weakness. They are also popular because they look like real engineering work: an API, state, and trade-offs.

How to Approach a Design Problem

1. **List the operations and their required costs.** Write them down: `get` O(1), `put` O(1), evict the least recently used key.
2. **For each operation, name a structure that does it fast.** Lookup by key: a hash map. Order of use: a list. Random access by position: an array.
3. **Connect the structures** so that they stay consistent. Usually the hash map points into the other structure.
4. **Walk through every operation**, including the edge cases: an empty structure, a full one, a repeated key.

The examples in this chapter replay a *session*: a list of operations applied to a fresh object, with the results of the queries collected in a list. That's the format most interview platforms use, and it's how the tests drive each class.

22.1 Recently Used Cache (Medium)

Design a cache with a fixed capacity. `get(key)` returns the value, or -1 if the key is absent. `put(key, value)` stores a value; if the cache is full, it first evicts the **least recently used** key. Both operations count as a "use", and both must take O(1) time.

```
>>> lru_session(
...     2,
...     [('put', 'a', 1),
...      ('put', 'b', 2),
...      ('get', 'a'),
...      ('put', 'c', 3),
...      ('get', 'b'),
...      ('get', 'c'),
...      ('put', 'd', 4),
...      ('get', 'a'),
...      ('get', 'd')],
... )
[1, -1, 3, -1, 4]
```

Think first: A hash map gives O(1) lookup, but how do you find the least recently used key in O(1), and move a key to "most recent" in O(1)?

The idea. Keep the keys in a **doubly linked list** ordered from oldest to newest, and a **hash map** from each key to its list node. Using a key unlinks its node and appends it at the newest end, both O(1) given the node. Evicting removes the node at the oldest end. Two *sentinel* nodes, `head` and `tail`, which never hold data, mean the list is never empty, so there are no special cases for the first or last node.

```
class Node:
    def __init__(self, key=None, value=None):
        self.key, self.value = key, value
        self.prev = self.next = None


class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.nodes = {}  # key -> node in the list
        self.head, self.tail = Node(), Node()  # sentinels: oldest..newest
        self.head.next, self.tail.prev = self.tail, self.head

    def _unlink(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _append(self, node):  # newest goes just before the tail
        node.prev, node.next = self.tail.prev, self.tail
        self.tail.prev.next = node
        self.tail.prev = node
```

```
    def get(self, key):
        node = self.nodes.get(key)
        if node is None:
            return -1
        self._unlink(node)
        self._append(node)  # now the most recently used
        return node.value

    def put(self, key, value):
        if key in self.nodes:
            self._unlink(self.nodes.pop(key))
        elif len(self.nodes) == self.capacity:
            oldest = self.head.next
            self._unlink(oldest)
            del self.nodes[oldest.key]
        node = Node(key, value)
        self.nodes[key] = node
        self._append(node)
```

Complexity: O(1) per operation; O(capacity) space.

In Python, `OrderedDict` already combines a hash map with a linked list, so the whole class shrinks to a few lines:

```
class LRUCacheOrdered:
    def __init__(self, capacity):
        self.capacity = capacity
        self.data = OrderedDict()

    def get(self, key):
        if key not in self.data:
            return -1
        self.data.move_to_end(key)
        return self.data[key]

    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)
```

> **Interviewer's View:** Most interviewers accept the `OrderedDict` version only after you've shown you could build it yourself. A good approach: say "in production I'd use `OrderedDict`", then ask whether they want the version with an explicit linked list.

> **Common Mistake:** Forgetting to delete the evicted key from the hash map, or to update the value when `put` is called with an existing key. The random tests against `OrderedDict` catch both.

22.2 Hit Counter (Medium)

Record hits with a timestamp in seconds; timestamps never decrease. `count(t)` returns the number of hits in the last 300 seconds, that is, with a timestamp in the range (t - 300, t].

```
>>> hit_counter_session(
...     [('hit', 10),
...      ('hit', 10),
...      ('hit', 200),
...      ('count', 250),
...      ('hit', 400),
...      ('count', 400),
...      ('count', 500)],
... )
[3, 2, 1]
```

The idea. Keep hits in a queue, oldest first. A query first discards hits that are too old; since time never goes backwards, they can never count again. Grouping hits in the same second into one `(timestamp, count)` entry keeps the queue short under heavy traffic.

```
class HitCounter:
    WINDOW = 300

    def __init__(self):
        self.hits = deque()  # (timestamp, count), oldest first

    def hit(self, t):
        if self.hits and self.hits[-1][0] == t:
            self.hits[-1] = (t, self.hits[-1][1] + 1)  # group same second
        else:
            self.hits.append((t, 1))

    def count(self, t):
        while self.hits and self.hits[0][0] <= t - self.WINDOW:
            self.hits.popleft()  # too old to ever count again
        return sum(c for _, c in self.hits)
```

Complexity: amortized O(1) for `hit`; `count` is O(entries in the window), at most 300 thanks to the grouping. Keeping a running total, adjusted as entries enter and leave, makes `count` amortized O(1) too.

> **Follow-Up:** "Hits arrive at millions per second from many servers." Discuss a fixed array of 300 buckets indexed by `t % 300`, which uses constant memory, and then sharding the counter across machines and summing the shards. That turns a coding question into a small system design discussion (Chapter 25).

22.3 Versioned Store (Medium)

Design a key-value store that keeps history. `set(key, value, time)` records a value at a time, and `get(key, time)` returns the value as of that time: the value set at the latest time less than or equal to it, or "" if there was none. Times for each key are given in increasing order.

```
>>> store_session(
...     [('set', 'price', '10', 1),
...      ('get', 'price', 1),
...      ('get', 'price', 3),
...      ('set', 'price', '12', 4),
...      ('get', 'price', 4),
...      ('get', 'price', 5),
...      ('get', 'price', 0)],
... )
['10', '10', '12', '12', '']
```

The idea. For each key, append the times and values to two lists. Because times arrive in increasing order, the lists stay sorted for free, and `get` is a binary search for the last time that is not after the query time.

```
class VersionedStore:
    def __init__(self):
        self.times = defaultdict(list)  # key -> sorted times
        self.values = defaultdict(list)  # key -> values in the same order

    def set(self, key, value, time):
        self.times[key].append(time)
        self.values[key].append(value)

    def get(self, key, time):
        i = bisect_right(self.times[key], time)  # first time > time
        return self.values[key][i - 1] if i else ""
```

Complexity: O(1) amortized for `set`, O(log n) for `get`.

22.4 Random Pick Set (Medium)

Design a set with `add(x)` and `remove(x)`, each returning whether the set changed, and `pick()`, which returns a member chosen uniformly at random. All must take O(1) on average.

```
>>> random_set_session(
...     [('add', 1), ('add', 1), ('remove', 2), ('add', 2), ('remove', 1)],
... )
([True, False, False, True, True], [2])
```

Think first: A Python `set` adds and removes in O(1), but can't pick a random member in O(1). A list can pick randomly, but removing from the middle is O(n). How do you get both?

The idea. Store the members in a list for random picking, plus a hash map from each member to its index. To remove a member in O(1), move the *last* item into its slot, update that item's index, and pop the end of the list. The order of the list doesn't matter, so the swap is harmless.

```
class RandomSet:
    def __init__(self, rng=random):
        self.items = []  # the members, in no particular order
        self.where = {}  # member -> its index in items
        self.rng = rng

    def add(self, x):
        if x in self.where:
            return False
        self.where[x] = len(self.items)
        self.items.append(x)
        return True

    def remove(self, x):
        if x not in self.where:
            return False
        i, last = self.where[x], self.items[-1]
        self.items[i] = last  # move the last item into x's slot
        self.where[last] = i
        self.items.pop()
        del self.where[x]
        return True

    def pick(self):
        return self.rng.choice(self.items)
```

Complexity: O(1) average per operation.

> **Tip:** How do you test randomness? This book's test picks 30,000 times from three members with a fixed random seed and checks that each count is between 9,000 and 11,000. Mentioning a statistical check like this answers the inevitable "how would you test it?"

22.5 Least-Frequently Used Cache (Hard)

A variation of 22.1: when the cache is full, evict the key used the **fewest times**; among keys tied for the fewest uses, evict the least recently used. Every operation must still be O(1).

```
>>> lfu_session(
...     2,
...     [('put', 'a', 1),
...      ('put', 'b', 2),
...      ('get', 'a'),
...      ('put', 'c', 3),
...      ('get', 'b'),
...      ('get', 'c'),
...      ('get', 'c'),
...      ('put', 'd', 4),
...      ('get', 'a'),
...      ('get', 'c'),
...      ('get', 'd')],
... )
[1, -1, 3, 3, -1, 3, 4]
```

The idea. Group keys into buckets by their use count, and keep each bucket in least-recently-used order: a dictionary from count to an `OrderedDict` of keys. Using a key moves it from bucket u to bucket u + 1. Track `min_uses`, the smallest non-empty bucket: a new key always resets it to 1, and touching a key raises it only when the key's old bucket becomes empty and was the minimum.

```
class LFUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.data = {}  # key -> (value, uses)
        self.by_uses = defaultdict(OrderedDict)  # uses -> keys, LRU first
        self.min_uses = 0

    def _touch(self, key):
        value, uses = self.data[key]
        del self.by_uses[uses][key]
        if not self.by_uses[uses] and self.min_uses == uses:
            self.min_uses += 1  # that bucket is empty now
        self.data[key] = (value, uses + 1)
```

```
        self.by_uses[uses + 1][key] = None

    def get(self, key):
        if key not in self.data:
            return -1
        self._touch(key)
        return self.data[key][0]

    def put(self, key, value):
        if self.capacity == 0:
            return
        if key in self.data:
            self.data[key] = (value, self.data[key][1])
            self._touch(key)
            return
        if len(self.data) == self.capacity:
            old, _ = self.by_uses[self.min_uses].popitem(last=False)
            del self.data[old]
        self.data[key] = (value, 1)
        self.by_uses[1][key] = None
        self.min_uses = 1
```

Complexity: O(1) per operation.

> **Interviewer's View:** This is a hard problem mainly because there are many states to keep consistent. Talk through a sequence of operations on the whiteboard before writing code; then the code is a transcription. A simple "brute force" class that scans for the victim, as in this book's tests, is a good first step if time is short.

22.6 Flatten Iterator (Medium)

Write an iterator that yields the integers of an arbitrarily nested list in order, lazily: one at a time, without building the flattened list first.

```
>>> flatten([1, [2, [3, []]], 4])
[1, 2, 3, 4]
>>> flatten([[], [[]]])
[]
```

The simple idea. A recursive generator with `yield from` is three lines:

```
def flatten_gen(nested):
    for item in nested:
        if isinstance(item, list):
            yield from flatten_gen(item)
        else:
            yield item
```

The robust idea. Recursion fails on lists nested more deeply than Python's recursion limit, about 1,000 levels. An explicit stack of iterators, one for each list currently open, removes that limit: take the next item from the top iterator; open a new list by pushing its iterator; pop iterators that are finished.

```
class Flattener:
    def __init__(self, nested):
        self.stack = [iter(nested)]  # one iterator per open list

    def __iter__(self):
        return self

    def __next__(self):
        while self.stack:
            try:
                item = next(self.stack[-1])
            except StopIteration:
                self.stack.pop()  # this list is finished
                continue
            if isinstance(item, list):
                self.stack.append(iter(item))
            else:
                return item
        raise StopIteration
```

Complexity: O(1) amortized per item; O(depth) space.

> **Follow-Up:** "Add `has_next()`." Python iterators don't have one, but Java's do. Implement it by fetching the next item in advance and keeping it until `next()` is called.

Key Takeaways

- Design problems combine structures: a hash map for lookup, plus a list, linked list or heap for order.
- The hash map usually stores *where* an item is in the other structure, so both can be updated in O(1).

- Sentinel nodes remove the special cases at the ends of a linked list.
- Swap-with-last makes removal from an unordered list O(1).
- Test design problems with long random sessions against a simple, slow reference class.

PART III

Beyond Algorithms

CHAPTER 23

Practical Coding Rounds

Not every coding interview is a puzzle. Many companies also run rounds that look more like a day at work: parse a log file, join two tables, validate a sequence of commands, build a rate limiter. Candidates report these "practical" or "real-world" rounds especially often at companies that build physical products and infrastructure, and in later rounds for experienced engineers. The algorithms are simple; what's being tested is whether your code is **correct on messy input, readable, and easy to change**.

What Changes in a Practical Round

Algorithm round	Practical round
Clean input, exactly as described	Malformed lines, missing fields, unexpected values
One clever idea	Several simple steps, done carefully
Optimal complexity is the goal	Clarity and correctness come first; efficiency where it matters
One function	Often a small class or several functions, sometimes in an existing codebase
Follow-ups make it faster	Follow-ups add requirements: "now also report...", "now handle..."

Habits that score well:

- **Ask what to do with bad input**, and then do it consistently: skip and count, raise an error, or use a default.
- **Name things after the domain**: `orders`, `status`, `slowest`, not `a`, `x`, `tmp`.
- **Keep parsing separate from the logic.** A function that turns text into records, and another that works on records, are each easy to test and change.
- **Use the standard library**: `csv`, `re`, `collections`, `datetime`, `json`. Writing your own CSV parser is a red flag, because quoted fields with commas will break it.
- **Test as you go**, with one normal line, one bad line and one edge case.

23.1 Log Summary (Medium)

A web server writes one line per request, such as `2026-10-01T12:00:03Z GET /api/orders 200 123ms`: a timestamp, a method, a path, a status code and a response time. For each path, report the number of requests, the number of server errors (status 500 or above) and the slowest response time. Skip lines that don't match the format, and report how many were skipped.

```
>>> log_summary(
...     ['2026-10-01T12:00:03Z GET /api/orders 200 123ms',
...      '2026-10-01T12:00:04Z POST /api/orders 503 870ms',
...      '2026-10-01T12:00:05Z GET /health 200 2ms',
...      'garbage line',
...      '2026-10-01T12:00:06Z GET /api/orders 200 98ms'],
... )
({'/api/orders': (3, 1, 870), '/health': (1, 0, 2)}, 1)
>>> log_summary([])
({}, 0)
```

Think first: What are all the ways a line could be malformed?

The idea. One regular expression describes a valid line exactly, with named groups for the fields; anything that doesn't match is skipped and counted. A dictionary accumulates the statistics for each path.

```
LINE = re.compile(
    r"^\S+ (?:GET|POST|PUT|DELETE) (?P<path>/\S*) (?P<status>\d{3}) "
    r"(?P<ms>\d+)ms$"
)


def log_summary(lines):
    stats = {}
    skipped = 0
    for line in lines:
        m = LINE.match(line.strip())
        if not m:
            skipped += 1
            continue
        path, status, ms = m["path"], int(m["status"]), int(m["ms"])
        count, errors, slowest = stats.get(path, (0, 0, 0))
        stats[path] = (
            count + 1,
            errors + (status >= 500),
            max(slowest, ms),
        )
    return dict(sorted(stats.items())), skipped
```

Complexity: O(total length of the lines).

A version without regular expressions, which splits each line and checks each field, gives the same results on thousands of random lines, including deliberately broken ones such as a four-digit status or a missing time. Both are fine in an interview; the regular expression is shorter, the explicit checks are easier for some interviewers to read.

> **Interviewer's View:** Practical rounds often add a requirement halfway through: "also report the 95th-percentile time" or "group by hour". If your parsing is separate from your counting, the change is small. Say this when you structure the code.

23.2 Telemetry Spikes (Medium)

A sensor sends a stream of readings. Flag every reading that differs from the average of the previous k readings by more than a limit. The first k readings are never flagged. Return the indices of the flagged readings.

```
>>> spikes([10, 11, 10, 30, 10, 11, 9], 3, 5)
[3, 4, 5, 6]
>>> spikes([5, 5], 3, 1)
[]
```

Brute force. Recompute the average of the previous k readings at every step: O(n × k).

The idea. A sliding window with a running sum (Chapter 7): add each reading after checking it, and subtract the reading that leaves the window. Each step is O(1).

```
def spikes(readings, k, limit):
    window = deque()
    total = 0.0
    flagged = []
    for i, r in enumerate(readings):
        if len(window) == k and abs(r - total / k) > limit:
            flagged.append(i)
        window.append(r)
        total += r
        if len(window) > k:
            total -= window.popleft()  # keep a running sum, O(1) per step
    return flagged
```

Complexity: O(n) time, O(k) space.

Look closely at the first example. The spike at index 3 is flagged, as it should be, but so are the normal readings at indices 4, 5 and 6. The spike is still inside the window, pulling the average up to about 17, so readings near 10 now look abnormal. The code does exactly what the specification says, and the specification is the problem.

> **Follow-Up:** "How would you stop one spike from flagging the readings after it?" Good answers: leave flagged readings out of the window; compare with the *median* of the window, which a single outlier barely moves; or use an exponentially weighted average with a limit that adapts to the noise. Raising this yourself, before the interviewer does, is exactly the kind of engineering judgment practical rounds look for.

23.3 Launch Sequence Validator (Medium)

A (fictional) rocket controller moves through states: IDLE, FUELING, ARMED, IGNITION and LIFTOFF, plus SAFED after an abort. Given a list of events, return the final state, or the index of the first event that isn't allowed in the current state.

```
IDLE --fuel--> FUELING --arm--> ARMED --ignite--> IGNITION
IGNITION --release--> LIFTOFF
any state before LIFTOFF --abort--> SAFED
SAFED --reset--> IDLE
```

```
>>> run_sequence(['fuel', 'arm', 'ignite', 'release'])
('ok', 'LIFTOFF')
>>> run_sequence(['fuel', 'ignite'])
('invalid', 1)
>>> run_sequence(['fuel', 'abort', 'reset', 'fuel'])
('ok', 'FUELING')
>>> run_sequence(['fuel', 'arm', 'ignite', 'release', 'abort'])
('invalid', 4)
>>> run_sequence([])
('ok', 'IDLE')
```

The idea. A *state machine* is best written as data: a dictionary from (state, event) to the next state. The loop then only looks up each transition. The rule "abort is allowed from any state before liftoff" is a set of states rather than five separate entries.

```
TRANSITIONS = {
    ("IDLE", "fuel"): "FUELING",
    ("FUELING", "arm"): "ARMED",
    ("ARMED", "ignite"): "IGNITION",
    ("IGNITION", "release"): "LIFTOFF",
    ("SAFED", "reset"): "IDLE",
}


ABORTABLE = {"IDLE", "FUELING", "ARMED", "IGNITION"}


def run_sequence(events):
    state = "IDLE"
    for i, event in enumerate(events):
        if event == "abort" and state in ABORTABLE:
            state = "SAFED"
        elif (state, event) in TRANSITIONS:
            state = TRANSITIONS[(state, event)]
        else:
            return ("invalid", i)
    return ("ok", state)
```

Complexity: O(number of events).

The same logic as a chain of `if` statements works too, and the tests compare the two, but every new state or event means editing the chain. With a table, adding a "hold" state is one line, and the table can be printed or drawn as a diagram for review.

> **Tip:** Safety-critical systems are full of state machines: flight software, medical devices, payment flows. In an interview with an engineering company, mentioning that invalid transitions should be *rejected and logged*, never silently ignored, shows you understand why the design matters.

23.4 Join Two Tables (Medium)

Two CSV files arrive as text: orders (`order_id,customer_id,amount`) and customers (`customer_id,name`). Produce `(order_id, customer_name, amount)` for every order, sorted by order ID. Orders whose customer is missing get the name "UNKNOWN". (In database terms, this is a *left join*.)

```
>>> join_tables(
...     'order_id,customer_id,amount\n3,c2,9.50\n1,c1,20\n2,c9,5\n',
...     'customer_id,name\nc1,Asha\nc2,"Lee, Jordan"\n',
... )
[(1, 'Asha', 20.0), (2, 'UNKNOWN', 5.0), (3, 'Lee, Jordan', 9.5)]
```

Brute force. For each order, search the customer list: O(orders × customers).

The idea. Build a dictionary from customer ID to name first, then look up each order in O(1). That's a *hash join*, the same method databases use for this kind of query.

```
def join_tables(orders_csv, customers_csv):
    names = {}
    for row in csv.DictReader(io.StringIO(customers_csv)):
        names[row["customer_id"]] = row["name"]  # O(1) lookups later
    out = []
    for row in csv.DictReader(io.StringIO(orders_csv)):
        out.append(
            (
                int(row["order_id"]),
                names.get(row["customer_id"], "UNKNOWN"),
                float(row["amount"]),
            )
        )
```

```
    return sorted(out)
```

Complexity: O(orders + customers) plus the sort.

> **Common Mistake:** Splitting lines on commas yourself. The customer `"Lee, Jordan"` contains a comma inside quotes, which `line.split(",")` breaks into two fields. The `csv` module handles quoting correctly; using it is the professional answer.

23.5 Token Bucket Rate Limiter (Medium)

A service limits each client with a *token bucket*: the bucket holds up to `capacity` tokens and refills continuously at `rate` tokens per second. Each request takes one token if one is available, and is rejected otherwise. The bucket starts full. Given request times in seconds, in non-decreasing order, return whether each request is allowed.

```
>>> limit(2, 1, [0, 0, 0, 1, 1.5, 3])
[True, True, False, True, False, True]
```

Think first: Do you need a timer that adds tokens every second?

The idea. No timer is needed. Store the number of tokens and the time of the last request. When a request arrives, add the tokens earned since then, capped at the capacity, and then try to take one. This *lazy* refill computes the state only when it's needed.

```
class TokenBucket:
    def __init__(self, capacity, rate):
        self.capacity, self.rate = capacity, rate
        self.tokens = float(capacity)
        self.last = None

    def allow(self, t):
        if self.last is not None:
            elapsed = t - self.last
            self.tokens = min(
                self.capacity, self.tokens + elapsed * self.rate
            )
        self.last = t
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False
```

Complexity: O(1) per request, O(1) memory per client.

The test reference recomputes the bucket from the beginning for every request, a slow but obviously correct "replay". Comparing the two on random request times is how you would test a rate limiter in real life.

> **Follow-Up:** "How does this work across many servers?" Store each client's bucket (tokens and last time) in a shared store such as Redis, and update it atomically. Chapter 25 discusses rate limiting as a system design topic.

23.6 Config Diff (Medium)

Compare two nested configuration dictionaries, and list the differences as `(dotted.path, kind, old, new)` tuples sorted by path, where kind is "added", "removed" or "changed".

```
>>> config_diff(
...     {'db': {'host': 'a.example', 'port': 5432}, 'debug': True},
...     {'db': {'host': 'b.example', 'port': 5432, 'pool': 10}},
... )
[('db.host', 'changed', 'a.example', 'b.example'),
 ('db.pool', 'added', None, 10),
 ('debug', 'removed', True, None)]
>>> config_diff({}, {})
[]
```

The idea. Recurse into keys that are dictionaries on both sides, extending the path; compare everything else directly. Iterating over the sorted union of the keys produces the output in order.

```
def config_diff(old, new, prefix=""):
    changes = []
    for key in sorted(set(old) | set(new)):
        path = f"{prefix}{key}"
        if key not in new:
            changes.append((path, "removed", old[key], None))
        elif key not in old:
            changes.append((path, "added", None, new[key]))
        elif isinstance(old[key], dict) and isinstance(new[key], dict):
            changes.extend(config_diff(old[key], new[key], path + "."))
        elif old[key] != new[key]:
            changes.append((path, "changed", old[key], new[key]))
    return changes
```

Complexity: O(total number of keys) plus sorting at each level.

How do you test a diff when there is no simpler program to compare against? This book's tests use a **property**: applying the changes to the old configuration must produce the new one. A small `apply_changes` function makes that check, and thousands of random nested dictionaries confirm it.

```
def apply_changes(old, changes):
    """Apply a diff to a copy of old; used to check config_diff."""
    import copy

    result = copy.deepcopy(old)
    for path, kind, _, new_value in changes:
        *parents, last = path.split(".")
        node = result
        for p in parents:
            node = node[p]
        if kind == "removed":
            del node[last]
        else:
            node[last] = new_value
    return result
```

An earlier, "flattened" reference implementation disagreed with this one when a key changed from a dictionary to a plain value: one reported a single change, the other reported every old nested key as removed and the new value as added. Neither was wrong; the specification didn't say. The property test settles what matters, that the diff is complete and correct, without forcing one arbitrary choice.

> **Interviewer's View:** Ask about ambiguous cases like this one before coding: "If `db` changes from a dictionary to a string, should that be one change or several?" Interviewers in practical rounds often leave such gaps deliberately.

23.7 Parse a Duration (Easy)

Turn strings such as "1h30m", "45s" or "2h5s" into a number of seconds. Each of the units h, m and s may appear at most once, in that order. Anything else is an error. (The examples show errors as `'error'`; the function itself raises `ValueError`.)

```
>>> parse_duration('1h30m')
5400
>>> parse_duration('45s')
45
>>> parse_duration('2h5s')
```

```
7205
>>> parse_duration('')
'error'
>>> parse_duration('5m1h')
'error'
>>> parse_duration('10')
'error'
>>> parse_duration('1h1h')
'error'
>>> parse_duration('h')
'error'
```

The idea. A regular expression with three optional groups describes every valid string. `fullmatch` requires the whole string to match, and an explicit check rejects the empty string, which matches a pattern where everything is optional.

```
PATTERN = re.compile(r"(?:(\d+)h)?(?:(\d+)m)?(?:(\d+)s)?")


def parse_duration(text):
    m = PATTERN.fullmatch(text)
    if not text or not m:
        raise ValueError(f"bad duration: {text!r}")
    h, mins, s = (int(g) if g else 0 for g in m.groups())
    return h * 3600 + mins * 60 + s
```

Complexity: O(length of the text).

> **Common Mistake:** Using `match` or `search` instead of `fullmatch`. Then "1h30mXYZ" would be accepted as 5,400 seconds, ignoring the garbage at the end. Validation code should always describe the entire input.

Key Takeaways

- Practical rounds reward clean structure, careful handling of bad input, and the standard library.
- Write state machines and rules as data (dictionaries and sets), not long `if` chains.
- Build a dictionary first, then look up: the hash join turns O(n × m) into O(n + m).
- When there's no simple reference to compare against, test a property, such as "applying the diff gives the new version".

- Question the specification: a spike that corrupts its own baseline is a design flaw, and noticing it is a strong signal.

CHAPTER 24

Object-Oriented Design

An object-oriented design (OOD) interview asks you to model a small system in code: a parking garage, a vending machine, a library, a card game. There is no single right answer. The interviewer wants to see how you turn vague requirements into classes with clear responsibilities, how you keep the design open to change, and whether you can write the core of it in working code within the hour. OOD rounds are more common for experienced engineers, and at companies whose products involve many interacting objects, but anyone can be asked a lighter version.

A Repeatable Approach

1. **Clarify the scope (5 minutes).** Who uses the system? What are the main actions? What's out of scope? Write a short list of use cases: "a rider rents a bike", "a rider returns a bike to any station", "the system charges a fee".
2. **Find the nouns and verbs.** Nouns become candidate classes (rider, bike, station, ride); verbs become methods (rent, return, charge).
3. **Assign responsibilities.** Each class should have one clear job and own its data. A station knows its docks; a pricing plan knows how to compute a fee; the system coordinates.
4. **Identify what varies.** Wherever the requirements say "different kinds of" (bikes, plans, payment methods), plan for an interface with several implementations, rather than `if` statements on a type.
5. **Write the core code**, then walk through a use case, including a failure.

6. **Discuss extensions**: what changes if we add a new bike type, reservations, or a second city?

Principles Worth Naming

You don't need to recite design-pattern catalogs, but a few principles help you explain choices:

- **Encapsulation:** an object protects its own state; other objects call its methods instead of changing its data directly. `station.dock(bike)` checks capacity; code outside the class never appends to the list.
- **Single responsibility:** a class has one reason to change. Pricing rules change for business reasons; docking rules change for hardware reasons. Keep them in separate classes.
- **Open for extension, closed for modification:** adding a new pricing plan should mean writing a new class, not editing existing ones.
- **Composition over inheritance:** a rider *has* a pricing plan, rather than being a "MemberRider" subclass. Riders can then change plans without becoming a different object.
- **Fail loudly:** an invalid action (renting a second bike, returning to a full station) raises a clear error instead of corrupting state.

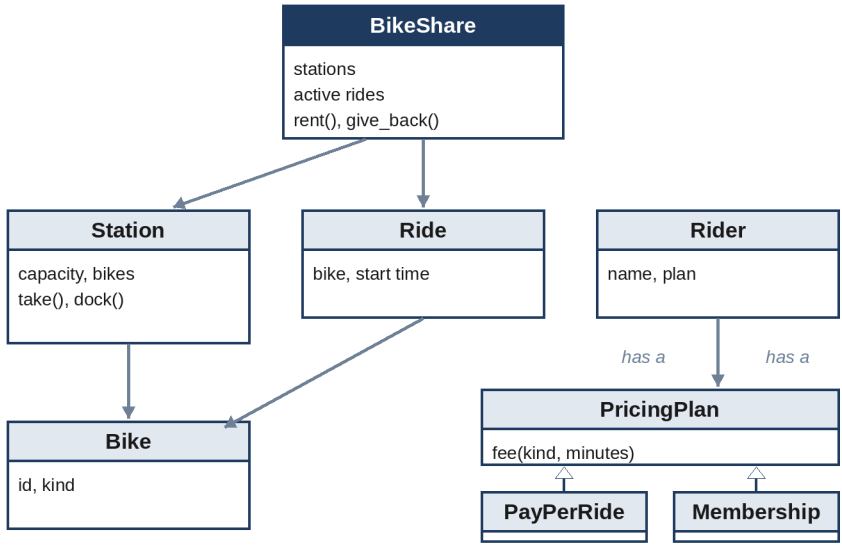
Worked Example: A Bike-Share System

The requirements, after clarifying: stations have a fixed number of docks. Bikes are classic or electric. A rider rents one bike at a time from a station and returns it to any station with a free dock. Casual riders pay an unlock fee plus a per-minute rate, higher for electric bikes; members get a lower electric rate, and classic rides up to 45 minutes are included. Payments, maps and maintenance are out of scope.

The simple, stable parts are data classes and an enumeration:

```
class BikeKind(Enum):
    CLASSIC = "classic"
    ELECTRIC = "electric"


@dataclass
```



Classes in the bike-share design: BikeShare coordinates Stations and Rides; Riders have a PricingPlan, with PayPerRide and Membership as two implementations.

```
class Bike:
    bike_id: str
    kind: BikeKind


@dataclass
class Station:
    station_id: str
    capacity: int
    bikes: list = field(default_factory=list)

    def has_free_dock(self):
        return len(self.bikes) < self.capacity

    def take(self, kind):
        for i, bike in enumerate(self.bikes):
            if bike.kind == kind:
                return self.bikes.pop(i)
        return None

    def dock(self, bike):
        if not self.has_free_dock():
            raise ValueError(f"station {self.station_id} is full")
        self.bikes.append(bike)
```

What varies is pricing, so it's an abstract base class with one method, and each plan is a small class:

```
class PricingPlan(ABC):
    @abstractmethod
    def fee(self, kind, minutes):
        """The fee in cents for one ride."""


class PayPerRide(PricingPlan):
    UNLOCK = 100
    PER_MINUTE = {BikeKind.CLASSIC: 20, BikeKind.ELECTRIC: 40}

    def fee(self, kind, minutes):
        return self.UNLOCK + self.PER_MINUTE[kind] * minutes


class Membership(PricingPlan):
    FREE_MINUTES = 45  # classic rides up to this length are included
    OVER_PER_MINUTE = 20  # classic minutes beyond the free ones
    ELECTRIC_PER_MINUTE = 15

    def fee(self, kind, minutes):
        if kind == BikeKind.ELECTRIC:
            return self.ELECTRIC_PER_MINUTE * minutes
        return self.OVER_PER_MINUTE * max(0, minutes - self.FREE_MINUTES)
```

The coordinating class keeps track of stations and of the ride each rider currently has. Notice that it never computes a fee itself: it asks the rider's plan.

```
@dataclass
class Rider:
    name: str
    plan: PricingPlan


@dataclass
class Ride:
    bike: Bike
    start: int


class BikeShare:
    def __init__(self):
        self.stations = {}
        self.active = {}  # rider name -> current Ride

    def add_station(self, station_id, capacity, bikes=()):
        station = Station(station_id, capacity)
        for bike in bikes:
            station.dock(bike)
        self.stations[station_id] = station

    def rent(self, rider, station_id, kind, now):
        if rider.name in self.active:
            raise ValueError(f"{rider.name} already has a bike")
```

```
        bike = self.stations[station_id].take(kind)
        if bike is None:
            raise ValueError(f"no {kind.value} bike at {station_id}")
        self.active[rider.name] = Ride(bike, now)
        return bike.bike_id

    def give_back(self, rider, station_id, now):
        ride = self.active.get(rider.name)
        if ride is None:
            raise ValueError(f"{rider.name} has no bike")
        self.stations[station_id].dock(ride.bike)  # raises if full
        del self.active[rider.name]
        return rider.plan.fee(ride.bike.kind, now - ride.start)
```

A short session shows the design at work, including a failed return to a full station:

```
def demo():
    share = BikeShare()
    share.add_station(
        "Harbor",
        3,
        [Bike("C1", BikeKind.CLASSIC), Bike("E1", BikeKind.ELECTRIC)],
    )
    share.add_station("Museum", 1)
    ana = Rider("Ana", PayPerRide())
    ben = Rider("Ben", Membership())
    print(share.rent(ana, "Harbor", BikeKind.ELECTRIC, now=0))
    print(share.rent(ben, "Harbor", BikeKind.CLASSIC, now=5))
    print(share.give_back(ana, "Museum", now=12))  # 100 + 12 * 40
    try:
        share.give_back(ben, "Museum", now=30)
    except ValueError as err:
        print("error:", err)
    print(share.give_back(ben, "Harbor", now=65))  # 60 min: 15 over
```

```
E1
C1
580
error: station Museum is full
300
```

The book's test for this design checks every failure path (a second rental, no bike of the requested kind, returning without a bike, returning to a full station) as well as the fees.

> **Interviewer's View:** The strongest signal in this example is the `PricingPlan` interface. When the interviewer asks "what if we add a student plan?", the answer is "one new class, no changes elsewhere". Designs full of `if rider.type == "member"` checks make that same change risky.

Common Follow-Up Questions

- **"How would you add reservations?"** A `Reservation` object holds a bike at a station for a few minutes. `Station.take` must then skip reserved bikes, so the change stays inside the class that owns the bikes.
- **"Two riders try to take the last bike at the same moment."** In a real service, the station's bike list is shared state. Discuss a lock per station, or an atomic database update that succeeds for only one rider. Naming the race condition is more important than the exact mechanism.
- **"How would you store this?"** Tables for stations, bikes, riders and rides; a ride row is created at rental and completed at return. That's a bridge to system design (Chapter 25).
- **"How would you test it?"** Unit tests for each pricing plan's fees at the boundaries (45 and 46 minutes), and scenario tests for the failure paths.

Other Classic Design Prompts

Practice the same approach on these prompts. For each, list the use cases, the classes, the one or two things that vary, and the failure cases.

Prompt	What usually varies	A failure case to handle
Parking garage	Vehicle and spot sizes; pricing	Garage full; a car leaves without a ticket
Vending machine	Payment method; product types	Product sold out; not enough change
Elevator controller	Scheduling strategy	Requests while moving; overweight
Library lending	Item types (book, audio, film); loan rules	Item already lent; overdue fines

Prompt	What usually varies	A failure case to handle
Card game (deck and hands)	Game rules	Empty deck; invalid move
Chat room	Message types; notification channels	User not in room; message too long
Food delivery order	Order states; delivery options	Restaurant closes mid-order

> **Tip:** The order and elevator prompts contain hidden state machines, like the launch sequence in Chapter 23. Writing the allowed transitions as a table is a quick way to show rigor.

Key Takeaways

- Clarify use cases first, then turn nouns into classes and verbs into methods.
- Give each class one responsibility, and let objects protect their own state.
- Put what varies behind an interface; new behavior should mean new classes, not edited ones.
- Prefer composition: a rider has a plan.
- Write and walk through working code for the core, including failures, and be ready to extend it.

CHAPTER 25

System Design Fundamentals

In a system design interview, you're asked to design a large service, such as a link shortener, a chat system or a news feed, in about 45 minutes, on a whiteboard or in a drawing tool. Nobody expects a complete design. The interviewer is evaluating how you handle ambiguity, how you reason about scale, whether you know the standard building blocks and their trade-offs, and whether you can go deep on one part when asked. It's usually part of the loop for mid-level roles and above, and often the round that decides the level of the offer.

This chapter gives you a framework, the building blocks, the numbers you need for quick estimates, and two worked designs. It's an introduction: system design is a large subject, and the "Further Reading" list at the end points to good next steps.

A Framework for 45 Minutes

Step	Time	What you do
1. Requirements	5 min	Functional: what the system does. Non-functional: scale, latency, availability, consistency. Agree on what's out of scope.
2. Estimates	5 min	Requests per second, storage, bandwidth: rough orders of magnitude.

Step	Time	What you do
3. API and data model	5 min	The main endpoints, and the main tables or records.
4. High-level design	10 min	Boxes and arrows: clients, load balancers, services, databases, caches, queues. Walk one request through it.
5. Deep dives	15 min	The hardest parts, often chosen by the interviewer: scaling the database, caching, consistency, failures.
6. Wrap-up	5 min	Bottlenecks, monitoring, what you would do next.

> **Tip:** Drive the conversation, but check in: "I'd like to go deeper on storage next. Is that the most useful area, or would you prefer something else?" Interviewers often have a specific deep dive in mind.

The Building Blocks

Component	What it does	Trade-offs to mention
Load balancer	Spreads requests over many identical servers	Needs health checks; sticky sessions complicate scaling
Stateless service	Handles requests without storing data between them	Easy to scale out; state must live elsewhere
Relational database (SQL)	Tables, transactions, joins, strong consistency	Harder to scale writes beyond one machine
Key-value / document store (NoSQL)	Fast lookups by key; scales across many machines	Fewer joins and transactions; often eventually consistent
Cache	Keeps hot data in memory, e.g. Redis or Memcached	Stale data; invalidation is hard; LRU eviction (Chapter 22)
Replication	Copies of the data on several machines	Read scaling and fault tolerance; replicas may lag behind
Sharding (partitioning)	Splits data across machines by key	Hot keys; resharding; cross-shard queries are expensive
Message queue / log	Decouples producers from consumers, e.g. Kafka	Absorbs bursts; adds delay; consumers must handle duplicates
Object storage	Stores large files cheaply, e.g. images and video	High latency per request; pair with a CDN

Component	What it does	Trade-offs to mention
CDN	Serves static content from servers near the user	Great for static data; cache invalidation again

Two theoretical ideas come up repeatedly:

- **The CAP theorem**: when a network partition splits the system, a distributed store must choose between consistency (every read sees the latest write) and availability (every request gets an answer). In practice, the question is which parts of *your* system need which.
- **Consistency models**: *strong* consistency (reads always see the latest write) versus *eventual* consistency (replicas agree after a short delay). A bank balance needs the first; a "likes" counter is fine with the second.

Numbers for Quick Estimates

You don't need precise figures, only the right order of magnitude.

Fact	Value
Seconds in a day	86,400, roughly 100,000 for estimates
1 million requests per day	about 12 per second
1 billion requests per day	about 12,000 per second
Peak traffic	often 2 to 10 times the average; state your assumption
One character of ASCII text	1 byte
A short database row	a few hundred bytes
Reading from memory vs. a network round trip in a data center vs. a disk seek	roughly 100 nanoseconds vs. hundreds of microseconds vs. milliseconds

> **Warning:** Latency figures depend on hardware and change over time. Use them to compare options ("a cache hit avoids a network call to the database"), not as exact predictions.

Worked Design 1: A Link Shortener

Requirements. Users submit a long URL and get a short link such as `sho.example/aZ3kQ9x`. Visiting the short link redirects to the long URL. Optional: custom aliases and expiry dates. Non-functional: redirects must be fast and highly available; creating links can be slightly slower.

Estimates. Assume 100 million new links per month and 100 redirects per link created.

- Writes: 100 million / (30 × 86,400 seconds) ≈ 40 per second on average.
- Reads: 100 times that, about 4,000 per second on average, and perhaps 20,000 at peak.
- Storage: about 500 bytes per link × 1.2 billion links per year ≈ 600 GB per year; 3 TB over five years.

The system is **read-heavy**, which shapes everything that follows.

Short keys. Seven characters from the 62 letters and digits give $62^7 \approx 3.5$ trillion possible keys, enough for thousands of years at this rate. Two common ways to generate them: encode a unique, increasing ID in base 62 (simple and collision-free, but predictable), or hash the URL with a random component and retry on the rare collision (unpredictable, needs a uniqueness check).

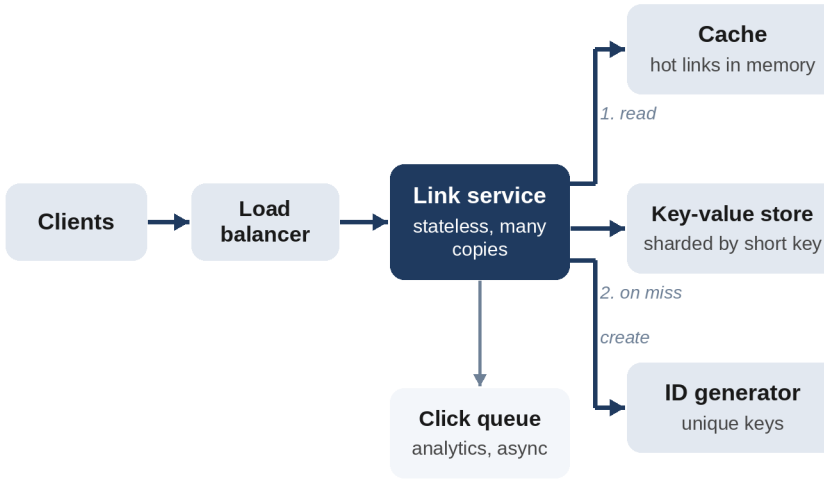
API.

```
POST /links    body {"url": "https://...", "alias": "optional"}
               returns {"short": "aZ3kQ9x"}
GET  /{short}  redirects (301 or 302) to the long URL
```

High-level design.

Deep dives.

- **Storage:** the data is a simple key-value mapping with no joins, so a distributed key-value store fits well, sharded by the short key. A relational database also works at this size, with read replicas.
- **Caching:** a small fraction of links receive most visits. An in-memory cache in front of the store, with LRU eviction, serves most redirects without touching the database.



A link shortener: a load balancer in front of stateless services; redirects read from a cache first and fall back to a key-value store; link creation writes to the store via an ID generator.

- **Redirect code:** a *301* (permanent) redirect lets browsers cache it, reducing load, but hides later visits from analytics; a *302* (temporary) redirect sends every visit through the service. Which one depends on whether click statistics are a requirement: a good question to ask.
- **Analytics:** don't slow down redirects by writing statistics synchronously. Publish a click event to a queue, and let a separate consumer aggregate counts.
- **Abuse:** rate-limit link creation per user (the token bucket from Chapter 23) and check URLs against a list of known malicious sites.

Worked Design 2: A Telemetry Ingestion Pipeline

Requirements. A fleet of 10,000 vehicles (delivery robots, satellites or cars) each send a 1 KB status message every second. Operators need a live dashboard of the latest status of each vehicle, alerts within seconds when a reading is abnormal, and the ability to query history for investigations.

Estimates. 10,000 messages per second; 10 MB per second; about 864 GB per day before compression. This system is **write-heavy**, the opposite of the link shortener.

High-level design.

1. **Ingestion gateways** behind a load balancer accept messages, authenticate the vehicle, validate the format and publish to a partitioned **log** (such as Kafka), keyed by vehicle ID, so each vehicle's messages stay in order.
2. **Consumers** read the log independently:

- a *latest-state* consumer updates a key-value store holding each vehicle's last status, which the dashboard reads;
- an *alerting* consumer runs rules like the spike detector from Chapter 23 and sends alerts;
- an *archive* consumer writes batches to cheap object storage in a columnar format, partitioned by date, for historical queries.

Deep dives.

- **Bursts and failures:** the log absorbs bursts and lets a consumer catch up after a failure, because messages stay in the log for a retention period.
- **Duplicates:** networks retry, so the same message can arrive twice. Include a sequence number per vehicle and make consumers *idempotent*: processing a message twice has the same effect as once.
- **Out-of-order data:** a vehicle that was offline sends old messages late. The latest-state consumer should ignore messages older than the stored one.
- **Hot partitions:** if one vehicle sends far more data, its partition becomes a bottleneck; monitor partition lag.

> **Interviewer's View:** Notice how the estimates drove the design: the read-heavy system centered on caching, the write-heavy one on a log and independent consumers. Explicitly linking your numbers to your choices is what separates strong answers from a list of buzzwords.

Key Takeaways

- Follow a framework: requirements, estimates, API, high-level design, deep dives, wrap-up.
- Estimates tell you whether a system is read-heavy or write-heavy, and that drives the design.
- Know the building blocks and one trade-off for each.
- Caches, queues and replication solve problems and create new ones (staleness, duplicates, lag); say so.
- Ask which deep dive the interviewer wants, and go deep there.

Further Reading

- Martin Kleppmann, *Designing Data-Intensive Applications* (O'Reilly): the standard reference on storage, replication, partitioning and streams.
- Alex Xu, *System Design Interview: An Insider's Guide* (volumes 1 and 2): many worked interview designs.
- The engineering blogs of large technology companies, which describe real systems and the trade-offs behind them.

CHAPTER 26

Behavioral Interviews

Technical candidates often treat behavioral questions as small talk before the real interview. That's a costly mistake. At most large companies, behavioral interviews are scored with the same care as coding rounds, and a weak one can sink an otherwise strong loop. Amazon is the clearest example: its own guides say behavioral questions are based on its Leadership Principles, and that for software roles roughly half of the interview process covers them. Other companies use different names, "Googleyness and leadership" in candidate reports about Google, for example, but they ask very similar questions.

The good news: behavioral interviews are the most *preparable* part of the process. The questions are predictable, and the stories come from your own experience.

What Interviewers Are Assessing

- **Ownership:** do you take responsibility for outcomes, including problems you didn't cause?
- **Impact:** did your work make a measurable difference?
- **Collaboration and conflict:** can you disagree constructively and work with difficult people?
- **Learning:** do you reflect on failures and improve?
- **Judgment under ambiguity:** how do you decide with incomplete information?

- **Leadership,** at any level: did you influence others, mentor, or raise the bar?
- **Communication:** can you tell a clear, structured story?

The STAR Method

Amazon's interview guides recommend structuring answers with **STAR**, and it works everywhere:

Part	What to say	Share of the answer
Situation	The context: team, project, what was at stake	about 15%
Task	Your specific responsibility or goal	about 10%
Action	What *you* did, step by step, and why	about 50%
Result	The outcome, with numbers if possible, and what you learned	about 25%

The most common failure is spending two minutes on the situation and twenty seconds on the actions. The actions are what's being scored.

> **Tip:** Say "I", not "we", when describing your actions. Amazon's guidance makes this point explicitly: interviewers need to know what *you* did. Credit the team in the situation and result, and be precise about your part in the action.

An Example Answer

Question: "Tell me about a time you disagreed with a technical decision."

Situation: Our team of five was building the checkout service for a food delivery app, with a launch date six weeks away. Our tech lead proposed storing orders in a new document database the team hadn't used before.

Task: I owned the order data model, so I was responsible for the storage decision working in production.

Action: I was worried about two things: we needed transactions across orders and payments, and nobody on call knew how to operate the new database. Instead of arguing in the meeting, I asked for two days to compare options. I wrote a one-page comparison and built a small prototype of the payment flow on both databases. The prototype showed that the new database would need a custom retry mechanism to keep orders and payments consistent. I shared the results with the lead first, privately, and then we presented them together to the team.

Result: We kept our existing relational database for orders and used the new one only for the menu catalog, where it fitted well. We launched on time, with no data-consistency incidents in the first six months. I learned that a quick prototype settles a disagreement faster than a debate, and since then I've suggested time-boxed prototypes whenever the team is split.

Why it works: the disagreement is about substance, not personality; the candidate gathers data instead of escalating; the relationship with the lead is respected; the result is concrete; and there's a lesson.

Build a Story Bank

Prepare **eight to ten stories** from the last few years, and map each to several themes. One good story can answer three or four different questions.

Theme	Typical questions
Conflict	"Tell me about a disagreement with a teammate or manager."
Failure	"Tell me about a time you failed." "What's a mistake you made?"
Ownership	"Tell me about a time you went beyond your role."
Impact	"What's the project you're proudest of?"
Ambiguity	"Tell me about a decision you made without all the information."
Deadline pressure	"Tell me about a time you had to cut scope."
Influence	"How did you convince others to adopt your idea?"
Learning	"Tell me about something you learned quickly."
Customer focus	"Tell me about a time you improved things for users."

Theme	Typical questions
Mentoring or leadership	"Tell me about helping someone grow."

For each story, write bullet points, not a script: the situation in one line, your three or four key actions, the result with a number, and the lesson. Practice telling each one aloud in about two minutes.

> **Common Mistake:** Choosing a "failure" that is really a disguised success ("I work too hard"). Interviewers see through it. Choose a real failure with real consequences, show what you did to recover, and, most important, what you changed afterward.

Questions About You and the Company

Expect these at every company, and prepare them as carefully as the stories:

- **"Tell me about yourself."** A 60- to 90-second summary: what you do now, one or two highlights, and why you're interested in this role. It isn't your life story.
- **"Why this company?"** Be specific: a product you use, a technical problem that interests you, something from their engineering blog. Generic praise ("it's a great company") is a weak answer.
- **"Do you have questions for me?"** Always have two or three. Apple's interview tips suggest asking about the team and how success is measured. Good examples: "What does a successful first six months look like in this role?" and "What's the hardest technical problem the team is working on?"

Behavioral Questions in Technical Rounds

Behavior is assessed in every round, not only in behavioral interviews. In a coding round, interviewers notice whether you take hints well, stay calm when stuck, and admit a bug instead of hiding it. In system design, they notice whether you listen. Treat every round as a conversation with a future colleague.

Key Takeaways

- Behavioral rounds are scored seriously; at some companies they're half the process.
- Use STAR, and spend most of the time on your own actions.
- Build a bank of eight to ten stories that cover the common themes, with numbers and lessons.
- Say "I" for your actions; give credit to the team elsewhere.
- Prepare "tell me about yourself", "why this company" and your own questions.

Sources

- Amazon: "Your complete guide to the Amazon interview process" and related interview guides on aboutamazon.com and amazon.jobs (STAR method, Leadership Principles, use "I").
- Apple careers: interview tips (open-ended questions about past performance; questions about the team).

CHAPTER 27

AI-Assisted Rounds, Take-Homes and Online Assessments

Not every coding test is a live conversation at a whiteboard. Many candidates meet an **online assessment** before talking to anyone; some roles use a **take-home project**; and a growing number of companies run **AI-assisted rounds**, where using an AI coding assistant is part of the test. Each format rewards slightly different habits. This chapter covers all three.

Online Assessments

An online assessment (OA) is a timed test on a coding platform, usually taken alone at home: typically two to four problems in 60 to 120 minutes, graded automatically against hidden test cases. They are common for internships, new graduate roles and high-volume hiring, where they decide who gets a live interview.

How to do well:

- **Read every problem first**, then solve the easiest one. Partial credit usually depends on the number of hidden tests passed, so a fully working easy problem beats a half-finished hard one.

- **Check the limits** in the statement. An input of up to 10^5 items rules out $O(n^2)$; up to 20 items invites backtracking. Chapter 3 has a table of what fits.
- **Write a brute force first** if the optimal solution isn't clear; it may pass most tests, and it's useful to check the faster version.
- **Test edge cases yourself**: the empty input, one element, all equal, the maximum size. Hidden tests always include them.
- **Know your language's input handling.** Some platforms give you a function to complete; others make you read standard input. Practice both.
- **Practice in the same conditions**: a timer, no autocomplete, and no outside help unless the instructions allow it.

> **Warning:** Follow the rules of each test exactly. If an assessment says no outside help, using an AI assistant or searching for the problem is cheating. Companies use proctoring, plagiarism checks and follow-up interviews that ask you to explain your submitted code. Being caught ends the process, and a solution you can't explain is worthless in the next round anyway.

Take-Home Projects

A take-home asks you to build something small, such as a command-line tool, an API endpoint or a data-processing script, over a few hours or days. It's closer to real work than any other format, and it's graded like real work.

What reviewers look at, roughly in order:

1. **Does it work?** Can they run it from your README in two minutes?
2. **Is it tested?** A focused set of tests is a strong signal; none is a red flag.
3. **Is it readable?** Clear structure, names and small functions, as in Chapter 23.
4. **Did you make sensible trade-offs, and explain them?** A short "Design notes" section in the README: what you assumed, what you left out, and what you would do with more time.

> **Tip:** Respect the time limit. If the instructions suggest four hours, a 20-hour masterpiece isn't what's being asked for, and it can raise questions about how you prioritize. Do the core well and list the extensions in the README.

Take-homes are usually followed by a review conversation. Expect "walk me through your design", "why did you choose this library?" and "how would you add feature X?". Re-read your own code before that call.

AI-Assisted Rounds

Some companies now include an approved AI assistant in technical interviews. Meta's hiring process page says that select roles include an authorized AI assistant within the coding environment during technical interviews, and that candidates can practice in the same tool beforehand. Candidates who have taken such rounds describe a longer session, around an hour, built around a small multi-file codebase and a task with several stages, such as fixing a bug, adding a feature and extending it; they report being judged on problem solving, code quality, how they verify the assistant's output, and communication. Those details come from candidates' reports rather than official descriptions, and formats differ between companies and roles, so ask your recruiter exactly what to expect.

What changes when an assistant is allowed:

- **Typing speed matters less; judgment matters more.** The assistant writes code quickly. Your job is to decide what to ask for, to notice when its answer is wrong, and to keep the overall design coherent.
- **Reading code becomes central.** You must understand an unfamiliar codebase quickly, and review generated code as carefully as a colleague's pull request.
- **Verification is visible.** Interviewers watch whether you run the code, write tests, and check edge cases, or simply accept what the assistant produced.

A Good Workflow

1. **Understand before you prompt.** Read the task and the relevant files. Say aloud what you think needs to change.
2. **Ask for small, specific pieces**: "write a function that returns the k most frequent tags, ties broken alphabetically", not "solve the problem".
3. **Review every line.** Check edge cases, complexity, and whether the code fits the existing conventions.
4. **Test.** Run the existing tests, and ask the assistant to write new ones, then check that those tests actually test something.
5. **Explain.** Narrate your decisions, including when you reject a suggestion and why.

Catching an Assistant's Mistake

Here is the kind of moment these rounds are designed to create. A candidate asks for a prefix check on the trie from Chapter 20, and the assistant suggests:

```
def has_prefix(self, prefix):
    return self._walk(prefix) is not None
```

It looks right, and it passes the obvious tests. A strong candidate tries the edge cases out loud: "What about the empty prefix on an empty trie? `_walk("")` returns the root, which exists, so this says some word starts with the empty string, even though there are no words at all." The fix, `return bool(self._walk(prefix))`, works because the root of an empty trie is an empty dictionary.

This is exactly the bug the book's own random tests found in an early version of Problem 20.1. The lesson for AI-assisted rounds: an assistant makes the same kinds of mistakes people do, especially on edge cases, and your value is in catching them.

> **Interviewer's View:** In an AI-assisted round, "I'd like to check that before we move on" is one of the strongest things you can say. Accepting large blocks of generated code without reading them is one of the weakest.

How to Practice

- Use an AI coding assistant on the problems in this book, but only *after* solving each problem yourself, then compare and critique its solution.
- Practice reading unfamiliar code: open a small open-source project, pick an issue, and find the relevant files within 15 minutes.
- Use the practice environment the company provides, if there is one, so that the tool itself holds no surprises on the day.

Key Takeaways

- Online assessments: solve the easy problems fully, respect the limits, and test edge cases yourself.
- Take-homes: working, tested, readable code with a clear README, within the time suggested.
- AI-assisted rounds reward judgment, code reading and verification, not typing.
- Follow every format's rules on outside help exactly.
- In every format, you must be able to explain every line you submit.

Sources

- Meta careers: hiring process page and software engineering interview preparation pages (authorized AI assistant for select roles; practice environment).
- Descriptions of AI-assisted rounds beyond Meta's own pages are based on candidates' reports published in 2025 and 2026 and may change.

PART IV

Practice

CHAPTER 28

Four Full Mock Interviews

This chapter replays four complete 45-minute coding interviews, as conversations between a candidate and an interviewer. They are written to show the parts of an interview that solutions alone can't: clarifying questions, thinking aloud, getting stuck, taking hints, finding a bug, and answering follow-ups. Each one ends with the interviewer's notes, the kind of written feedback that goes to a hiring committee or a hiring manager.

The interviews are fictional, and the problems are original to this book, but the structure matches the six-step method of Chapter 2.

How to use this chapter: before reading each interview, read only the problem statement and try it yourself for 30 minutes, out loud, with a timer. Then read the conversation and compare: not just your code, but your clarifying questions, your explanation and your testing.

Mock 1: Bus Transfers (Medium)

The problem. A city has bus routes; `routes[i]` lists the stops that bus i visits, in a loop, in both directions. You may change buses at any stop they share. What is the fewest number of buses you must ride to get from stop `src` to stop `dst`? Return 0 if they're the same stop, and -1 if it's impossible.

Interviewer: Here's the problem. Take a moment to read it, and tell me how you'd like to start.

Candidate: Let me make sure I understand. We're minimizing the number of buses, not the number of stops traveled. So riding one bus through ten stops counts as one. Is that right?

Interviewer: Yes.

Candidate: And a bus can be ridden from any of its stops to any other of its stops, because it loops. Can a stop appear twice in a route, and roughly how large are the inputs?

Interviewer: No repeated stops within a route. Up to about 500 routes, and up to 10^5 stops in total across all routes.

Candidate: OK. Let me write an example. Routes `[1, 4, 9]`, `[9, 5, 6]` and `[6, 2, 8]`, from stop 1 to stop 8. I take the first bus from 1 to 9, change to the second bus to 6, then the third to 8. Three buses. And if `src` equals `dst`, the answer is 0 without riding anything.

Interviewer: Good. How would you approach it?

Candidate: "Fewest" with each step costing the same, one bus, suggests BFS. My first instinct is a BFS over stops, where neighbors are stops reachable on any bus through the current stop... but then each level is one bus ride, so the BFS level is exactly the number of buses. That works.

Interviewer: What's the cost?

Candidate: If I'm careless, a lot. A stop with many buses would rescan each bus's entire route every time any of its stops is dequeued. The trick is that once I've ridden a bus, all its stops are reachable at that level, so I never need to expand that bus again. I'll keep a set of used buses. Then each bus is expanded once and each stop is queued once, so it's linear in the total size of the routes.

Interviewer: Sounds good. Go ahead.

The candidate writes the code, talking through each part: the map from stop to buses, the queue of (stop, rides) pairs, and the two "seen" sets.

```
def bus_transfers(routes, src, dst):
    if src == dst:
        return 0
    buses_at = defaultdict(list)  # stop -> buses that visit it
    for bus, stops in enumerate(routes):
```

```
        for stop in stops:
            buses_at[stop].append(bus)
    queue = deque([(src, 0)])
    seen_stops, used_buses = {src}, set()
    while queue:
        stop, rides = queue.popleft()
        for bus in buses_at[stop]:
            if bus in used_buses:
                continue  # its stops were all queued already
            used_buses.add(bus)
            for nxt in routes[bus]:
                if nxt == dst:
                    return rides + 1
                if nxt not in seen_stops:
                    seen_stops.add(nxt)
                    queue.append((nxt, rides + 1))
    return -1
```

Candidate: Let me trace my example. The queue starts with (1, 0). Stop 1 has bus 0, unused: I mark it and scan stops 1, 4 and 9. None is 8, so 4 and 9 are queued with one ride. From 4, bus 0 is used, so nothing. From 9, bus 1 is new: stops 9, 5, 6 are scanned and 5 and 6 queued with two rides. From 5, nothing new; from 6, bus 2: stop 8 is the destination, return 2 + 1 = 3. Correct.

Interviewer: What if `dst` isn't on any route?

Candidate: Then the queue empties and we return -1. And `src == dst` is handled first, even if the stop isn't on any route. One more edge case: `src` on no route at all. `buses_at[src]` is an empty list, the loop does nothing, and we return -1. Good.

```
>>> bus_transfers([[1, 4, 9], [9, 5, 6], [6, 2, 8]], 1, 8)
3
>>> bus_transfers([[2, 3], [4, 5]], 2, 5)
-1
>>> bus_transfers([[1, 2]], 3, 3)
0
```

Interviewer: Nice. Last question: is there another way to model the graph?

Candidate: Make the buses the nodes, and connect two buses if they share a stop. Then BFS from all the buses through `src`. It's conceptually clean, but building the edges by comparing every pair of buses is $O(B^2 \times \text{route length})$ in the worst case, so I prefer the stop-based version with the used-bus set.

Interviewer's notes. *Strong hire signal. Clarified the objective (buses, not stops) and the input size before designing. Identified BFS immediately and, importantly, anticipated the cost of re-expanding buses before coding. Clean*

code with clear names. Traced an example accurately and covered edge cases without prompting. Discussed an alternative model and its trade-off.

Mock 2: My Calendar (Medium)

The problem. Implement a calendar class with a method `book(start, end)` that adds the half-open interval `[start, end)` and returns True, unless it would overlap an existing booking, in which case it returns False and adds nothing.

Candidate: Half-open, so a booking ending at 12 and one starting at 12 don't overlap. Are start and end integers, and is `start < end` guaranteed?

Interviewer: Yes to both.

Candidate: The simplest solution keeps a list of bookings and checks each one: two intervals overlap when each starts before the other ends. That's O(n) per call. Is that acceptable, or should I aim for something faster?

Interviewer: Start there, then improve it.

Candidate: For the brute force, the overlap test is `s < be and bs < e`, and a booking is OK if it overlaps none. I'll write it quickly, as a reference for testing:

```
def calendar_session_brute(requests):
    booked, out = [], []
    for s, e in requests:
        ok = all(e <= bs or be <= s for bs, be in booked)
        if ok:
            booked.append((s, e))
        out.append(ok)
    return out
```

Candidate: To improve it, I'll keep the bookings sorted by start. Since they never overlap, sorting by start also sorts them by end. A new booking can only conflict with its two neighbors in sorted order: the one before it, which must end by our start, and the one after it, which must start at or after our end. Binary search finds the position.

Interviewer: Show me.

```
class Calendar:
    def __init__(self):
        self.starts, self.ends = [], []  # sorted, non-overlapping

    def book(self, start, end):
        i = bisect_right(self.starts, start)  # bookings before i start <=
        if i > 0 and self.ends[i - 1] > start:
            return False  # the previous booking runs into ours
        if i < len(self.starts) and self.starts[i] < end:
            return False  # ours runs into the next booking
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True
```

Candidate: `bisect_right` gives the index of the first booking starting after our start. So `i - 1` is the last booking that starts at or before us. It must end by our start; and the booking at `i`, if any, must start at or after our end. Let me test: book (9, 12) succeeds. (11, 13): `i` is 1, the previous booking ends at 12, which is after 11, so it's rejected. (12, 14): the previous ends at 12, not after 12, so no conflict; nothing after; accepted. Then (8, 9): `i` is 0, nothing before, the next starts at 9, which is not before 9; accepted.

```
>>> calendar_session([(9, 12), (11, 13), (12, 14), (8, 9)])
[True, False, True, True]
```

Interviewer: What's the complexity now?

Candidate: The binary search is O(log n). But I should be honest: `list.insert` shifts elements, so it's O(n) in the worst case. It's a fast memory move, so in practice it's quick for thousands of bookings. For a guaranteed O(log n), I'd use a balanced binary search tree, for example `SortedList` from the third-party `sortedcontainers` package, or a `TreeMap` in Java.

Interviewer: Good. Follow-up: now allow double booking, but never triple booking.

Candidate: Then I'd keep two structures: all bookings, and the overlaps between them. A new booking is rejected if it overlaps any existing *overlap*; otherwise, I add its overlaps with existing bookings to the overlap list. That's O(n) per call. For many bookings, a sweep over +1/-1 events, as in the Rooms Needed problem, generalizes to "at most k at once".

Interviewer's notes. *Hire. Asked about the interval convention up front, which is where most bugs in this problem come from. Wrote a brute force for*

testing, then a correct binary-search solution, and traced boundary cases carefully. Was candid that list insertion is O(n), and knew the right structure for a guaranteed bound. Follow-up answer was reasonable, slightly slow to start.

Mock 3: Snapshot Array (Medium)

The problem. Design an array of n zeros with three operations: `set(i, value)`; `snap()`, which takes a snapshot and returns its ID (0, then 1, then 2, and so on); and `get(i, snap_id)`, which returns the value at index i when that snapshot was taken.

Candidate: The obvious solution copies the whole array on every snap, so `get` is a lookup. What sizes should I expect?

Interviewer: n up to 50,000, and up to 50,000 calls in total.

Candidate: Then copying is too expensive: 50,000 snaps of 50,000 elements is 2.5 billion values. Memory must grow with the number of `set` calls, not with n times the number of snaps.

Interviewer: Agreed. What would you store instead?

Candidate: For each index, its *history*: a list of (snap ID, value) pairs, recording a change only when `set` is called. A `snap` then just increments a counter. To answer `get(i, s)`, I need the last change to index i made in a snapshot period up to s. The history is sorted by snap ID, because snap IDs only increase, so binary search finds it.

Interviewer: What does "made in a snapshot period" mean exactly?

Candidate: Good point, let me be precise. Sets between snap k - 1 and snap k belong to snapshot k, because `snap()` returns k and captures them. So I tag each set with the current `snap_id`, the ID the next snapshot will get. And if index i is set twice before the next snap, only the last value matters, so I overwrite the last entry instead of appending. To avoid special cases, every history starts with (-1, 0): the initial zero.

```
class SnapshotArray:
    def __init__(self, n):
        self.history = [[(-1, 0)] for _ in range(n)]  # (snap, value)
        self.snap_id = 0

    def set(self, i, value):
        h = self.history[i]
        if h[-1][0] == self.snap_id:
            h[-1] = (self.snap_id, value)  # overwrite within this snap
        else:
            h.append((self.snap_id, value))

    def snap(self):
        self.snap_id += 1
        return self.snap_id - 1

    def get(self, i, snap_id):
        h = self.history[i]
        k = bisect_right(h, (snap_id, float("inf"))) - 1
        return h[k][1]
```

Candidate: For the search, I want the last pair whose snap ID is at most `snap_id`. Searching for `(snap_id, infinity)` with `bisect_right` puts me just after all pairs with that snap ID, whatever their values, and stepping back one gives the answer.

Interviewer: Let's trace an example.

Candidate: Two elements. `set(1, 4)`: history of 1 becomes [(-1, 0), (0, 4)]. `snap()` returns 0. `set(1, 7)`: appended as (1, 7). `snap()` returns 1. `get(1, 0)`: the last pair with ID at most 0 is (0, 4), so 4. `get(1, 1)`: (1, 7), so 7. `get(0, 1)`: index 0 has only (-1, 0), so 0.

```
>>> snapshot_session(
...     2,
...     [('set', 1, 4),
...      ('snap',),
...      ('set', 1, 7),
...      ('snap',),
...      ('get', 1, 0),
...      ('get', 1, 1),
...      ('get', 0, 1)],
... )
[0, 1, 4, 7, 0]
```

Interviewer: How did you test this while preparing? Assume you had more time.

Candidate: I'd keep the slow copy-on-snap version as a reference, because it's obviously correct, and compare the two on thousands of random

sequences of operations. That would catch off-by-one errors in the snap IDs, which are the risky part here.

Interviewer's notes. *Strong hire. Rejected the naive copy using the stated limits, with a quick estimate. Designed per-index history with binary search independently. When asked to be precise, worked out the snap-ID semantics carefully instead of guessing, and handled repeated sets within one snapshot. The (snap_id, infinity) search key was a neat detail. Testing strategy, a slow reference plus random comparisons, was excellent.*

Mock 4: Easiest Hike (Medium-Hard)

The problem. A grid gives the heights of a terrain. A hike goes from the top-left cell to the bottom-right cell, moving up, down, left or right. The effort of a hike is the *largest* height difference between two consecutive cells along it. Find the smallest possible effort.

Candidate: So the cost of a path isn't a sum, it's the maximum step. And we can move in all four directions. Can heights be negative, and how big is the grid?

Interviewer: Heights are non-negative integers up to a million; the grid is up to 100 by 100.

Candidate: My first idea is dynamic programming, like counting grid paths: the best effort to reach each cell from the cell above or the cell to the left.

Interviewer: Does that handle every path?

Candidate: Hmm. No: it only considers paths that move right and down. Since we can move up and left, the best path might snake around. For example, if there's a wall of high cells in the middle with a gap at the top, the best route might go down, across and back up. Let me write one:

```
1 9 1 1 1
1 9 1 9 1
1 1 1 9 1
```

Candidate: The path down the left column, along the bottom, up the middle column, across the top and down the right side uses only 1s: effort 0.

Right-and-down paths must cross a 9 somewhere, effort 8. So DP over right and down moves is wrong.

Interviewer: Good catch. So what instead?

Candidate: It's a shortest path where the "length" of a path is its largest edge instead of the sum. Dijkstra's algorithm still works, because extending a path can never decrease its maximum, just as extending a path can never decrease its sum with non-negative weights. So I'll run Dijkstra with `max(effort, step)` in place of `distance + weight`.

```
def easiest_hike(heights):
    rows, cols = len(heights), len(heights[0])
    best = [[float("inf")] * cols for _ in range(rows)]
    best[0][0] = 0
    heap = [(0, 0, 0)]  # (effort so far, row, col)
    while heap:
        effort, r, c = heapq.heappop(heap)
        if (r, c) == (rows - 1, cols - 1):
            return effort
        if effort > best[r][c]:
            continue  # stale entry
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                step = abs(heights[nr][nc] - heights[r][c])
                e = max(effort, step)  # effort is the worst step so far
                if e < best[nr][nc]:
                    best[nr][nc] = e
                    heapq.heappush(heap, (e, nr, nc))
    return 0
```

Candidate: The heap pops the cell with the smallest effort so far. When the bottom-right cell is popped, its effort is final. The "stale entry" check skips cells that were already reached more cheaply. With r × c cells and up to four edges each, it's O(rc log(rc)).

```
>>> easiest_hike([[4, 6, 9], [5, 12, 8], [7, 6, 10]])
3
>>> easiest_hike([[1, 10, 1], [1, 10, 1], [1, 1, 1]])
0
>>> easiest_hike([[1, 9, 1, 1, 1], [1, 9, 1, 9, 1], [1, 1, 1, 9, 1]])
0
>>> easiest_hike([[7]])
0
```

Interviewer: Any other approach?

Candidate: Binary search on the answer. For a given limit, I can check with BFS whether the destination is reachable using only steps no bigger than that

limit. If a limit works, any larger one works, so the smallest working limit can be found by binary search over 0 to the height range. That's O(rc × log(height range)): about 20 BFS runs for heights up to a million.

```
def reachable(heights, limit):
    rows, cols = len(heights), len(heights[0])
    seen, queue = {(0, 0)}, deque([(0, 0)])
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if (
                0 <= nr < rows
                and 0 <= nc < cols
                and (nr, nc) not in seen
                and abs(heights[nr][nc] - heights[r][c]) <= limit
            ):
                seen.add((nr, nc))
                queue.append((nr, nc))
    return (rows - 1, cols - 1) in seen


def easiest_hike_binary(heights):
    """Binary search on the answer, checking each guess with BFS."""
    lo, hi = 0, max(map(max, heights)) - min(map(min, heights))
    while lo < hi:
        mid = (lo + hi) // 2
        if reachable(heights, mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Interviewer: Which would you choose?

Candidate: Dijkstra is a single pass and doesn't depend on the height range. The binary search is easier to get right under pressure, because the BFS is very standard, and it's a good cross-check: I'd test them against each other.

Interviewer's notes. *Hire, leaning strong. Started with an incorrect DP, but found the flaw when prompted and, impressively, built a concrete counterexample. Adapted Dijkstra correctly and justified why it works for a max-cost path. Offered binary search on the answer as an alternative with accurate complexity. Initial mistake is normal at this difficulty; how it was handled is a positive signal.*

What the Four Interviews Have in Common

- Every candidate **clarified** before designing: the objective, the interval convention, the input sizes.
- Every candidate **estimated** costs from the limits, and used the estimate to reject an approach.
- Mistakes happened, and the candidates **recovered** by reasoning aloud, not by guessing.
- Every candidate **traced an example** through the final code, and named edge cases.
- Every candidate had a view on **testing**: a brute force or slow reference to compare against.

You won't be perfect in an interview, and you don't need to be. Interviewers are looking for the habits above.

Key Takeaways

- Practice full interviews out loud, with a timer, not just problems.
- Clarify the objective and constraints before choosing an algorithm.
- Use the input limits to estimate, and say the estimate aloud.
- When an interviewer asks "does that handle every case?", look for a counterexample.
- Trace your code on an example and name the edge cases before saying you're done.

CHAPTER 29

Your Study Plan

Knowing what to study matters less than studying it consistently and in the right way. This chapter gives three plans, for two, six and twelve weeks, and the practice habits that make the hours count. Adjust them to your situation: the plans assume about one to two hours on weekdays and a longer session on the weekend.

How to Practice a Problem

Most candidates practice by reading solutions. It feels productive, but it builds recognition, not recall. In an interview you have to produce a solution, so practice producing them:

1. **Set a timer for 25 to 30 minutes.** Read the problem only, and cover the rest of the page.
2. **Talk aloud** through the six steps of Chapter 2, even when alone. Speaking feels strange at first; it's a skill, and it improves with practice.
3. **Write the code by hand or in a plain editor**, without autocomplete or running it, at least some of the time. Then run it against the examples.
4. **Compare with the book's solution.** Note what you missed: the idea, an edge case, the complexity, or a cleaner way to write it.
5. **Log the problem** in a simple table: date, pattern, solved or not, and the one thing you learned.

6. **Repeat it later.** Return to any problem you couldn't solve after three days, and again after two weeks. Spaced repetition is what moves a pattern from "I've seen it" to "I can do it".

> **Tip:** If you're stuck for more than 10 minutes with no ideas, read only the "Think first" hint and continue. If you're still stuck after 25 minutes, read the idea, not the code, then write the code yourself.

The Two-Week Plan

For an interview that's close. Focus on the most common patterns and on interview technique.

Days	Focus
1-2	Chapters 2 to 4: the six-step method, Big-O, Python tools. Then Chapter 5 (arrays and hashing)
3	Chapters 6 and 7: two pointers, sliding window (first three problems of each)
4	Chapters 9 and 10: stacks, binary search (first three of each)
5	Chapters 11 and 12: linked lists, trees (first three of each)
6	Chapter 13 (heaps) and Chapter 16 (graphs), first three of each
7	Mock 1 from Chapter 28, timed; review weak areas
8	Chapter 18: dynamic programming, first four problems
9	Chapters 14 and 15: intervals, backtracking (first three of each)
10	Chapter 22: LRU cache; Chapter 23 if practical rounds are expected
11	Mock 2, timed; behavioral story bank (Chapter 26)
12	System design basics (Chapter 25) if your level requires it; otherwise re-solve logged failures
13	Mock 3 or 4, timed, ideally with a friend acting as interviewer
14	Light review of Appendix A, your log and your stories. Rest

The Six-Week Plan

The recommended plan for most candidates: every pattern, every problem, with review built in.

Week	Focus	Also
1	Part I; Chapters 5 to 7 (arrays, two pointers, sliding window)	Start your problem log
2	Chapters 8 to 11 (prefix sums, stacks, binary search, linked lists)	Re-solve week 1 failures
3	Chapters 12 to 15 (trees, heaps, intervals, backtracking)	Write your behavioral story bank
4	Chapters 16 to 19 (graphs, advanced graphs, both DP chapters)	Mock 1 and 2, timed
5	Chapters 20 to 23 (strings, bits, design, practical)	Chapter 24 (OOD); Chapter 25 if needed
6	Mocks 3 and 4; re-solve every logged failure	Chapters 26 and 27; practice stories aloud

The Twelve-Week Plan

For career changers, students preparing for their first interviews, or anyone returning after years away. It's the six-week plan at half the speed, with additional practice:

- **Weeks 1 to 2:** Part I slowly, plus any Python basics you need. Solve the Chapter 5 problems twice.
- **Weeks 3 to 10:** two pattern chapters per week. At the end of each week, pick two problems from earlier chapters at random and solve them again, timed.
- **Week 6 and week 10:** a mock interview with another person. Swap roles: being the interviewer teaches you what interviewers notice.
- **Weeks 11 to 12:** Part III, all four mocks, system design if your level requires it, and the story bank.

Practicing With Other People

Solo practice can't fully prepare you for being watched while you think. Ways to add an audience:

- **A study partner:** take turns as interviewer, using the mocks in Chapter 28 as a model. The interviewer reads the problem aloud, answers clarifying questions, gives hints only when needed, and writes notes like the ones at the end of each mock.
- **Record yourself** solving a problem aloud, and watch it back. It's uncomfortable and very effective.
- **Explain a solution** to someone less experienced. If you can't explain why it works, you don't fully own it yet.

The Week Before and the Day Of

- **Confirm the format** with your recruiter: the number of rounds, languages allowed, the coding environment, and whether AI assistance is permitted in any round.
- **Prepare your setup** for virtual interviews: a quiet room, a reliable connection, a charged laptop, and the coding tool tested in advance.
- **Don't cram new topics** in the last two days. Review your log, Appendix A, and your stories.
- **Sleep.** Problem solving degrades sharply when you're tired.
- **During the interview,** if you get stuck, say what you're thinking and what you're trying. Silence is the one thing an interviewer can't evaluate.
- **After each round,** let it go. You can't judge your own performance accurately, and the next round deserves your full attention.

If You Don't Get the Offer

Most successful candidates have been rejected before, often by the same companies that later hired them. Ask the recruiter for any feedback they can share; many companies allow you to reapply after a waiting period, often six months to a year. Use the time to work on the weakest area your log shows, and keep interviewing elsewhere: every interview is practice for the next one.

Key Takeaways

- Practice producing solutions, aloud and timed, not reading them.
- Keep a problem log and re-solve failures after a few days and again after two weeks.
- Choose the plan that matches your time: two, six or twelve weeks.
- Practice with other people, and take turns as interviewer.
- Confirm the format with your recruiter, prepare your setup, and rest before the day.

Appendix A: Pattern Cheat Sheet

Use this table to go from the wording of a problem to a likely pattern. The signals are hints, not rules: always check that the pattern's requirements hold (sorted input, non-negative values, small limits).

If the problem says or implies...	Try	Typical cost	Chapter
"Have I seen this before?", counts, pairs with a sum, grouping	Hash set or dictionary	O(n)	5
Sorted array, pair or triple with a target, palindrome, in-place	Two pointers from the ends or same direction	O(n)	6
Longest/shortest contiguous block with a property	Sliding window	O(n)	7
Many range-sum queries; subarrays summing to k	Prefix sums plus a dictionary	O(n)	8
Matching brackets, undo, "previous/next greater"	Stack; monotonic stack	O(n)	9
Sliding window maximum	Monotonic deque	O(n)	9
Sorted input; "smallest value such that..."	Binary search; binary search on the answer	O(log n)	10
Linked list cycle, middle, k-th from end	Fast and slow pointers	O(n)	11
Reverse or merge lists	Pointer rewiring with a dummy head	O(n)	11

If the problem says or implies...	Try	Typical cost	Chapter
Tree height, paths, validity	DFS recursion returning a value	O(n)	12
Tree levels	BFS with a queue, level by level	O(n)	12
Top k, k-th largest, merge k sorted	Heap of size k	O(n log k)	13
Running median	Two heaps	O(log n) per item	13
Overlapping intervals, meeting rooms	Sort by start (merge) or end (select); heap of ends	O(n log n)	14
"Can you reach...", "where to start"	Greedy with one running value	O(n)	14
All subsets, orders, combinations; small n	Backtracking	exponential	15
Grid regions, connected components	DFS or BFS flood fill	O(V + E)	16
Fewest steps, unweighted	BFS	O(V + E)	16
Spreading from several sources	Multi-source BFS	O(V + E)	16
Dependencies, ordering, cycle in directed graph	Topological sort	O(V + E)	17
"Already connected?", redundant edge	Union-find	nearly O(1) per operation	17
Cheapest path, non-negative weights	Dijkstra	O(E log V)	17
Cheapest path with at most k steps	Bellman-Ford, k rounds	O(k × E)	17
Connect everything at minimum cost	Minimum spanning tree	O(E log V)	17
Count ways, min/max over choices, overlapping subproblems	Dynamic programming	states × choices	18, 19
Two strings compared	2D DP over prefixes	O(m × n)	19
Choose items under a capacity	Knapsack DP	O(n × capacity)	19

If the problem says or implies...	Try	Typical cost	Chapter
Many words, prefixes	Trie	O(length) per word	20
Pattern search in text	KMP (or built-in find)	O(n + m)	20
Pairs cancel, single missing value, powers of two	Bit manipulation, XOR	O(n)	21
O(1) get/put with eviction	Hash map plus linked list	O(1)	22
Parse, validate, aggregate messy input	Regex or explicit checks, dictionaries	O(input)	23

Questions to Ask Yourself When Stuck

1. What would the brute force be, and which part of it repeats work?
2. Would sorting the input help?
3. Would a dictionary remove an inner loop?
4. Is there a monotonic property I can binary search on?
5. Is this a graph in disguise? What are the nodes and edges?
6. Can I define the answer for a smaller input and build up (DP)?
7. What do the input limits allow? (Chapter 3)

Templates to Know by Heart

- **Sliding window (variable):** expand right every step; while the window is invalid, shrink from the left; record the answer.
- **Binary search for the first "yes":** `lo, hi = low, high`; while `lo < hi`: `mid = (lo + hi) // 2`; if `ok(mid)`: `hi = mid`, else `lo = mid + 1`.
- **BFS:** queue with the start; mark visited when queued; pop, process, push unvisited neighbors.
- **Backtracking:** if complete, record a copy; for each choice, apply, recurse, undo.
- **DP:** state in words; recurrence; base cases; order (memoized recursion or a table).

Appendix B: Complexity Reference

Python Built-In Operations

Average-case costs for CPython. n is the number of items in the container, k the size of the argument or slice.

Operation	list	dict	set	deque
Index `x[i]`	O(1)			O(n)
Look up `x in c`	O(n)	O(1)	O(1)	O(n)
Get by key `d[k]`		O(1)		
Append at the end	O(1) amortized	O(1)	O(1)	O(1)
Pop from the end	O(1)			O(1)
Insert or pop at the front	O(n)			O(1)
Insert or delete in the middle	O(n)	O(1) by key	O(1)	O(n)
Slice `x[a:b]`	O(k)			
Copy	O(n)	O(n)	O(n)	O(n)
Sort `sorted(x)`	O(n log n)			
Min or max	O(n)	O(n)	O(n)	O(n)
Length `len(x)`	O(1)	O(1)	O(1)	O(1)

Other operations	Cost
`heapq.heappush`, `heapq.heappop`	O(log n)
`heapq.heapify`	O(n)
`heapq.nsmallest(k, x)`	O(n log k)
`bisect.bisect_left` on a sorted list	O(log n)
`bisect.insort`	O(n), because of the insertion
`Counter(x)`	O(n)
String concatenation `s + t`	O(len(s) + len(t))
`"".join(parts)`	O(total length)
`s in t` for strings	O(len(s) × len(t)) worst case, usually much faster
Set union `a | b`	O(len(a) + len(b))
Set intersection `a & b`	O(min(len(a), len(b))) on average

> **Warning:** Building a string with `s += ch` in a loop can be $O(n^2)$ in the worst case. Collect pieces in a list and `"".join` them.

Algorithms

Algorithm	Time	Space
Binary search	O(log n)	O(1)
Sorting (Timsort, merge sort, heap sort)	O(n log n)	O(n) for Timsort and merge sort
Quickselect (k-th element)	O(n) average, $O(n^2)$ worst	O(1)
Counting or bucket sort, keys up to K	O(n + K)	O(n + K)
BFS, DFS	O(V + E)	O(V)
Topological sort	O(V + E)	O(V)
Dijkstra with a binary heap	O(E log V)	O(V + E)
Bellman-Ford	O(V × E)	O(V)

Algorithm	Time	Space
Prim / Kruskal	O(E log V)	O(V + E)
Union-find with path compression and union by size	nearly O(1) per operation	O(V)
KMP string search	O(n + m)	O(m)
Sieve of Eratosthenes	O(n log log n)	O(n)
Fast power	O(log exponent)	O(1)

Input Size to Target Complexity

A rough guide for Python, assuming about a second of run time (Chapter 3):

n up to about	Acceptable complexity
10 to 12	O(n!)
20 to 25	$O(2^n)$
500	$O(n^3)$
5,000	$O(n^2)$
10^6	O(n log n) or O(n)
10^9 and above	O(log n) or O(1)

Growth at a Glance

n	$\log_2 n$	$n \log_2 n$	n^2	2^n
10	about 3	about 33	100	1,024
100	about 7	about 664	10,000	about 1.3×10^3
1,000	about 10	about 10,000	1,000,000	far too many
1,000,000	about 20	about 20,000,000	10^{12}	far too many

Appendix C: From Python to Java and C++

Most companies let you choose your interview language, and Python is the most popular choice. But some roles expect Java or C++, especially in systems, embedded and aerospace work, where candidates commonly report C++ being expected. The ideas in this book don't depend on the language; the tools do. This appendix maps each Python tool to its Java and C++ equivalents, and shows two solutions translated into both.

Containers

Python	Java	C++
`list`	`ArrayList<T>`	`std::vector<T>`
`dict`	`HashMap<K, V>`	`std::unordered_map<K, V>`
`set`	`HashSet<T>`	`std::unordered_set<T>`
Sorted dict / set (no built-in)	`TreeMap<K, V>`, `TreeSet<T>`	`std::map<K, V>`, `std::set<T>`
`collections.deque`	`ArrayDeque<T>`	`std::deque<T>`
Stack (a `list`)	`ArrayDeque<T>` with `push` / `pop`	`std::stack<T>` or `std::vector<T>`
`heapq` (min-heap)	`PriorityQueue<T>` (min-heap)	`std::priority_queue<T>` (**max**-heap by default)

Python	Java	C++
`collections.Counter`	`HashMap<K, Integer>` with `merge(k, 1, Integer::sum)`	`std::unordered_map<K, int>` with `++m[k]`
`defaultdict(list)`	`computeIfAbsent(k, x -> new ArrayList<>())`	`m[k].push_back(v)` (creates the entry)
Tuple `(a, b)`	a small `record` class, or `int[]`	`std::pair<A, B>` or `std::tuple`

Everyday Operations

Python	Java	C++
`len(x)`	`x.size()`, `arr.length`, `s.length()`	`x.size()`
`x in d`	`d.containsKey(x)`, `s.contains(x)`	`d.count(x)` or `d.find(x) != d.end()`
`sorted(x)` / `x.sort()`	`Collections.sort(x)`, `Arrays.sort(a)`	`std::sort(x.begin(), x.end())`
Sort with a key	`x.sort(Comparator.comparingInt(...))`	`std::sort(..., [](auto& a, auto& b) {...})`
`bisect_left(a, x)`	`Collections.binarySearch` (returns a negative value if absent)	`std::lower_bound(a.begin(), a.end(), x)`
`bisect_right(a, x)`	write it yourself	`std::upper_bound(...)`
`float("inf")`	`Integer.MAX_VALUE`, `Long.MAX_VALUE`	`INT_MAX`, `LLONG_MAX`, `std::numeric_limits`
`a // b`	`Math.floorDiv(a, b)`	careful: `/` truncates toward zero
`a % b` (never negative for positive b)	`Math.floorMod(a, b)`	`((a % b) + b) % b`

Python	Java	C++
`s[::-1]`	`new StringBuilder(s).reverse().toString()`	`std::string(s.rbegin(), s.rend())`
`"".join(parts)`	`String.join("", parts)` or `StringBuilder`	`std::string` with `+=`, or `std::ostringstream`
`@cache`	a `HashMap` as memo	a `std::unordered_map` or array as memo

Pitfalls When Switching From Python

- **Integer overflow.** Python integers never overflow; `int` in Java and C++ holds values up to about 2.1 billion. Sums of many values, products and prefix sums often need `long` (Java) or `long long` (C++).
- **Integer division and remainder with negatives.** In Java and C++, `-7 / 2` is `-3` and `-7 % 2` is `-1`; in Python they're `-4` and `1`. This matters in the "divisible runs" problem (8.6), where remainders must be normalized.
- **C++ heaps are max-heaps.** `std::priority_queue<int>` returns the largest item. Use `std::greater<int>` for a min-heap, or negate values, as in Python's max-heap trick.
- **Comparing objects in Java.** `==` compares references for `Integer` objects and strings. It seems to work for small integers, from -128 to 127, because Java reuses those objects, and then fails for larger values. Use `.equals`.
- **Recursion depth.** Java and C++ allow deeper recursion than Python's default limit of about 1,000, but very deep recursion can still overflow the stack. Iterative versions remain safer for large inputs.
- **Modifying a collection while iterating** over it throws `ConcurrentModificationException` in Java and invalidates iterators in C++. Python raises an error for dictionaries and sets, but not for lists, so habits from Python may not protect you.

Two Solutions, Translated

The Budget Pair (Problem 5.1) and Kth Largest Score (Problem 13.1) solutions in Java:

```
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import java.util.PriorityQueue;

public class BudgetPair {
    static int[] budgetPair(int[] prices, int budget) {
        Map<Integer, Integer> seen = new HashMap<>(); // price -> index
        for (int i = 0; i < prices.length; i++) {
            Integer j = seen.get(budget - prices[i]);
            if (j != null) {
                return new int[] {j, i};
            }
            seen.put(prices[i], i);
        }
        return new int[0];
    }

    static int kthLargest(int[] scores, int k) {
        PriorityQueue<Integer> heap = new PriorityQueue<>(); // min-heap
        for (int s : scores) {
            heap.add(s);
            if (heap.size() > k) {
                heap.poll(); // drop the smallest
            }
        }
        return heap.peek();
    }

    public static void main(String[] args) {
        int[] prices = {12, 7, 30, 18};
        System.out.println(Arrays.toString(budgetPair(prices, 25)));
        System.out.println(Arrays.toString(budgetPair(prices, 100)));
        int[] scores = {71, 88, 64, 93, 88, 50};
        System.out.println(kthLargest(scores, 3));
    }
}
```

And in C++:

```
#include <functional>
#include <iostream>
#include <queue>
#include <unordered_map>
#include <vector>

std::vector<int> budgetPair(const std::vector<int>& prices, int budget) {
    std::unordered_map<int, int> seen;  // price -> index
    for (int i = 0; i < (int)prices.size(); i++) {
        auto it = seen.find(budget - prices[i]);
        if (it != seen.end()) {
```

```
            return {it->second, i};
        }
        seen[prices[i]] = i;
    }
    return {};
}

int kthLargest(const std::vector<int>& scores, int k) {
    // std::priority_queue is a MAX-heap by default; std::greater makes
    // it a min-heap, like Python's heapq.
    std::priority_queue<int, std::vector<int>, std::greater<int>> heap;
    for (int s : scores) {
        heap.push(s);
        if ((int)heap.size() > k) {
            heap.pop();  // drop the smallest
        }
    }
    return heap.top();
}

void print(const std::vector<int>& v) {
    std::cout << "[";
    for (size_t i = 0; i < v.size(); i++) {
        std::cout << (i ? ", " : "") << v[i];
    }
    std::cout << "]\n";
}

int main() {
    std::vector<int> prices = {12, 7, 30, 18};
    print(budgetPair(prices, 25));
    print(budgetPair(prices, 100));
    std::cout << kthLargest({71, 88, 64, 93, 88, 50}, 3) << "\n";
}
```

Both programs print the same results as the Python versions:

```
[1, 3]
[]
88
```

The book's test suite compiles both programs, runs them and compares their output with these expected results, alongside the Python tests.

> **Tip:** If you will interview in Java or C++, re-solve one problem from each pattern chapter in that language. The ideas transfer immediately; fluency with the standard library takes practice.

Appendix D: Problem Index

All 120 problems in the book, with their difficulty and the chapter that teaches their pattern. Use it to pick problems for review, or to find problems of a given difficulty for a timed practice session. There are 27 easy, 77 medium, 1 medium-hard and 15 hard problems.

No.	Problem	Difficulty	Chapter
5.1	Budget Pair	Easy	Arrays and Hashing
5.2	First Repeat Visitor	Easy	Arrays and Hashing
5.3	Anagram Shelves	Medium	Arrays and Hashing
5.4	Longest Unbroken Streak	Medium	Arrays and Hashing
5.5	Product Without Me	Medium	Arrays and Hashing
5.6	Same Pattern	Easy	Arrays and Hashing
6.1	Palindrome Plate	Easy	Two Pointers
6.2	Sorted Pair	Easy	Two Pointers
6.3	Balanced Triples	Medium	Two Pointers
6.4	Widest Reservoir	Medium	Two Pointers
6.5	Merge Two Logs	Easy	Two Pointers
6.6	Trapped Rain	Hard	Two Pointers
7.1	Best Week	Easy	Sliding Window
7.2	Longest Unique Run	Medium	Sliding Window
7.3	Shortest Span	Medium	Sliding Window

No.	Problem	Difficulty	Chapter
7.4	Repaint the Fence	Medium	Sliding Window
7.5	Hidden Anagram	Medium	Sliding Window
7.6	Smallest Cover	Hard	Sliding Window
8.1	Range Totals	Easy	Prefix Sums
8.2	Exact Total Runs	Medium	Prefix Sums
8.3	Balance Point	Easy	Prefix Sums
8.4	Even Split	Medium	Prefix Sums
8.5	Region Sums	Medium	Prefix Sums
8.6	Divisible Runs	Medium	Prefix Sums
9.1	Balanced Brackets	Easy	Stacks and Queues
9.2	Minimum Stack	Medium	Stacks and Queues
9.3	Postfix Calculator	Medium	Stacks and Queues
9.4	Next Warmer Day	Medium	Stacks and Queues
9.5	Largest Banner	Hard	Stacks and Queues
9.6	Window Peaks	Hard	Stacks and Queues
10.1	Find in a Sorted Catalog	Easy	Binary Search
10.2	First Bad Build	Easy	Binary Search
10.3	Rotated Catalog	Medium	Binary Search
10.4	Truck Capacity	Medium	Binary Search
10.5	Integer Square Root	Easy	Binary Search
10.6	Median of Two Sorted Lists	Hard	Binary Search
11.1	Reverse a Linked List	Easy	Linked Lists
11.2	Detect a Loop	Easy	Linked Lists
11.3	Merge Two Sorted Linked Lists	Easy	Linked Lists
11.4	Remove the k-th Node From the End	Medium	Linked Lists
11.5	Interleave a List	Medium	Linked Lists

No.	Problem	Difficulty	Chapter
11.6	Merge k Sorted Linked Lists	Hard	Linked Lists
12.1	Tree Depth	Easy	Trees and Binary Search Trees
12.2	Mirror a Tree	Easy	Trees and Binary Search Trees
12.3	Level by Level	Medium	Trees and Binary Search Trees
12.4	Valid Search Tree	Medium	Trees and Binary Search Trees
12.5	Closest Common Manager	Medium	Trees and Binary Search Trees
12.6	Save and Restore a Tree	Hard	Trees and Binary Search Trees
12.7	Best Path	Hard	Trees and Binary Search Trees
13.1	Kth Largest Score	Easy	Heaps and Top-K
13.2	Most Common Tags	Medium	Heaps and Top-K
13.3	Closest Stations	Medium	Heaps and Top-K
13.4	Running Median	Hard	Heaps and Top-K
13.5	Cooldown Scheduler	Medium	Heaps and Top-K
13.6	No Two Alike	Medium	Heaps and Top-K
14.1	Merge Bookings	Medium	Intervals and Greedy Choices
14.2	Insert a Booking	Medium	Intervals and Greedy Choices
14.3	Rooms Needed	Medium	Intervals and Greedy Choices
14.4	Fewest Cancellations	Medium	Intervals and Greedy Choices
14.5	Reach the Last Stone	Medium	Intervals and Greedy Choices
14.6	Charging Loop	Medium	Intervals and Greedy Choices
15.1	All Bundles	Medium	Backtracking
15.2	All Orders	Medium	Backtracking
15.3	Exact Change	Medium	Backtracking
15.4	Grid Word	Medium	Backtracking
15.5	Safe Queens	Hard	Backtracking
15.6	Bracket Sets	Medium	Backtracking
16.1	Count Islands	Medium	Graphs: BFS and DFS

No.	Problem	Difficulty	Chapter
16.2	Paint Bucket	Easy	Graphs: BFS and DFS
16.3	Warehouse Route	Medium	Graphs: BFS and DFS
16.4	Spreading Mold	Medium	Graphs: BFS and DFS
16.5	Word Steps	Hard	Graphs: BFS and DFS
16.6	Two Teams	Medium	Graphs: BFS and DFS
17.1	Can I Graduate?	Medium	Advanced Graphs
17.2	Build Order	Medium	Advanced Graphs
17.3	The Extra Cable	Medium	Advanced Graphs
17.4	Signal Time	Medium	Advanced Graphs
17.5	Cheapest Trip With Few Stops	Medium	Advanced Graphs
17.6	Cheapest Wiring	Medium	Advanced Graphs
18.1	Stair Ways	Easy	Dynamic Programming I: One Dimension
18.2	Quiet Street	Medium	Dynamic Programming I: One Dimension
18.3	Fewest Coins	Medium	Dynamic Programming I: One Dimension
18.4	Longest Rising Sequence	Medium	Dynamic Programming I: One Dimension
18.5	Split Into Words	Medium	Dynamic Programming I: One Dimension
18.6	Decode Ways	Medium	Dynamic Programming I: One Dimension
19.1	Grid Paths	Medium	Dynamic Programming II: Grids, Strings and Knapsacks
19.2	Longest Shared Subsequence	Medium	Dynamic Programming II: Grids, Strings and Knapsacks
19.3	Typo Distance	Hard	Dynamic Programming II: Grids, Strings and Knapsacks
19.4	Backpack	Medium	Dynamic Programming II: Grids, Strings and Knapsacks

No.	Problem	Difficulty	Chapter
19.5	Fair Split	Medium	Dynamic Programming II: Grids, Strings and Knapsacks
19.6	Longest Mirror	Medium	Dynamic Programming II: Grids, Strings and Knapsacks
20.1	Prefix Tree	Medium	Tries and String Techniques
20.2	Autocomplete	Medium	Tries and String Techniques
20.3	Board Words	Hard	Tries and String Techniques
20.4	Shared Start	Easy	Tries and String Techniques
20.5	Pack and Unpack	Medium	Tries and String Techniques
20.6	Find Every Match	Hard	Tries and String Techniques
21.1	The Lonely Number	Easy	Bit Manipulation and Math
21.2	Count the Ones	Easy	Bit Manipulation and Math
21.3	The Missing Ticket	Easy	Bit Manipulation and Math
21.4	Power of Two	Easy	Bit Manipulation and Math
21.5	Fast Power	Medium	Bit Manipulation and Math
21.6	Primes Below n	Medium	Bit Manipulation and Math
22.1	Recently Used Cache	Medium	Designing Data Structures
22.2	Hit Counter	Medium	Designing Data Structures
22.3	Versioned Store	Medium	Designing Data Structures
22.4	Random Pick Set	Medium	Designing Data Structures
22.5	Least-Frequently Used Cache	Hard	Designing Data Structures
22.6	Flatten Iterator	Medium	Designing Data Structures
23.1	Log Summary	Medium	Practical Coding Rounds
23.2	Telemetry Spikes	Medium	Practical Coding Rounds
23.3	Launch Sequence Validator	Medium	Practical Coding Rounds
23.4	Join Two Tables	Medium	Practical Coding Rounds
23.5	Token Bucket Rate Limiter	Medium	Practical Coding Rounds

No.	Problem	Difficulty	Chapter
23.6	Config Diff	Medium	Practical Coding Rounds
23.7	Parse a Duration	Easy	Practical Coding Rounds
Mock 1	Bus Transfers	Medium	Four Full Mock Interviews
Mock 2	My Calendar	Medium	Four Full Mock Interviews
Mock 3	Snapshot Array	Medium	Four Full Mock Interviews
Mock 4	Easiest Hike	Medium-Hard	Four Full Mock Interviews

Appendix E: Glossary

Adjacency list. A graph representation that maps each node to the list of its neighbors (Chapter 16).

Amortized cost. The average cost per operation over a long sequence, even if a single operation is occasionally expensive; appending to a Python list is amortized O(1) (Chapter 3).

Backtracking. Building candidates one choice at a time, exploring each choice fully, then undoing it to try the next (Chapter 15).

Bar Raiser. At Amazon, an interviewer from outside the hiring team who helps maintain the hiring standard (Chapter 1).

Base case. The smallest input of a recursive function or DP, whose answer is known directly.

Bellman-Ford. A shortest-path algorithm that relaxes every edge in rounds; it handles negative weights and "at most k steps" limits (Chapter 17).

BFS (breadth-first search). Exploring a graph in order of distance from the start, using a queue; finds shortest paths when every step costs the same (Chapter 16).

Big-O notation. A description of how the time or memory an algorithm needs grows with the input size, ignoring constant factors (Chapter 3).

Binary search. Finding a value or boundary in a sorted range by repeatedly halving it (Chapter 10).

Binary search tree (BST). A binary tree in which every node's left subtree holds smaller values and its right subtree larger values (Chapter 12).

Bipartite graph. A graph whose nodes can be split into two groups with every edge between the groups; equivalently, two-colorable (Chapter 16).

Brute force. The straightforward solution that tries every possibility; the usual starting point in an interview (Chapter 2).

Cache. A fast store of recently or frequently used data, such as an LRU cache (Chapters 22 and 25).

CAP theorem. In a distributed store, during a network partition you must choose between consistency and availability (Chapter 25).

Dijkstra's algorithm. A shortest-path algorithm for non-negative edge weights that always expands the closest unsettled node, using a heap (Chapter 17).

DFS (depth-first search). Exploring a graph by going as deep as possible before backtracking, using recursion or a stack (Chapter 16).

Dynamic programming (DP). Solving a problem by combining answers to overlapping smaller subproblems, each computed once (Chapters 18 and 19).

Edge case. An unusual input at the boundary of what's allowed: empty, one element, all equal, the maximum size.

Eventual consistency. A guarantee that replicas of data will agree after a delay, if no new writes arrive (Chapter 25).

Greedy algorithm. An algorithm that makes the locally best choice at each step; correct only when that choice is provably safe (Chapter 14).

Hash map. A structure that maps keys to values with O(1) average lookup; Python's `dict` (Chapter 5).

Heap. A tree-shaped structure, stored in a list, that gives the smallest (or largest) item in O(1) and updates in O(log n) (Chapter 13).

Idempotent. An operation that has the same effect whether it's applied once or several times (Chapter 25).

In-order traversal. Visiting a binary tree's left subtree, then the node, then the right subtree; yields a BST's values in sorted order (Chapter 12).

Invariant. A condition that stays true throughout a loop or data structure, such as "the window never contains a repeated character".

KMP (Knuth-Morris-Pratt). A linear-time string-search algorithm that never re-reads the text (Chapter 20).

Load balancer. A component that spreads incoming requests across several servers (Chapter 25).

Loop (interview loop). The set of interviews, often four to six, that a candidate takes after passing the screening stages (Chapter 1).

LRU (least recently used). An eviction policy that removes the item unused for the longest time (Chapter 22).

Memoization. Caching the results of a function so repeated calls with the same arguments are free; `functools.cache` in Python (Chapter 18).

Minimum spanning tree. The cheapest set of edges that connects all nodes of a graph (Chapter 17).

Monotonic stack or deque. A stack or deque whose items are kept in increasing or decreasing order, used for "next greater" and sliding-window maximum problems (Chapter 9).

Online assessment (OA). A timed, automatically graded coding test taken before live interviews (Chapter 27).

Prefix sum. A running total such that the sum of any block equals the difference of two prefix sums (Chapter 8).

Pruning. Skipping branches of a search that can't lead to a valid or better solution (Chapter 15).

Rate limiter. A component that limits how many requests a client can make in a period, such as a token bucket (Chapter 23).

Recurrence. An equation that defines a problem's answer in terms of answers to smaller instances (Chapter 18).

Replication. Keeping copies of data on several machines for availability and read scaling (Chapter 25).

Sentinel node. A dummy node at the start or end of a linked list that removes special cases (Chapters 11 and 22).

Sharding. Splitting data across machines by key (Chapter 25).

Sliding window. A block of consecutive items that moves through a sequence, updated incrementally (Chapter 7).

Space complexity. How the extra memory an algorithm uses grows with the input size (Chapter 3).

STAR. Situation, Task, Action, Result: a structure for answering behavioral questions (Chapter 26).

State machine. A model with a fixed set of states and allowed transitions between them (Chapter 23).

Topological sort. An ordering of a directed graph's nodes in which every edge points forward; exists only if there are no cycles (Chapter 17).

Trie. A tree that stores strings letter by letter, so words with a common prefix share a path (Chapter 20).

Two pointers. Two indexes moving through a sequence, from both ends or in the same direction, to avoid nested loops (Chapter 6).

Union-find (disjoint set union). A structure that tracks which items are connected and merges groups in nearly O(1) (Chapter 17).

XOR. The bitwise "exclusive or": 1 where exactly one of two bits is 1; `x ^ x == 0` (Chapter 21).

About the Author

Aadhavan Muthurengan is an automation and artificial intelligence professional with more than 15 years of experience in the automotive industry.

Throughout a career focused on product development and validation, Aadhavan has designed and built a wide range of in-house tools that automate complex engineering workflows, speed up testing, and help teams deliver reliable, high-quality results. Much of that work comes down to the skills this book teaches: choosing the right data structure, writing code that handles messy real-world input, and testing it until it can be trusted.

That experience shaped this book. Every problem is original, every solution is tested, most of them against a simpler version on thousands of random inputs, and every example session in these pages was produced by running the code. The goal is a book you can trust as much as the code you'll write after reading it.

Aadhavan is also the author of *Prompt Engineering Mastery*, a beginner-to-advanced guide to getting better results from AI; *Vibe Coding for Beginners*, a guide to building real software with AI coding tools; *Building AI Agents with Claude*, a guide to agents that use tools, remember, and get real work done; *Python Programming Hands-On*, a practical, tested introduction to programming; and *How Large Language Models Are Made*, a clear explanation of how today's AI models are built.

Thank you for reading *The Coding Interview Playbook*. Good luck with your interviews: you've done the preparation, now go and show them how you think.